

Università di Genova

PhD Program in Science and Technology for Electronic and Telecommunication Engineering

Advanced AI Tools for Augmented Observability in the 5/6G Environment

Shima Afshar Borji

Thesis submitted for the degree of *Doctor of Philosophy* (38° cycle)

TO BE COMPLETED: defence date

Prof. Raffaele Bolla	Tutor
Prof. Roberto Bruschi	Co-tutor
Prof. Maurizio Valle	Head of the PhD program

Università di Genova

DITEN DIPARTIMENTO
DI INGEGNERIA NAVALE, ELETTRICA,
ELETTRONICA E DELLE TELECOMUNICAZIONI

Advanced AI Tools for Augmented Observability in the 5/6G Environment

Shima Afshar Borji

⋈

Committee:

President of the committee
TO BE COMPLETED PRESIDENT, President, Affiliation, University, Country

Reviewers
Antonella MOLINARO, Full Professor, Università Mediterranea di Reggio Calabria, Italy
Giuseppe PIRO, Full Professor, Politecnico di Bari, Italy

Tutors
Raffaele BOLLA, Full Professor, University of Genoa, Italy
Roberto BRUSCHI, DITEN, University of Genoa & CNIT, Genoa, Italy

University of Genoa

Shima Afshar Borji

Advanced AI Tools for Augmented Observability in the 5/6G Environment

xx+127 p.

Abstract

Energy-aware control of softwarised mobile networks requires more information than current telemetry alone provides. A controller must estimate the power attributable to an individual network function, anticipate traffic demand, and compare the performance of alternative configurations before selecting one. This thesis investigates how measurement-grounded artificial intelligence can supply these additional quantities through augmented observability, using the selection of User Plane Function (UPF) implementations in 5G networks as its experimental application.

The proposed pipeline connects physical profiling, cluster-level traffic forecasting, digital-twin evaluation and shared-policy reinforcement learning. Profiling of two UPF deployments provides the measurements used to fit power and performance surrogates. A cluster-first forecasting workflow converts mobile-network traffic and base-station geography into predictions at the service-area scale required by the controller. It predicts four 15-minute horizons, with the first aligned to control. A digital twin combines load inputs with the surrogates and activation-cost accounting to evaluate candidate UPF choices. Shared policies pool experience across sites, while a controlled comparison examines the additional contribution of a centralised training critic.

The results show that forecasting the aggregated service-area signal achieves 0.21–0.23 skill over persistence under the tested fixed-budget comparison, whereas summing per-station forecasts provides no improvement over persistence. Across eight training seeds, a shared-policy controller (MAPPO) outperforms independently trained per-site learners and reduces the combined cost by 10.2 % relative to the test-tuned hysteresis reference. Its advantage over Shared-PPO is modest and concerns the balance between energy and model-predicted QoS penalties; the comparison does not establish an energy advantage over Shared-PPO. Re-evaluation places MAPPO first by reward among six controllers in eight of ten alternative surrogate calibrations, while boundary-focused validation identifies uncertainty in the predicted safe operating region.

These findings demonstrate the utility of an integrated augmented-observability pipeline within the studied modelled conditions. They also identify the measurements needed to assess transfer to physical operation. The resulting contribution is a measurement-grounded framework for connecting inferred network state to control decisions, with a clear separation between performance in a calibrated twin and validation of a live deployment. The 5G study provides a basis for investigating this approach in future 6G networks.

Keywords: augmented observability; user plane function; energy-aware control; network digital twin; traffic forecasting; shared-policy reinforcement learning; 5G core networks.

Riassunto

Il controllo energeticamente efficiente delle reti mobili basate su funzioni software richiede informazioni ulteriori rispetto a quelle fornite dalla sola telemetria. Un controllore deve stimare la potenza attribuibile a una singola funzione di rete, anticipare la domanda di traffico e confrontare le prestazioni di configurazioni alternative prima di sceglierne una. Questa tesi studia come l'intelligenza artificiale ancorata alle misure possa fornire tali informazioni attraverso l'osservabilità aumentata, utilizzando la selezione di implementazioni della User Plane Function (UPF) nelle reti 5G come applicazione sperimentale.

La pipeline proposta collega profilazione fisica, previsione del traffico per cluster, valutazione mediante digital twin e apprendimento per rinforzo con politiche condivise. La profilazione di due deployment UPF fornisce le misure per costruire modelli surrogati di potenza e prestazioni. La previsione cluster-first trasforma il traffico della rete mobile e la geografia delle stazioni radio in previsioni alla scala delle aree di servizio richiesta dal controllore. Sono previsti quattro orizzonti di 15 minuti, il primo dei quali è allineato al controllo. Un digital twin combina il carico con i surrogati e la contabilizzazione dei costi di attivazione per valutare le alternative UPF. Le politiche condivise aggregano l'esperienza dei siti, mentre un confronto controllato esamina il contributo aggiuntivo di un critico centralizzato durante l'addestramento.

I risultati mostrano che prevedere il segnale aggregato dell'area di servizio raggiunge uno skill di 0.21–0.23 rispetto alla persistenza nel confronto a budget fissato, mentre sommare le previsioni delle singole stazioni non supera il riferimento di persistenza. Su otto seed di addestramento, un controllore a politica condivisa (MAPPO) supera gli agenti indipendenti addestrati per sito e riduce il costo combinato del 10.2 % rispetto al riferimento a isteresi ottimizzato sul test. Il vantaggio rispetto a Shared-PPO è modesto e riguarda il bilanciamento tra energia e penalità di QoS previste dal modello; il confronto non dimostra un vantaggio energetico rispetto a Shared-PPO. La rivalutazione colloca MAPPO al primo posto per ricompensa tra sei controllori in otto delle dieci calibrazioni alternative dei surrogati. La validazione vicino alla soglia di QoS evidenzia incertezza nella regione operativa sicura prevista dal modello.

Questi risultati dimostrano l'utilità della pipeline integrata nelle condizioni modellate e identificano le misure necessarie per valutarne il trasferimento al funzionamento fisico. Il contributo è un quadro metodologico ancorato alle misure che collega lo stato di rete inferito alle decisioni di controllo, distinguendo le prestazioni nel twin calibrato dalla validazione di un deployment reale. Lo studio 5G fornisce una base per investigare questo approccio nelle future reti 6G.

Parole chiave: osservabilità aumentata; user plane function; controllo energeticamente efficiente; digital twin di rete; previsione del traffico; apprendimento per rinforzo con politiche condivise; reti core 5G.

Contents

List of Figures ix

List of Tables xiv

List of Algorithms xvii

Notations xix

Acronyms xxi

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Augmented observability: the central idea . . . 1
- 1.3 Problem statement and scope . . . 3
- 1.4 Research questions . . . 4
- 1.5 Contributions . . . 5
- 1.6 Thesis outline . . . 5
- 1.7 Code and data availability . . . 6

2 Related Work and Positioning 7
- 2.1 The measurement premise . . . 7
- 2.2 UPF-level metric attribution . . . 8
- 2.3 Counterfactual digital-twin evaluation . . . 8
- 2.4 Spatiotemporal mobile traffic forecasting . . . 9
- 2.5 Energy-aware control of network functions . . . 10
- 2.6 Standards alignment and terminology . . . 10
- 2.7 What counts as validation . . . 11
- 2.8 Positioning . . . 12

3 Energy Profiling of 5G User Plane Functions 13
- 3.1 Introduction . . . 13
- 3.2 Measurement Methodology . . . 14
  - 3.2.1 Testbed Configuration . . . 14
  - 3.2.2 Dataset . . . 15
  - 3.2.3 Metrics . . . 15
- 3.3 Variant Profiles . . . 16
  - 3.3.1 Overview . . . 16
  - 3.3.2 SD-CORE DPDK . . . 17

3.3.3 USR-UPF . . . 18
3.4 USR-UPF Saturation Analysis . . . 19
3.4.1 Operating Points: Definitions . . . 19
3.4.2 Saturation Onset . . . 20
3.4.3 Power Behaviour at Saturation . . . 21
3.4.4 Per-Run Characterisation . . . 22
3.5 Specific Energy Consumption Analysis . . . 23
3.5.1 Definitions . . . 23
3.5.2 The Poll-Mode Measurement Paradox . . . 23
3.5.3 Implications for Energy Metric Selection . . . 24
3.6 Feature Correlation Analysis . . . 25
3.6.1 Correlates of Power Consumption . . . 25
3.6.2 Temporal Dynamics . . . 26
3.7 Discussion and Implications . . . 26
3.7.1 Architectural Trade-offs . . . 26
3.7.2 Empirical Operating Regions . . . 27
3.7.3 Implications for Predictive Energy Modelling . . . 28
3.8 Variant Performance Summary . . . 28
3.9 Surrogate Models for the Digital Twin . . . 30
3.9.1 Training Strategy . . . 31
3.9.2 Deployed models and scope of validation . . . 33
3.9.3 Model Performance . . . 34
3.10 NetMob 2023 Compatibility . . . 34
3.11 Summary . . . 37

4 The Spatiotemporal Traffic Forecaster 39
4.1 Introduction . . . 39
4.2 Data and Spatial Substrate . . . 40
4.2.1 Dataset and normalisation . . . 40
4.2.2 From tiles to base stations: Voronoi attribution . . . 41
4.2.3 The base-station graph . . . 42
4.2.4 Traffic characterisation . . . 43
4.3 Service-Area Clustering . . . 44
4.3.1 Requirements and method . . . 45
4.3.2 Resulting service areas . . . 46
4.3.3 Coarsening to the forecasting graph . . . 47
4.4 Forecasting Models . . . 48
4.5 Outputs and the Twin Data Contract . . . 49
4.6 Evaluation Methodology . . . 50
4.7 Forecaster Benchmark . . . 51
4.8 Anatomy of the Forecast Error . . . 53
4.9 Effect of Cluster Count K . . . 55
4.10 Generalisation Across Traffic Types . . . 56

4.11 Cluster-First versus Per-Station Forecasting . . . 57
4.12 Geographic versus Random Aggregation . . . 59
4.13 From Forecast Error to Decision Error . . . 61
4.14 Summary . . . 63

5 The UPF Digital Twin 65
5.1 Introduction . . . 65
5.2 From Profiling to a Predictive Surrogate . . . 66
5.2.1 Two-layer structure . . . 66
5.2.2 Which variants the twin uses . . . 67
5.2.3 Full and lite feature sets . . . 67
5.3 Twin Architecture . . . 68
5.3.1 Interactive front-end . . . 68
5.4 Inputs and Load Calibration . . . 70
5.4.1 Normalised load to physical load . . . 70
5.4.2 Lite input adapter . . . 71
5.5 Predicted KPIs and Safety Semantics . . . 71
5.6 Switching-Cost Model . . . 73
5.6.1 Activation duration and the energy-spike rule . . . 74
5.6.2 Within-step accounting . . . 75
5.7 Operating Thresholds Derived from the Surrogate . . . 76
5.7.1 Validating the boundary, not the fit . . . 77
5.7.2 Anti-flapping band . . . 78
5.8 Evaluation Methodology and Demonstration . . . 80
5.8.1 Rollout engine . . . 80
5.8.2 Metrics . . . 81
5.8.3 Controllers used in this chapter . . . 82
5.8.4 Headroom for energy-aware switching . . . 82
5.8.5 The twin under a hysteresis controller . . . 83
5.8.6 Aggregate behaviour . . . 84
5.9 Sensitivity to the Load Calibration . . . 85
5.10 Summary . . . 87

6 Learning Controllers for Energy-Aware UPF Orchestration 89
6.1 Introduction . . . 89
6.2 Starting point: a single-site prototype . . . 90
6.3 Problem formulation and rule-based baselines . . . 91
6.4 Single-site reinforcement learning . . . 94
6.5 Explicit versus Recurrent Temporal Information . . . 98
6.6 Limitations of independent per-site learning . . . 99
6.7 Cooperative multi-agent control (MAPPO) . . . 100
6.8 Sensitivity to Fleet Size . . . 105
6.8.1 Load-calibration sweep . . . 108

6.9 Additional runs: shared actor with local or centralised critic 108
6.9.1 Shared actor, local or centralised critic 110
6.9.2 Sensitivity to surrogate calibration 111
6.9.3 Sensitivity to reward weights 112
6.10 Synthesis: what the control results do and do not establish 113
6.11 Summary 115

7 General conclusion **116**
7.1 Synthesis 116
7.2 Answers to the research questions 116
7.3 Limits and future work 118
7.4 Concluding perspective 119

Acknowledgements **120**

Publication record **121**

Bibliography **122**

List of Figures

1.1 System architecture of the augmented-observability pipeline. 2

3.1 Physical testbed architecture for UPF profiling. 16
3.2 Power consumption versus throughput for each UPF variant. SD-CORE DPDK (left) exhibits a near-constant power profile due to continuous CPU polling. USR-UPF (right) shows a strong load-proportional relationship but saturates above 0.5 Gbps. 18
3.3 Load-stratified variant profiles. Throughput bins on the x-axis (Gbps). (a) Power consumption; (b) CPU utilisation; (c) packet loss incidence; (d) mean downlink delay. Error bars in (a) denote ±1 standard deviation. DPDK maintains flat power and zero loss at all loads; USR scales with load but saturates above λ_{env}. 19
3.4 USR-UPF saturation transition in 50 Mbps throughput bins. The dashed vertical line marks the saturation onset $\lambda_{\text{onset}} \approx 0.17$ Gbps (Table 3.2). (a) Fraction of samples with packet loss; (b) mean number of packets lost per interval (when loss occurs); (c) mean downlink delay; (d) mean power and CPU utilisation (dual axis). 21
3.5 USR-UPF per-run saturation summary. Each point represents one measurement run. (a) Overall packet loss versus maximum offered throughput, coloured by mean power; (b) mean power versus maximum offered throughput, coloured by packet loss fraction. 22
3.6 Specific Energy Consumption versus throughput. (a) $\text{SEC}_{\text{total}}$: DPDK appears efficient due to low power denominator relative to flat numerator. (b) SEC_{net}: SD-CORE DPDK's SEC_{net} collapses to near zero because its incremental power $P_{\text{net}} \approx 0$. Both metrics understate the operational energy cost of running a poll-mode UPF. Extreme low-throughput outliers are clipped to the 98th percentile for clarity. 24
3.7 Pearson correlation heatmaps for a subset of key features, computed separately for each variant. The DPDK heatmap (left) shows uniformly low correlations with power, consistent with the near-constant power profile. The USR heatmap (right) reveals strong positive correlations between throughput, CPU, and power. 26
3.8 Temporal power and throughput traces for representative high-load runs. SD-CORE DPDK (top): power remains flat despite throughput variations. USR-UPF (bottom): power rises with load until the saturation plateau. . . . 27

3.9 Measured KPI distributions for each deployment variant. Each violin combines a kernel-density estimate with the observed range; horizontal bars mark the median. Points are jittered for visibility. DPDK power exhibits near-constant behaviour (poll-mode driver draws fixed power regardless of load), explaining its low model R^2. USR (restricted range, `usr_safe`) loss and delay have lower variance than their full-range counterparts, which limits their model predictability compared with the full-range USR variant. 29
3.10 Predicted versus measured power for the three trained variants (SD-CORE DPDK, USR-UPF full range, USR-UPF restricted training range). Solid markers: full model; open markers: lite (NetMob-compatible) model. The dashed line is the identity. 30
3.11 R^2 for all prediction slots (variant × target), full model (left) and lite model (right). Green = high accuracy; red = low. DPDK CPU and DPDK delay are structurally unpredictable (poll-mode artefacts). 35
3.12 Algorithm selection for the L2 power model. Each group shows Ridge, Random Forest, and Gradient Boosting test R^2 for the full and lite feature sets. Filled bars and gold stars (★) mark the selected winner per group; transparent bars are runners-up. Ridge wins for `usr_safe` (linear regime); Random Forest wins for `usr_full` (non-linear saturation effects). 35
3.13 Algorithm comparison for all L1 prediction slots. Rows: prediction targets; columns: algorithm types. Gold borders mark the winning algorithm per target row. Ridge dominates the linear throughput–power relationship; tree models offer marginal gains only for CPU and delay targets. 36
3.14 L2 power model interpretability for each variant. *Left panels*: Ridge regression normalised absolute coefficients. *Right panels*: feature importances from the best tree-based model (Random Forest or Gradient Boosting). Both panels share the same feature ordering (sorted by tree importance) to allow direct comparison regardless of which algorithm was selected (marked ★). The dominance of L1-predicted CPU and throughput confirms the causal pathway load → CPU → power. 37

4.1 Spatial substrate. (a) Voronoi coverage cells of the 965 base stations, clipped to the Lyon footprint and shaded by mean Netflix load density (logarithmic scale). (b) The base-station adjacency graph: two stations are connected when their cells share a border (2 772 edges). 42
4.2 Statistics of the spatial substrate: (a) Voronoi cell area (logarithmic axis), (b) node degree, and (c) edge length of the base-station adjacency graph. Dashed lines mark medians. 43

4.3 Traffic characterisation. (a) Mean diurnal profile of the city-wide load per service, relative to the service mean (solid: weekdays, dashed: weekends). (b) Empirical distribution over base stations of the share of slots with $\hat{d} < 0.01$. (c) Autocorrelation of the Netflix signal at three aggregation levels. (d) Pearson correlation between the Netflix series of two base stations as a function of their hop distance in the adjacency graph (median and inter-quartile range). 44
4.4 SKATER service areas for $K = 5$, 10 and 20. Thin white lines are Voronoi cells, dark lines service-area boundaries; numbers are cluster identifiers. Colours only separate neighbouring areas. The partitions are nested: each area at a larger K lies inside one area at a smaller K. 46
4.5 The K=10 partition used as the reference configuration. (a) Service areas with the coarse forecasting graph overlaid; node area is proportional to mean cluster load, and an edge joins two clusters that share a border. (b) Number of base stations and (c) mean and 99th-percentile Netflix load per cluster, ordered by decreasing mean load. 48
4.6 Cluster-first forecasting pipeline. 48
4.7 Spatial-first GAT–GRU forecaster. 50
4.8 Skill over persistence of the six learned forecasters at K=10 (left) and K=50 (right); error bars are ±1 seed std. The whole field spans only ∼0.06 skill: the graph models lead the graph-free GRU by a small margin, and even a linear model is close behind. 52
4.9 Observed and forecast load, 60 minutes ahead, over five days of the test window for the largest, a mid-sized and the smallest-load cluster of the K=10 partition (Graph WaveNet, one seed). The forecast tracks the diurnal envelope at every scale; short spikes, increasingly prominent as the cluster shrinks, are not anticipated. 53
4.10 Breakdown of the Graph WaveNet test error at K=10 (Netflix; mean over three seeds, error bars ±1 std). (a) Skill over persistence per cluster, clusters ordered by decreasing mean load. (b) WAPE by lead time, against both naive baselines. (c) WAPE by quartile of the realised load, computed per cluster. 54
4.11 Graph WaveNet preserves its skill over the naive baselines from 4 to 100 clusters even as the raw error grows with the intrinsically harder small-cluster signal. The dashed red curve (raw WAPE, right axis) rises while the solid skill curves (left axis) stay flat. 56
4.12 Graph WaveNet's advantage over both baselines holds for aggregate and sparse traffic, scaling with the predictability of each service. The seasonal-naive panel (right) shows DailyMotion's skill rising to ∼0.37, above Netflix, despite its sparsity. 57

4.13 Forecasting the aggregated cluster signal directly (cluster-first) versus forecasting 965 base stations and summing (per-base-station), scored on the same K=10 target. The per-station route collapses to persistence-level skill (≈ 0); cluster-first reaches 0.21–0.23. 58
4.14 Partition control with Graph WaveNet at K=10 and K=50 (Netflix). Bars are medians and dots individual training runs. (a) Raw test WAPE, with the persistence WAPE of the same target as a black tick. (b) Skill over persistence. (c) Mean pairwise correlation between the K cluster signals. Random groupings of the same sizes are easier to forecast and slightly *more* skilful, but their signals are nearly interchangeable. 60

5.1 Digital-twin architecture and environment boundary. 69
5.2 Twin-predicted UPF responses versus offered load (lite surrogate). (a) Power: SD-CORE DPDK flat near 0.82 W, USR-UPF rising and crossing SD-CORE DPDK at low load; (b) CPU; (c) downlink delay (log scale) with the 200 μs QoS budget marked; (d) predicted packet loss (symmetric-log scale). The USR-UPF staircase reflects the piecewise-constant nature of the tree-based lite models. 74
5.3 Within-step switching and activation-time detail. 76
5.4 Operating thresholds derived from the surrogate. USR-UPF power (red) crosses SD-CORE DPDK power (blue) at the energy break-even (91 Mbps); the QoS limit (149 Mbps) is where USR-UPF first becomes unsafe. The decision threshold $\lambda_{\uparrow}$ = 81 Mbps is the binding (energy break-even) limit minus a 10 Mbps margin; the shaded band down to $\lambda_{\downarrow}$ = 61 Mbps is the hysteresis region. The green region marks loads where USR-UPF is both safe and cheaper than SD-CORE DPDK. 80
5.5 Distribution of calibrated offered load across all clusters and time windows. Only about 14 % of cluster-steps lie below the derived decision threshold (dashed), i.e. in the regime where USR-UPF is the safe and cheaper action; the long right tail extends well beyond the USR-UPF onset of saturation λ_{onset}, where SD-CORE DPDK is the only viable action. 83
5.6 A representative window of the hysteresis controller driving the twin (one cluster). Upper: actual and predicted load with the switch points $\lambda_{\uparrow}$, $\lambda_{\downarrow}$; shaded spans mark USR-UPF-served steps. Lower: the twin's predicted power, near the SD-CORE DPDK floor when SD-CORE DPDK serves and low when USR-UPF serves. 84
5.7 Aggregate comparison of the non-learning controllers. (a) Total energy (with saving vs. static SD-CORE DPDK annotated); (b) unsafe USR-UPF rate; (c) USR-UPF usage. Static USR-UPF is both the most expensive and by far the least safe option in this regime, since it runs a saturated data path; hysteresis achieves a modest saving while keeping the unsafe rate low. . . 86

6.1 Controller–environment interaction loop. 95

6.2 Single-site PPO against static and threshold baselines on the held-out test period for a low-load cluster. The learner improves total reward and safety relative to the threshold policy at similar energy, and reduces energy relative to static DPDK (cluster 0). 96
6.3 Per-cluster, per-step action of the centralised PPO on the test period (green = SD-CORE DPDK, red = USR-UPF). The solid red band (cluster c6 in this seed) is a low-load cluster collapsed to near-always-USR-UPF: the monolithic joint policy could not factor the per-site substructure at this training budget. 100
6.4 Shared actor with centralised or local training critic. 101
6.5 Test-period reward, mean over eight training seeds, with standard deviation. MAPPO is the best controller and the most stable trained one. 101
6.6 Per-cluster test reward, independent single-site learners (IPPO ensemble) versus cooperative MAPPO, mean over eight seeds. Shaded clusters are where MAPPO exceeds the ensemble by more than 15 reward units, identifying sites with the largest observed benefit from the shared-policy design. 104
6.7 MAPPO per-site, per-step actions on the test period. The low-load clusters show the expected pattern of selecting USR-UPF at low predicted load, which the collapsed centralised policy (Figure 6.3) did not learn. 104
6.8 Safety versus reward on the test period. Top-left is ideal. Cooperative and independent learners (eight seeds each) cluster there; the centralised policy spreads along the unsafe axis. 105
6.9 Additional runs. Left: reward mean with sample SD; right: attributed energy and model-predicted QoS violations. Deterministic baselines have no training-seed error bar; "Hysteresis" is the 50 Mbps band. 109
6.10 Energy–QoS behaviour of the two shared policies (MAPPO and Shared-PPO). (a) USR-UPF use by offered-load band, with individual seeds and means. (b) Contributions of the QoS, SEC and combined activation/-cooldown terms to MAPPO's net reward gain over Shared-PPO. (c) Median reward gain per cluster, ordered by mean load, with whiskers spanning the middle 50 % of seeds; orange marks negative medians. 111
6.11 Sensitivity to surrogate calibration across five omitted sweeps (S0–S4) and five omitted load groups (L0–L4). (a) MAPPO's mean reward advantage over each controller. (b) Predicted onset of USR-UPF QoS violations, expressed as offered load; the dotted line is the reference 149 Mbps. (c) MAPPO actions that differ from the reference calibration. 112
6.12 Ranking sensitivity to the reward weights, using the saved decisions of all six controllers. Each curve sorts the 27 QoS, switching and cooldown weight combinations for one SEC multiplier. All 81 margins are positive. 113

List of Tables

1.1 Roles of the four modules in the augmented-observability pipeline. 2

2.1 Where each pipeline stage stands against the prior art, and the claim the evidence supports. 12

3.1 UPF variant profiles: key performance and energy metrics. 17
3.2 Measured USR-UPF operating points. All are offered downlink load in Gbps. λ_{qos} and λ_{be} are *derived* from the surrogate in Chapter 5 rather than measured here, and are listed only to keep the vocabulary in one place. . . 20
3.3 Specific Energy Consumption at high throughput (> 1 Gbps): median values. 23
3.4 Top feature correlations with power consumption (pooled dataset). Rolling features are marked † to indicate data-leakage risk when used as predictors. 25
3.5 Layer 2 power metrics of the deployed models (row-level split; model family selected on that test split). Grouped and held-out-sweep scores are reported in Sections 3.9.2 and 5.7.1. 34

4.1 Structure of the SKATER partition as a function of K (Netflix load). Radius is the mean distance of a station to its cluster centroid; load CV is the coefficient of variation of mean cluster load; "largest" columns give the share of all base stations and of all load held by the largest cluster; $\bar{\rho}_{in}$ is the mean pairwise correlation between stations of the same cluster (0.24 for a random partition with the same cluster sizes, at every K). . . . 47
4.2 Model comparison at K=10 and K=50 (Netflix, seed-averaged over 3 seeds). Skill is over persistence (higher is better); WAPE in %. Best skill per column in bold; training time ranges over the two graph sizes and seeds. All models are compact reimplementations under a common protocol. 51
4.3 Graph WaveNet forecast quality vs. cluster count K (Netflix, seed-averaged). Raw WAPE rises and MAE falls with K, both intrinsic effects of shrinking clusters, while *skill* over the baselines stays flat. 55
4.4 Graph WaveNet skill over persistence by traffic type (seed-averaged). Skill is positive for every service, and the margin tracks predictability: largest on smooth aggregate traffic and smallest on sparse DailyMotion. 56
4.5 Partition control (Netflix). Structure: number of contiguous clusters, mean station-to-centroid radius, mean correlation between stations of a cluster ($\bar{\rho}_{in}$) and between cluster signals ($\bar{\rho}_{btw}$). Forecast: WAPE in %, skill over persistence; medians over runs (3 seeds; 3×3 for the random partitions). 60

5.1 Held-out-sweep validation of lite surrogates with nested run-grouped selection. Whole-range fit does not measure accuracy at a QoS decision boundary. 77
5.2 USR safety classification under nested LOSO. False-safe is the share of measured-unsafe samples predicted safe; false-unsafe is the share of measured-safe samples predicted unsafe. These denominators differ from the probability of a violation conditional on a predicted-safe verdict. . . . 78
5.3 Aggregate metrics over 10,090 cluster-steps (non-learning controllers; learned controllers are deferred to Chapter 6). Energy includes switching spikes. The informed oracle sees the actual load at each step and is reported as a clairvoyant reference, not as a deployable controller or a proven optimum. *Unsafe* USR-UPF is the fraction of USR-UPF-selected steps that violated QoS, so it is zero by construction for a controller that never selects USR-UPF; it is therefore not comparable with the all-steps violation rate reported for the learned controllers in Chapter 6. 85
5.4 Sensitivity of the energy result to the load calibration α. Savings are against static SD-CORE DPDK. *Capture* is the hysteresis saving as a fraction of the clairvoyant-reference saving. The row $\alpha = 1.0$ is the configuration used throughout this thesis. 86

6.1 Single-site PPO against static SD-CORE DPDK on every cluster, mean over four seeds. Energy saving is against static SD-CORE DPDK on the same cluster. Cluster 0 is omitted: it is the cluster reported above, at eight seeds rather than four, and remains the most favourable of the ten (14.5 % saving). †Static SD-CORE DPDK on cluster 9 violates on 3.67 % of steps, so almost all of that figure is shared with the static policy and reflects the model-evaluated DPDK safety response discussed in Section 5.5 rather than a consequence of the policy. 97
6.2 Test-period comparison across controllers (mean ± population standard deviation over seeds where trained). 102
6.3 MAPPO advantage over hysteresis as a function of fleet size, at constant per-cluster load ($\alpha/K = 0.05$). Reward is given per site per step (total reward divided by K and by the 1009 evaluation steps), because the total scales with fleet size and a growing total would say nothing on its own; the total margin is retained alongside for reference. MAPPO is the mean over eight seeds with the seed standard deviation shown; the classical controllers are deterministic. *Note that no row reproduces Table 6.2*, which is computed at $\alpha = 1.0$, $K = 10$, i.e. at $\alpha/K = 0.1$, twice the per-cluster load used here. The $K = 10$ row below sits at $\alpha = 0.5$. 106
6.4 MAPPO load-calibration sweep at $K = 10$: mean reward ± sample SD, deterministic classical rewards and mean MAPPO model-predicted QoS violations. 108

6.5 Results of the additional runs: reward mean ± sample SD and median over eight seeds (all seeds retained), mean attributed energy and model-predicted QoS-violation rate. Classical policies are deterministic. 109

List of Algorithms

1 Hysteresis controller with cooldown, per cluster. Thresholds $\lambda_{\uparrow}$, $\lambda_{\downarrow}$ are derived from the surrogate (Section 5.7); $\hat{\lambda}_t$ is the one-step load forecast. State is per cluster: the previous action and a cooldown counter. 79

2 Rollout engine. Evaluates one controller over the full horizon. The controller sees only the *forecast*; the twin is always evaluated at the *actual* load, so that forecast error is reflected in the evaluated outcome. 81

3 MAPPO with centralised training and decentralised execution. One actor is shared by all K sites, so experience gathered anywhere updates the policy used everywhere; this parameter sharing is the mechanism behind the spatial-observability claim. The critic sees the joint state and is used only during training. 94

Notations

Load and calibration	
$\hat{d}$	normalised per-cluster load emitted by the forecaster; the sum of per-base-station normalised demand over a cluster, so bounded by cluster size rather than by unity
α	load calibration constant, in Gbps per normalised unit; $\lambda_{\text{Gbps}} = \alpha\,\hat{d}$
λ	offered downlink load, in Gbps
K	number of clusters (sites) in the deployed partition
H	forecast horizon, in control steps
L	input sequence length of the forecaster
Measured operating points (Chapter 3)	
λ_{loss}	lowest per-run maximum offered load at which any loss was observed
λ_{safe}	empirical low-load reference for USR-UPF; not a guaranteed safe limit
λ_{onset}	saturation onset: first bin with more than 5 % of samples showing loss
λ_{env}	modelling envelope; upper bound of the `usr_safe` training set
λ_{50}	load at which half of all samples exhibit loss
Derived operating points (Chapter 5)	
λ_{be}	energy break-even load: USR-UPF and SD-CORE DPDK draw equal power
λ_{qos}	QoS limit: highest load at which USR-UPF still meets the budget
λ_{dec}	decision threshold, $\min(\lambda_{\text{be}}, \lambda_{\text{qos}})$ less a safety margin
$D_{\max}$	downlink delay budget
$L_{\max}$	tolerated predicted packet loss per interval
Power and energy	
P_{total}	power attributed to the UPF process under load
P_{idle}	the same quantity with no traffic offered
P_{net}	incremental power above idle, $P_{\text{total}} - P_{\text{idle}}$
k_v	burst ratio of variant v: activation power divided by source idle power
$t_{\text{act},v}$	activation duration of variant v, in seconds
$E_{\text{spike},v}$	energy attributed to one activation of variant v
Control	
τ	QoS compliance target
λ_{QoS}	QoS penalty weight in the reward
λ_{sw}	switching-energy weight in the reward
Sets	
$\mathbb{R}$	set of real numbers
$\mathbb{Z}$	set of integer numbers
$\emptyset$	empty set

Acronyms

AGCRN	adaptive graph convolutional recurrent network
AI	artificial intelligence
BESS	Berkeley extensible software switch
CTDE	centralised training with decentralised execution
DCRNN	diffusion convolutional recurrent neural network
DL	downlink
DPDK	Data Plane Development Kit
GAT	graph attention network
GRU	gated recurrent unit
GTP-U	GPRS tunnelling protocol, user plane
IDAF	infrastructure data analytics function
IPPO	independent proximal policy optimisation
KPI	key performance indicator
LSTM	long short-term memory
MAE	mean absolute error
MAPE	mean absolute percentage error
MAPPO	multi-agent proximal policy optimisation
MARL	multi-agent reinforcement learning
MDAF	management data analytics function
MDP	Markov decision process
MEC	multi-access edge computing
NAPI	new API (Linux interrupt-mitigation receive path)
NDT	network digital twin
NFV	network functions virtualisation
NWDAF	network data analytics function
OAI	OpenAirInterface
PPO	proximal policy optimisation
QoS	Quality of Service
RAN	radio access network
RAPL	running average power limit
RL	reinforcement learning
RMSE	root mean square error
SEC	specific energy consumption
SKATER	spatial K'luster analysis by tree edge removal
STGNN	spatio-temporal graph neural network
STIET	Science and Technology for Electronic and Telecommunication Engineering
UL	uplink
UPF	user plane function
VNF	virtualised network function
USR	user-space realisation (of the UPF data plane)
WAPE	weighted absolute percentage error
XDP	express data path
ZSM	zero-touch network and service management

CHAPTER 1

Introduction

1.1 Motivation

Softwarised mobile networks expose an increasing number of configuration choices, including the implementation and placement of the functions that forward user traffic. Choosing among these alternatives requires an estimate of their energy and service consequences under changing demand. For the User Plane Function (UPF), a lower-power implementation may be attractive at light load but unable to sustain the same operating range as a higher-throughput implementation. The useful choice therefore depends on both the condition of the network and the consequences of changing configuration.

Telemetry provides only part of the information needed for this decision. Hardware energy counters describe processor domains rather than individual network functions [1]; function-level energy accounting therefore requires attribution, consistent with the estimation approach described in 3GPP performance measurements [2]. Packet loss and delay can be measured for an active deployment, but those measurements do not reveal the performance an inactive alternative would provide under the same load. Future demand is also unavailable when a decision is taken. These gaps motivate models that complement telemetry with attributed, predictive and counterfactual information.

1.2 Augmented observability: the central idea

In this thesis, *augmented observability* denotes the construction of measurement-grounded estimates that extend the information available to a network controller. The term is used operationally: it concerns which quantities can be estimated, how reliably they can be estimated, and whether they support useful decisions. The work evaluates these properties empirically; it does not claim a formal state-observability result for the network dynamics.

The central research question is whether these estimates can be connected into a consistent measurement-to-control pipeline for energy-aware UPF selection, and what evidence

Table 1.1: Roles of the four modules in the augmented-observability pipeline.

Module	Method	Role in the decision process
Profiling (Ch. 3)	Measurement and surrogate fitting	models of attributed power and performance
Forecaster (Ch. 4)	Cluster-level prediction	future service-area demand; first horizon aligned to control
Digital twin (Ch. 5)	Candidate-realisation evaluation	predicted outcomes, including transition costs
Controllers (Ch. 6)	Policy learning and comparison	selection using the inferred signals; experience pooled across sites

is needed to assess that pipeline. Four modules address this question: profiling and surrogate construction, service-area traffic forecasting, digital-twin evaluation, and multi-site control (Figure 1.1, Table 1.1).

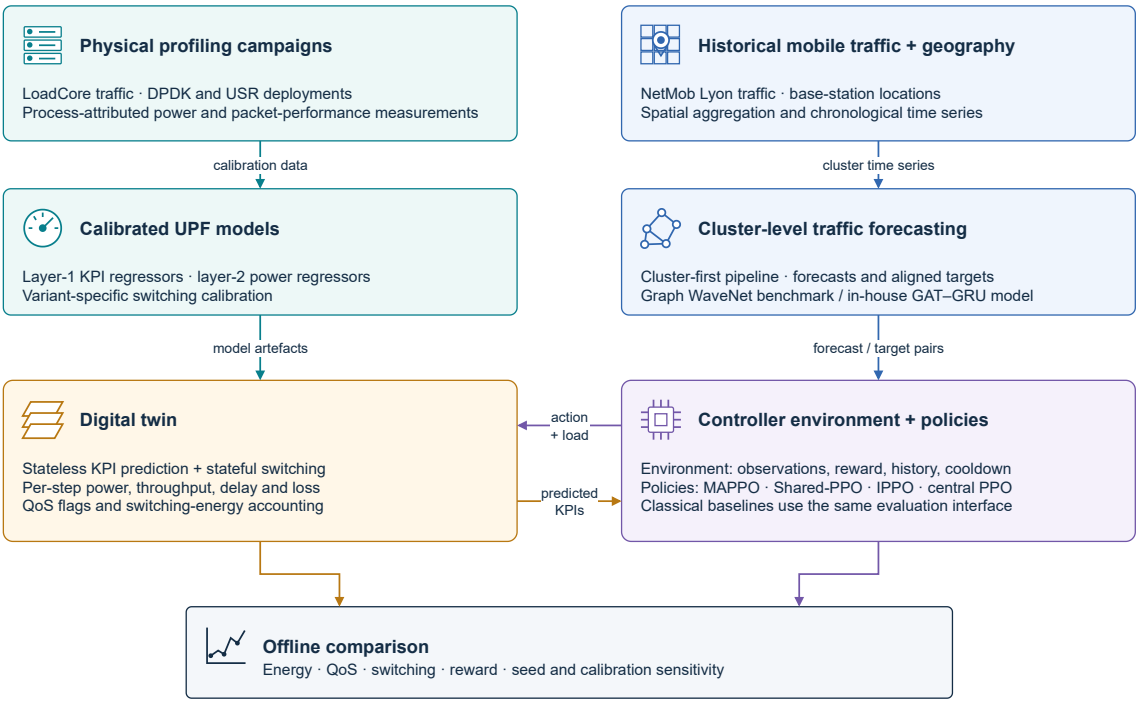


Figure 1.1: System architecture of the augmented-observability pipeline. Physical UPF profiling provides per-mode performance, power and switching models. Independently, NetMob traffic and base-station geography provide the cluster-level forecasting inputs. The digital twin predicts UPF behaviour, while the controller environment assembles observations, rewards and rollout state. Rule-based control, centralised PPO, IPPO, Shared-PPO and MAPPO are compared through this common offline interface.

Profiling characterises the recorded deployments and provides the measurements used to fit attributed-power and performance models. Forecasting supplies service-area demand for four 15-minute horizons, with the first aligned to the controller's decision interval. The digital twin evaluates candidate UPF choices by combining load inputs with the fitted models and transition accounting. These modules organise or infer information for a decision.

The controller module evaluates the use of that information and compares learning designs. Policy sharing pools experience during training: sites use the same fitted actor, while each decision depends on the local observation. Independent and shared actors receive the same fourteen local quantities. The experiments examine performance and variation across sites, and a shared-actor comparison tests local and centralised training critics. Establishing transfer specifically from data-rich to data-poor sites would require a separate experiment that controls the available training data.

The architecture is relevant to management-domain analytics, network-data analytics, digital twins and closed-loop orchestration [3, 4, 5, 6]. These functions provide the context discussed in Chapter 2; the thesis evaluates the measurement and decision methods rather than claiming a deployed standards integration.

1.3 Problem statement and scope

The empirical case is energy-aware UPF realisation selection. At each site and each 15-minute interval, a controller selects a SD-CORE DPDK-based kernel-bypass data plane or a user-space realisation (USR-UPF). The recorded deployments exhibit different power and performance characteristics: the former has a near-constant attributed-power floor and supports higher load, while the latter offers lower attributed power at light load within a narrower safe operating region. Selection must balance energy, QoS penalties and switching costs under imperfect demand forecasts.

The evidence combines physical measurements from the two 5G UPF deployments with traffic derived from the Lyon traces. Controller evaluation is offline against a measurement-calibrated digital twin. The comparison concerns complete deployments: the saved provenance does not establish identical hardware or isolate the effect of kernel bypass from the other implementation differences. The trace-to-load calibration is an explicit scenario assumption.

The title's 5G/6G setting motivates analytics-assisted control in evolving mobile networks. The experimental claims concern the evaluated 5G case; application to a future 6G deployment remains a direction for investigation. The scope is the observability pipeline and the decision logic it supports, with physical closed-loop evaluation required to assess deployment fidelity.

The associated conference paper on sustainable multimedia delivery [7] develops a broader 6G edge architecture in which digital twins support UPF orchestration, routing and compute-resource decisions. It provides context for the present study's focused evaluation of measurement-grounded UPF realisation selection.

1.4 Research questions

Five research questions connect the component studies to the integrated evaluation. Chapter 7 returns to each question and states the evidence supporting its answer.

RQ1 (measurement and attribution). How well can the profiling measurements support models of attributed UPF power and performance, particularly near the operating boundaries relevant to selection? *Chapters 3 and 5.*

RQ2 (forecasting granularity). Does forecasting service-area demand directly improve prediction compared with aggregating per-station forecasts at a comparable model budget, and what decision errors remain? *Chapter 4.*

RQ3 (counterfactual evaluation). How can fitted UPF models and transition measurements be combined to evaluate candidate realisations and derive model-dependent operating thresholds? *Chapter 5.*

RQ4 (shared-policy control). How do independent, shared-policy and centralised learning designs compare under a common evaluation protocol, and what additional benefit is associated with a centralised critic? *Chapter 6.*

RQ5 (integrated evidence and validity). What does the complete pipeline establish about control performance within the modelled environment, how sensitive are the conclusions to its assumptions, and what further evidence is needed for physical deployment? *Chapters 6 and 7.*

1.5 Contributions

The contribution is the empirical development and evaluation of an integrated measurement-to-control pipeline. Established learning methods are used to investigate four connected design questions and produce corresponding experimental artefacts:

1. **Measurement-based deployment characterisation.** Controlled profiling of the BESS/SD-Core DPDK and OpenAirInterface user-space deployments provides power and performance data under comparable traffic. Surrogate evaluation examines held-out runs and the uncertainty relevant to decision boundaries (Chapter 3).

2. **Forecasting at the service-area scale.** A fixed-budget comparison evaluates aggregation before forecasting against aggregation of per-station forecasts for the same delivered target. Model, cluster-count and partition comparisons assess the accuracy and cost of producing the demand signal needed by a site-level controller (Chapter 4).

3. **A common evaluator for candidate realisations.** The digital twin combines fitted surrogates, load calibration and activation-cost accounting. It derives model-dependent operating points and provides a controller-independent interface for replay and sensitivity analysis (Chapter 5).

4. **An evaluated comparison of controller designs.** The study compares classical policies and independent, centralised and shared-policy learners. Additional runs retrain all learned controllers under one protocol and include a shared-actor ablation of critic structure. Repeated seeds, per-site analysis and sensitivity studies assess performance, variability and dependence on modelling assumptions (Chapter 6).

Together, these contributions establish the utility of the inferred information within the studied modelled conditions. Surrogate agreement with measurements and controller performance inside the twin remain distinct forms of evidence. This distinction determines both the interpretation of the results and the next steps toward physical operation.

1.6 Thesis outline

Chapter 2 reviews the methods and architectural context of the four modules. Chapter 3 presents the physical campaign and surrogate evaluation. Chapter 4 develops the spatial

traffic pipeline, compares forecasting choices and examines decision-level error. Chapter 5 constructs the digital twin, defines transition accounting and evaluates its operating thresholds and limitations. Chapter 6 progresses from single-site control to multi-site learning, including a shared-policy critic comparison and sensitivity analyses. Chapter 7 synthesises the findings, answers the research questions and identifies the validation needed for deployment.

1.7 Code and data availability

The work is organised across four repositories covering the profiling campaign, traffic forecaster, digital twin and controllers. A source manifest records the code versions used for the reported experiments and the dependencies between their artefacts. Saved measurements, model outputs, checkpoints and evaluation reports provide the basis for the analyses, subject to the availability recorded for each experiment.

Re-evaluating a saved policy, retraining a model and repeating a physical campaign are different levels of reproduction. Saved checkpoints can be replayed exactly where they are available; for the multi-site comparison of Section 6.7, the original centralised-PPO checkpoints are not available, and replays of that controller use checkpoints trained with the same configuration. The additional runs of Section 6.9 are a separately trained set and are reported as such. Forecast outputs are version-pinned for downstream evaluation, while retraining can produce different weights and errors. Archived profiling data support model refitting, but repeating the measurement campaign requires the laboratory equipment. These boundaries determine which results can be reconstructed from saved artefacts and which require new experimental work.

CHAPTER 2

Related Work and Positioning

This chapter reviews the literature underlying the four pipeline stages introduced in Chapter 1: metric attribution, traffic forecasting, digital-twin evaluation, and cooperative control. It distinguishes established methods from the questions addressed by their application to UPF realisation selection, and examines how these methods can be validated. Detailed comparisons of individual tools, model architectures, and placement algorithms are developed in the technical chapters and are not repeated here.

2.1 The measurement premise

The thesis begins from the claim that per-network-function energy cannot be measured directly. That premise is not an assumption of this work; it is the position of both the hardware documentation and the standards.

Running Average Power Limit (RAPL) exposes energy counters for architectural *domains*, including package, cores, uncore, and DRAM, rather than for software entities. Systematic evaluations across platforms and workloads [1] map what each domain does and does not cover, and comparisons of software power meters report material disagreement between tools on identical hardware [8]. A value for one co-located network function is therefore necessarily inferred from aggregate counters by an attribution model, not read from a register.

3GPP takes the same position normatively. TS 28.554 [2] defines energy-consumption and energy-efficiency KPIs for network functions and states that the energy of a *virtualised* network function cannot be measured directly and is consequently defined as an estimate. The specification fixes the numerator, denominator and aggregation rules but deliberately leaves the estimation method and its validation open. That open method is where this thesis operates.

2.2 UPF-level metric attribution

Attributing shared-host power to processes and containers is a mature field. Virtual-machine and process-level estimators derive per-entity power from utilisation and calibrated models [9]; self-calibrating container meters such as SmartWatts learn the model online from RAPL and performance counters [10]; Kepler exports workload-level energy estimates in Kubernetes [11]; and EnergAt makes the allocation of *shared and idle* power an explicit policy choice rather than an implementation detail [12]. A systematic comparison of CPU power-model families [13] finds that model choice and training context strongly affect portability, which indicates that a per-function model is a calibrated artefact tied to its host rather than a transferable constant.

The closest prior art. Bellin, Granelli and Munaretto [14] deploy Open5GS and free5GC across bare-metal, virtual-machine and container configurations and combine external smart-plug readings with per-process estimates from the same power monitor used here, reporting values for individual 5G-core processes *including the UPF*. This work establishes prior application of per-NF power estimation to a 5G core. The remaining questions concern comparative measurement and control: the authors report inconsistent attribution in parts of their study, they do not compare data-plane realisations of the same function under matched traffic, and they do not carry the resulting signal into control. Earlier work by the same group [15] establishes the edge-deployment measurement platform without the finer-grained analysis.

Method lineage. Estimating a quantity that is expensive or impossible to instrument, from routinely measured proxies, is the established practice of *soft sensing* and *virtual metrology* [16]. The mature literature is explicit about what such a sensor requires before it can be treated as an observation: reference measurements, uncertainty, drift handling and recalibration when operating conditions change [17]. This thesis adopts that vocabulary deliberately. The method is not novel; the application to a UPF data plane, and the discipline of validating it, are where the contribution lies.

2.3 Counterfactual digital-twin evaluation

Using a cheap learned model to stand in for an expensive evaluation inside a decision loop is long-established as surrogate-based or metamodel-based optimisation [18]. The same

literature carries the warning most relevant here: an optimiser will exploit surrogate error if the model is wrong in a direction that appears favourable [19]. A controller choosing between UPF realisations on inferred counterfactuals is exactly such an optimiser, which is why Chapter 5 reports where the twin is queried outside its measured envelope rather than only reporting the controller's score.

At network scale the same idea appears as the network digital twin. ITU-T Y.3090 [5] specifies requirements and a functional architecture, including data collection, models, interfaces, and synchronisation, at a technology-neutral level, and the research literature converges on similar blocks [20]. Surveys of the area [21] consistently identify the same unresolved issue: the architectures say what a twin must contain, not how each physical relationship is identified, calibrated and kept valid. The twin in Chapter 5 is therefore not offered as a new architecture but as a concrete population-and-validation instance of a known one.

2.4 Spatiotemporal mobile traffic forecasting

Forecasting cellular demand from historical traces is a well-populated field. Spatiotemporal models of metropolitan cellular traffic [22] and convolutional citywide predictors [23] established the problem shape: strong daily and weekly periodicity, spatial correlation between nearby cells, and heavy-tailed peaks. The surveyed literature now spans classical, convolutional, recurrent and graph-based approaches [24]. Nothing in the forecasting component of this thesis is architecturally new: the models compared in Chapter 4 are published designs [25, 26, 27, 28].

Two things differ, and neither is an architectural claim. The first is the unit of prediction. Most of the literature forecasts per cell or per grid tile, because the consumer is a radio-side or planning application. The consumer here is a UPF controller acting for a whole MEC site, so the relevant signal is the aggregate over a cluster of base stations, and Chapter 4 asks where aggregation should happen, before or after forecasting. That question is specific to the deployment and is not, to my knowledge, settled in the cited literature. The second is that forecast quality is reported in decision terms rather than only as error: what matters for control is not the mean absolute error but how often the error changes the decision (Section 4.13).

The comparison in Chapter 4 is nonetheless confined to one city and one dataset, so the aggregation result should be read as evidence from a single deployment rather than a general property of cellular traffic.

2.5 Energy-aware control of network functions

Runtime energy management of network functions is not new, and this thesis does not claim it is. X-MAN adapts cloud-native network-function resources non-intrusively to save power under service objectives [29]; GreenNFV profiles and schedules functions against SLA constraints [30]; HADES integrates energy into placement and allocation over heterogeneous infrastructure [31]. Model-based analytics over NFV infrastructure telemetry has likewise been established for some time [32].

For the UPF specifically, the empirical picture is developing quickly. Measurements relate C-state configuration to UPF throughput, latency and power [33]; cloud-native 5/6G stacks are shown to consume power non-proportionally with load [34]; realisation choice is known to move the achievable performance envelope [35]. Most directly, Fuentes-Gonzales et al. [36] study DPDK-based UPFs under varying CPU allocation and traffic and derive energy-efficient management strategies, addressing precisely the cost of poll-mode processing that motivates realisation switching here.

These works occupy the space of *energy-aware UPF control*. What distinguishes the present work is not the lever but the signal chain behind it: the decision is taken on an attributed-power and QoS estimate whose accuracy is reported, a forecast whose error is converted into decision terms, and a counterfactual evaluation of the realisation that is not running.

2.6 Standards alignment and terminology

The four modules map onto roles the standardised architecture already anticipates. NWDAF [4] defines analytics services, identifiers and consumers within the 5G core; MDAF [3] provides the management-domain analytics counterpart; ETSI ZSM supplies the closed-loop sequence of observation, analysis, decision, and execution into which the modules fit [37, 6]. In every case the specification standardises the role and the interface while leaving the sensing and decision internals to the implementation, which is the space this thesis occupies rather than duplicates.

Infrastructure-level analytics. The term *Infrastructure Data Analytics Function* (IDAF) originates in research on energy-focused 6G management [38], where it names an infrastructure-level analytics component. Unlike NWDAF and MDAF, IDAF is not defined as a network function in the reviewed 3GPP specifications; its use here refers to that research concept.

Augmented observability. The phrase "AI-augmented observability" and close variants are used in adjacent fields. Here it denotes the application of four established methods—soft sensing, forecasting, surrogate evaluation, and policy transfer—to quantities a 5G core does not expose. The definition in Chapter 1 specifies the quantities inferred and their roles in the control pipeline.

2.7 What counts as validation

The most consequential lesson from the literature concerns the argument this thesis can support. A recurring risk in learned network control is to treat a controller's success inside a learned environment as evidence that the environment is faithful. The field does not accept that inference. A policy optimised against a model can exploit the model's errors, so its score is partly a measure of those errors [19, 39].

Accordingly, credible work in this area pairs learned-environment training with independent evidence. Aurora trains under randomised simulated conditions and then evaluates through a real networking stack [40]; Pensieve trains on traces and evaluates with real clients [41]; Pantheon supplies calibrated emulation alongside live-path measurement so that environment fidelity is testable rather than assumed [42].

These examples distinguish validation of a policy from validation of its environment. In this thesis, the controller results of Chapter 6 are reported as evidence of *internal consistency*: the inferred signals are mutually coherent and sufficient to act on. They are explicitly not reported as evidence of the twin's fidelity to hardware. The component-level evidence that does bear on fidelity is the held-out accuracy of the surrogates in Chapter 3, measured against physical measurement, and the robustness checks in Chapters 5 and 6 showing that the conclusions survive changes to the modelling assumptions. The physical measurements that would close the remaining gap are identified in Chapter 7.

2.8 Positioning

Table 2.1: Where each pipeline stage stands against the prior art, and the claim the evidence supports.

Pipeline stage	Status of the prior art	Claim made here
UPF-level metric attribution	Partly occupied: per-process meters exist, and per-NF estimation for a 5G core including the UPF is published [14].	Attributed power and QoS for two data-plane realisations under matched traffic, with held-out accuracy reported per model.
Spatiotemporal traffic forecasting	Occupied as to architecture: cellular traffic forecasting is a mature field [24] and the models used here are published designs.	Aggregation placed before forecasting rather than after, for a site-level consumer, with the error converted into decision-relevant terms. Evidenced on one city at equal model budget.
Counterfactual digital-twin evaluation	Method occupied: surrogate evaluators and network digital twins are established [18, 5].	A measurement-populated evaluator for the realisation not running, with its valid domain stated and its thresholds derived rather than assumed.
Cooperative cross-site control	Open in the screened UPF literature. Cooperative MARL with parameter sharing [43] and transfer in deep RL [44] are established generally, as is deep RL for networking [45].	Transfer of an operating policy between MEC sites, with the advantage measured as a function of fleet size.
Integration	No equivalent end-to-end realisation found.	An integrated, standards-aligned observability pipeline whose components are individually evidenced.

The contribution lies in integrating and evaluating established methods for energy-aware UPF realisation selection. The resulting measurement-grounded pipeline estimates per-UPF energy and QoS where direct instrumentation is unavailable, forecasts next-interval demand, evaluates the realisation that is not running, and transfers operating knowledge between sites. For each inferred signal, the accuracy is reported and the limits of the validation argument are stated explicitly.

CHAPTER 3

Energy Profiling of 5G User Plane Functions

3.1 Introduction

The User Plane Function (UPF) is the component of a 5G core network responsible for routing and forwarding GTP-U encapsulated user traffic between the Radio Access Network (RAN) and the Data Network (DN). Its power consumption is therefore directly coupled to network load, and understanding this relationship is a prerequisite for energy-aware network management and for meeting increasingly stringent sustainability targets in mobile telecommunications.

This chapter presents a systematic empirical study of two open-source UPF implementations under controlled laboratory conditions. The first implementation, SD-CORE DPDK, is the BESS-based user plane of the SD-Core project (`bess-upf`), which uses the Data Plane Development Kit (DPDK) poll-mode driver to bypass the kernel network stack. The second, USR-UPF, is the user-plane component of OpenAirInterface, deployed with its user-space application data plane: packets traverse the standard Linux networking stack and are processed in user space through the socket API, so packet reception is driven by the kernel's interrupt (NAPI/softirq) path rather than by a dedicated polling loop. Traffic was generated with a Keysight LoadCore emulator, and power was attributed per process using the Scaphandre power monitor. Power is polled at 1 s and the merged record is emitted on the 3 s cadence of the LoadCore export, which is the sample interval used throughout this chapter and the interval Δt of Equation (3.1).

This chapter provides the UPF-level metric attribution stage of the pipeline. It estimates quantities that a production core cannot natively measure. Hardware energy counters (RAPL) report per-*package* power, not the power drawn by an individual network function. Per-process power attribution is an established practice with a mature tool literature [10, 11, 12], and it has been applied to 5G-core functions including the UPF [14]; what the campaign below adds is a matched comparison of two data-plane realisations under identical offered

traffic. Related studies characterise UPF power against C-state configuration [33] and CPU allocation [36]. As Chapter 2 sets out, per-UPF attributed energy and QoS compliance are not directly available from standard telemetry in an operational system. The profiling campaign estimates those signals from controlled measurement, and the fitted surrogate that closes the chapter turns them into a *virtual sensor* the rest of the pipeline can query. The forecaster, the twin, and the controllers ultimately ground their decisions in the quantities made observable here.

The remainder of this chapter is organised as follows. Section 3.2 describes the measurement methodology and dataset. Section 3.3 characterises each variant across performance and energy dimensions. Section 3.4 investigates the saturation behaviour of USR-UPF in depth. Section 3.5 analyses the Specific Energy Consumption metric and its interpretation for poll-mode versus interrupt-driven drivers. Section 3.6 reports feature correlations relevant to predictive energy modelling. Section 3.7 discusses implications for network operation and modelling. Section 3.9 fits the surrogate models and reports their predictive accuracy; the twin built on them is developed in Chapter 5. Section 3.10 describes the NetMob 2023 compatibility adapter.

3.2 Measurement Methodology

3.2.1 Testbed Configuration

Both UPF variants were profiled using LoadCore traffic generation and Scaphandre process-attributed power monitoring under comparable offered-load schedules. The saved campaign records identify different Scaphandre exporters for the two variants and do not establish a common physical host or identical CPU and NIC hardware. The comparison is therefore between the recorded deployments; hardware equivalence remains unverified. What differs between the two campaigns is the whole software stack under test, not a single configuration switch. SD-CORE DPDK is the BESS user plane of SD-Core with the NIC detached from the kernel and bound to a DPDK poll-mode driver, which additionally requires hugepage backing and cores dedicated to the polling loop; USR-UPF is the OpenAirInterface user plane on the ordinary kernel networking path, which requests neither. They are separate projects with separate codebases.

What this permits the comparison to attribute. The treatment in this campaign is therefore the *deployment*, not the I/O model in isolation. A difference in power, delay or loss

between SD-CORE DPDK and USR-UPF is attributable to the pair (implementation, data-plane architecture) taken jointly: no configuration in the campaign varies one while holding the other fixed, so the contribution of kernel bypass cannot be separated from the contribution of the surrounding implementation. Isolating the I/O model would require one codebase built against both back-ends, which this campaign does not contain. Where the analysis below explains a measured behaviour by the polling loop, as in the flat power profile of Section 3.3, this is a mechanism proposed to account for the observation, consistent with what poll-mode drivers are known to do, and not a contrast this experiment identifies on its own. The surrogate models and everything downstream of them are unaffected by this: they are fitted per variant and predict the behaviour of a named deployment, which is exactly the quantity a controller choosing between two deployments needs.

LoadCore injected downlink UDP flows with a fixed packet size of 1250 B at progressively increasing offered loads. Each load step was sustained for a fixed duration to allow steady-state measurements; the raw time-series data were then merged with per-second power readings from Scaphandre. Figure 3.1 shows the full measurement architecture and the monitoring path. Timestamp alignment of the performance and power logs produces the surrogate training table described below.

3.2.2 Dataset

After preprocessing, the merged dataset comprises 9556 three-second measurement samples across 110 USR runs and an equivalent number of DPDK runs. Following feature engineering (removal of zero-variance and near-duplicate columns, unit conversions, target computation), the analysis dataset contains 8781 samples and 44 columns, split evenly between the two variants (4388 DPDK, 4393 USR).

3.2.3 Metrics

The following primary metrics are reported throughout this chapter:

Power (W) Power in watts attributed by Scaphandre to the UPF process (`power_watts`). This is a *per-process attribution*, not whole-host consumption: P_{total} denotes the power attributed to the UPF under load and P_{idle} the same quantity with the UPF running but no traffic offered.

Net power (W) Incremental power above the idle baseline: $P_{\text{net}} = P_{\text{total}} - P_{\text{idle}}$.

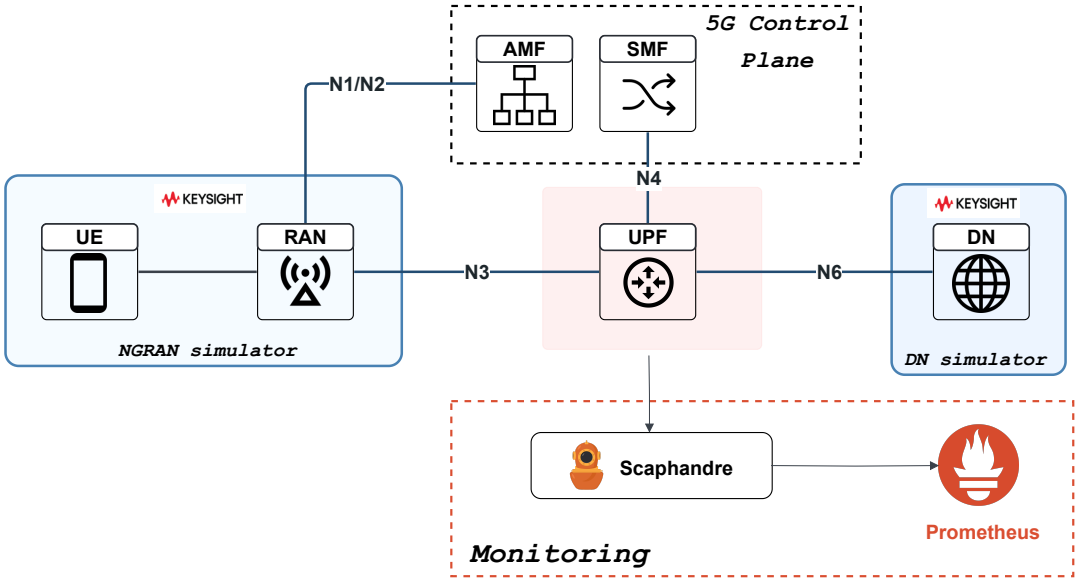


Figure 3.1: Physical testbed architecture used for UPF profiling. Keysight LoadCore emulates the UE/RAN and data-network endpoints. User-plane traffic passes through the UPF over N3 and N6; the AMF and SMF provide control-plane functions, with the SMF connected to the UPF over N4. Scaphandre exports process-attributed power measurements to Prometheus. The same logical arrangement is used for the two recorded UPF deployments.

$\mathrm{SEC}_{\mathrm{total}}$ Specific Energy Consumption based on total power: $\mathrm{SEC}_{\mathrm{total}} = P_{\mathrm{total}}/\lambda$, where λ is throughput in Gbps.

$\mathrm{SEC}_{\mathrm{net}}$ Marginal energy cost: $\mathrm{SEC}_{\mathrm{net}} = P_{\mathrm{net}}/\lambda$.

Packet loss Count of GTP-U packets lost per 3 s sample interval at the DN-side receiver.

Downlink delay Weighted mean one-way packet delay on the downlink path (μs).

3.3 Variant Profiles

3.3.1 Overview

Table 3.1 summarises the key performance and energy characteristics of both variants across all measurement conditions. Figure 3.2 illustrates the fundamental difference in the power–throughput relationship.

Table 3.1: UPF variant profiles: key performance and energy metrics.

Metric	SD-CORE DPDK	USR-UPF
Power		
Total power range (W)	0.807–0.859	0.000–4.497
Mean power (W)	0.821	1.435
Standard deviation (W)	0.005	1.527
Dynamic range (W)	0.052	4.497
Net power mean (W)	0.001	1.419
CPU		
Mean CPU (%)	1.064	1.549
Max CPU (%)	1.079	4.554
Idle CPU (%)	1.064	0.018
Throughput		
Maximum (Gbps)	5.06	2.33
Mean (Gbps)	0.788	0.528
Reliability		
Packet loss (sample rows, %)	0.0	32.7
Latency		
DL delay mean (μs)	311	5031
DL delay max (μs)	24978	25000
RAN jitter mean (μs)	1.3	1428
Energy efficiency at >1 Gbps		
SEC_{total} median (J/Gb)	0.27	1.61
SEC_{net} median (J/Gb)	0.002	1.61
L2/L3 overhead ratio (mean)	2.11	1.86

3.3.2 SD-CORE DPDK

The DPDK data-plane driver operates in poll mode: a dedicated CPU thread continuously spins on the receive queue, eliminating interrupt overhead at the cost of occupying the core regardless of traffic. This architectural choice manifests clearly in the measurements.

Power consumption remains almost constant at 0.821 ± 0.005 W across the entire load range (0–5.06 Gbps), yielding a dynamic range of only 52 mW. CPU utilisation is similarly invariant at 1.064 ± 0.004 %, and the Pearson correlation between throughput and CPU is $r = 0.016$, confirming that the polling loop consumes a fixed share of the core irrespective of packet rate.

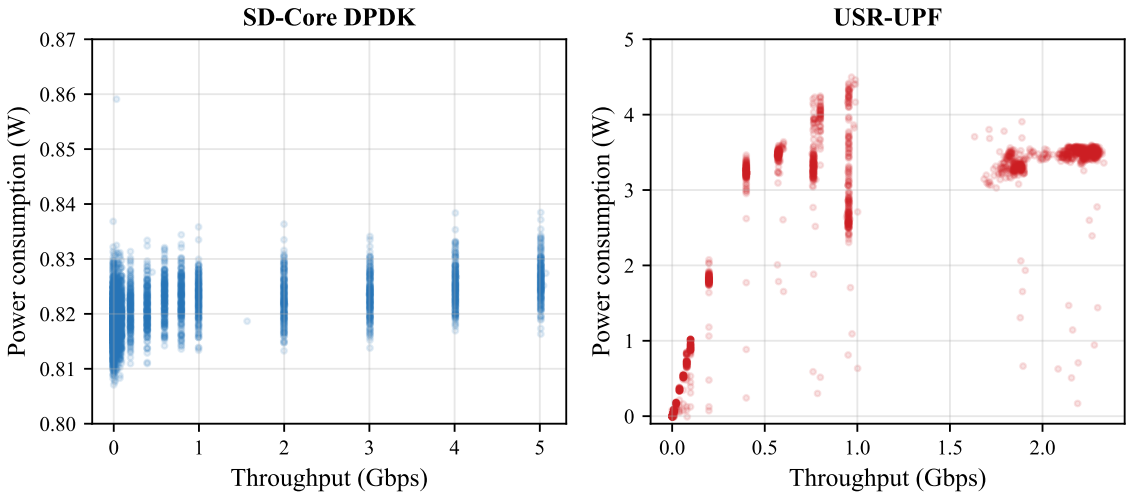


Figure 3.2: Power consumption versus throughput for each UPF variant. SD-CORE DPDK (left) exhibits a near-constant power profile due to continuous CPU polling. USR-UPF (right) shows a strong load-proportional relationship but saturates above 0.5 Gbps.

Despite its flat power profile, SD-CORE DPDK achieves the highest throughput of both variants (5.06 Gbps) with zero packet loss across all 4388 measurement samples. Downlink delay is low (mean 311 μs) and RAN jitter is negligible (mean 1.3 μs), making it the preferable choice for latency-sensitive traffic.

3.3.3 USR-UPF

USR-UPF processes packets in user space through the standard Linux socket API, on the kernel's interrupt-driven receive path. Power scales strongly with offered load: $r(\text{throughput, power}) = 0.81$ (compared with 0.45 for SD-CORE DPDK), and the system transitions from near-zero power at idle* to a plateau of approximately 3.4 W once saturated. CPU utilisation follows the same trend ($r = 0.76$), rising from 0 % at idle to 4.6 % at peak.

However, USR-UPF saturates at approximately 0.5 Gbps (see Section 3.4), beyond which virtually all packets are dropped. Delay is consistently higher than DPDK (mean 5 ms) and RAN jitter degrades severely above saturation (mean 1428 μs, maximum 199 ms).

*Measured over the 186 samples at exactly zero offered load, the USR-UPF process draws a *median* of 0.000 W: below the attribution floor of the per-process power monitor. The mean over the same samples is 0.131 W, inflated by a small number of transients as the process winds down after a load step, and the maximum is 3.14 W. The median is the appropriate summary and is the value quoted in Table 3.1; the switching-energy calibration of Chapter 5 uses a small non-zero figure instead, because a zero idle draw would make the burst-ratio scaling of Eq. 5.5 degenerate.

Figure 3.3 illustrates the load-stratified comparison of both variants across the four key metrics.

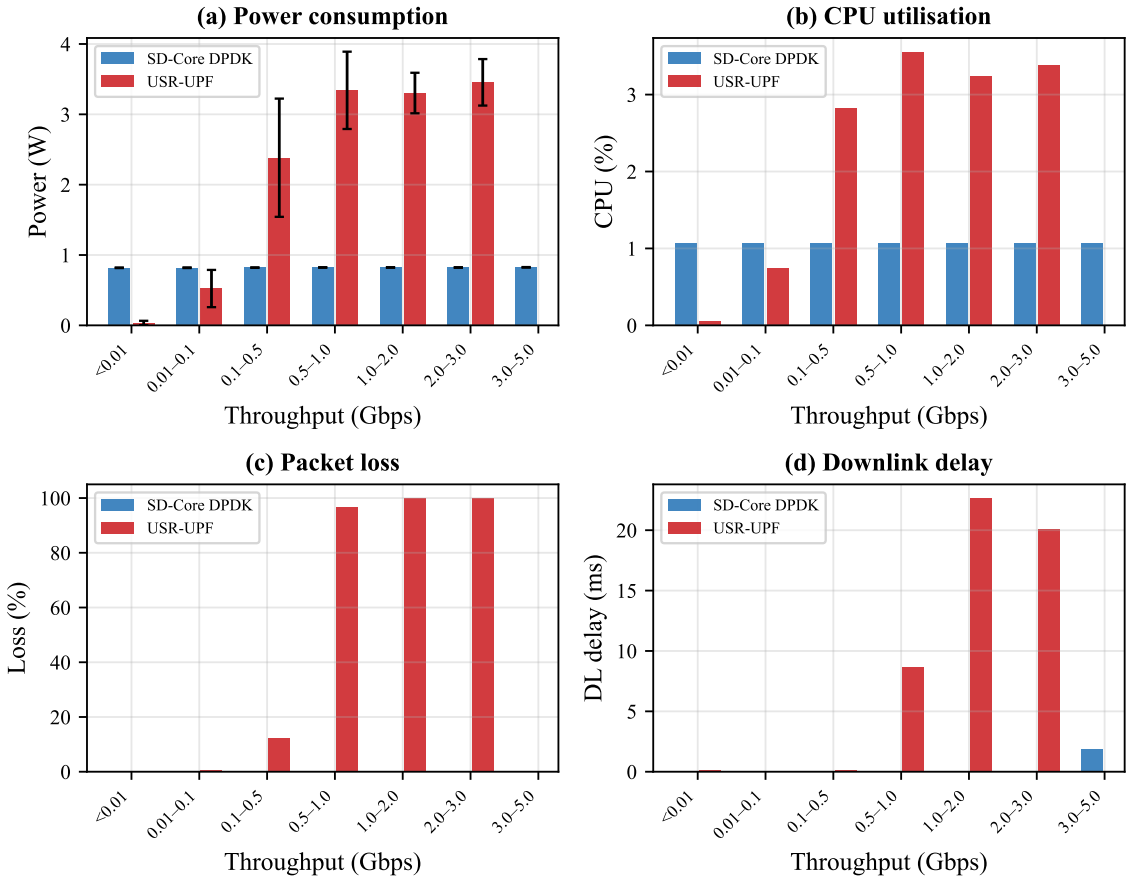


Figure 3.3: Load-stratified variant profiles. Throughput bins on the x-axis (Gbps). (a) Power consumption; (b) CPU utilisation; (c) packet loss incidence; (d) mean downlink delay. Error bars in (a) denote ±1 standard deviation. DPDK maintains flat power and zero loss at all loads; USR scales with load but saturates above λ_{env}.

3.4 USR-UPF Saturation Analysis

3.4.1 Operating Points: Definitions

The degradation of USR-UPF is gradual, and the analysis below identifies several distinct offered-load values along it. These are *different quantities*, not competing estimates of a single "saturation knee", and conflating them is the main source of apparent inconsistency

when the figures are read across chapters. They are fixed here and used with these names throughout the thesis.

Table 3.2: Measured USR-UPF operating points. All are offered downlink load in Gbps. λ_{qos} and λ_{be} are *derived* from the surrogate in Chapter 5 rather than measured here, and are listed only to keep the vocabulary in one place.

Symbol	Value	Meaning
λ_{loss}	≈ 0.06	Lowest per-run maximum offered load at which any packet loss was observed.
λ_{safe}	0.10	Empirical low-load reference from the per-run evidence; not a guaranteed safe limit. The controllers do *not* act on it directly; they act on the derived λ_{dec} below.
λ_{onset}	≈ 0.17	Saturation *onset*: first 50 Mbps bin in which more than 5 % of samples show loss.
λ_{env}	0.50	Modelling envelope: the upper bound of the `usr_safe` training set, and the load beyond which power plateaus.
λ_{50}	≈ 0.57	Half of all samples exhibit loss; beyond this USR-UPF is fully saturated.
λ_{qos}	0.149	Derived QoS limit (Chapter 5).
λ_{be}	0.091	Derived energy break-even (Chapter 5).

The modelling envelope λ_{env} defines the subset used to fit `usr_safe`; it does not certify loss-free operation. The empirical reference λ_{safe} summarises the loss-free runs but is also not a guarantee, since other runs below that value lost packets. Controllers use the separate surrogate-derived decision threshold λ_{dec}.

3.4.2 Saturation Onset

To characterise the saturation regime with fine granularity, measurements were binned into 50 Mbps throughput intervals. Figure 3.4 shows the transition across four dimensions: loss incidence, loss magnitude, delay, and the power/CPU trajectory.

The first bin where more than 5 % of samples exhibit packet loss occurs at $\lambda_{onset} \approx 0.17$ Gbps, the onset of saturation. Loss increases steeply, reaching 50 % at $\lambda_{50} \approx 0.57$ Gbps, beyond which essentially all samples report loss. From this point, USR-UPF enters a fully saturated regime: throughput measurements reflect the offered load

injected by LoadCore, not the effective forwarding rate, and the encoded delay summary approaches the 25 ms representative assigned to the open-ended highest bin.

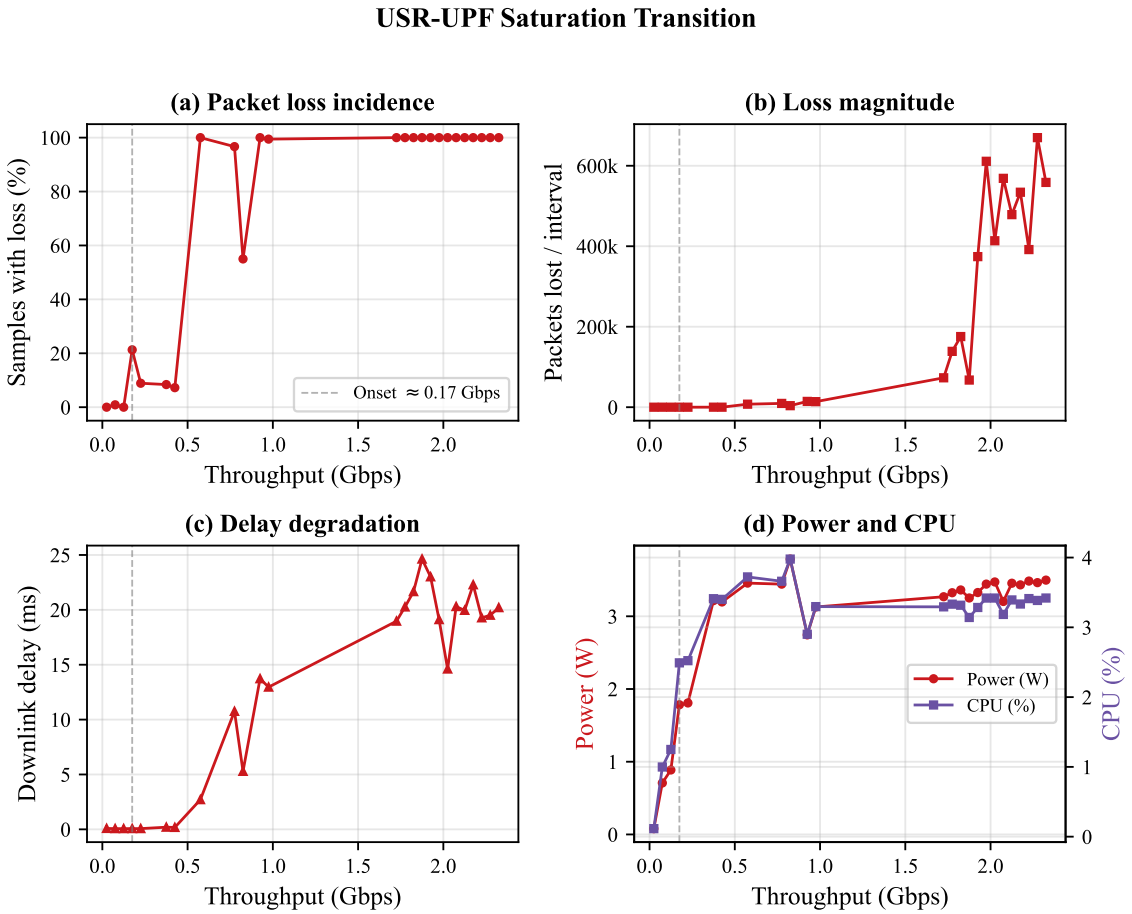


Figure 3.4: USR-UPF saturation transition in 50 Mbps throughput bins. The dashed vertical line marks the saturation onset $\lambda_{\text{onset}} \approx 0.17$ Gbps (Table 3.2). (a) Fraction of samples with packet loss; (b) mean number of packets lost per interval (when loss occurs); (c) mean downlink delay; (d) mean power and CPU utilisation (dual axis).

3.4.3 Power Behaviour at Saturation

Despite losing packets, USR-UPF continues to consume power proportionally to the offered load up to $\lambda_{\text{env}} = 0.5$ Gbps, after which power plateaus at approximately 3.4 W (Figure 3.4d). This indicates that the CPU is fully committed to packet processing at saturation; additional offered load does not increase power but increases loss instead.

The behaviour is summarised as follows:

Pre-saturation (< 0.3 Gbps): mean power = 0.338 W; loss-free operation is common but not guaranteed.

Transition (0.3–λ_{50} Gbps): power rises sharply, loss grows.

Post-saturation ($> \lambda_{\text{env}}$): mean power $\approx$ 3.4 W, near-total loss.

3.4.4 Per-Run Characterisation

Figure 3.5 aggregates the loss and power behaviour at the granularity of individual measurement runs. Of the 110 USR runs, 61 (55 %) completed without any packet loss; these runs had a maximum offered throughput of at most 0.1 Gbps. The lowest maximum offered load at which *any* run incurred loss is $\lambda_{\text{loss}} = 0.06$ Gbps. Between λ_{loss} and 0.10 Gbps the outcome is mixed: some runs in that interval lost packets and others did not, so neither value alone establishes a guaranteed safe boundary. The value $\lambda_{\text{safe}} = 0.10$ Gbps denotes the largest maximum offered load among the 61 loss-free runs and is retained as an empirical low-load reference. Because runs with loss also occur below it, it must not be interpreted as a conservative guarantee. Controllers instead use the model-derived $\lambda_{\text{dec}} = 81$ Mbps, with residual violation risk evaluated separately.

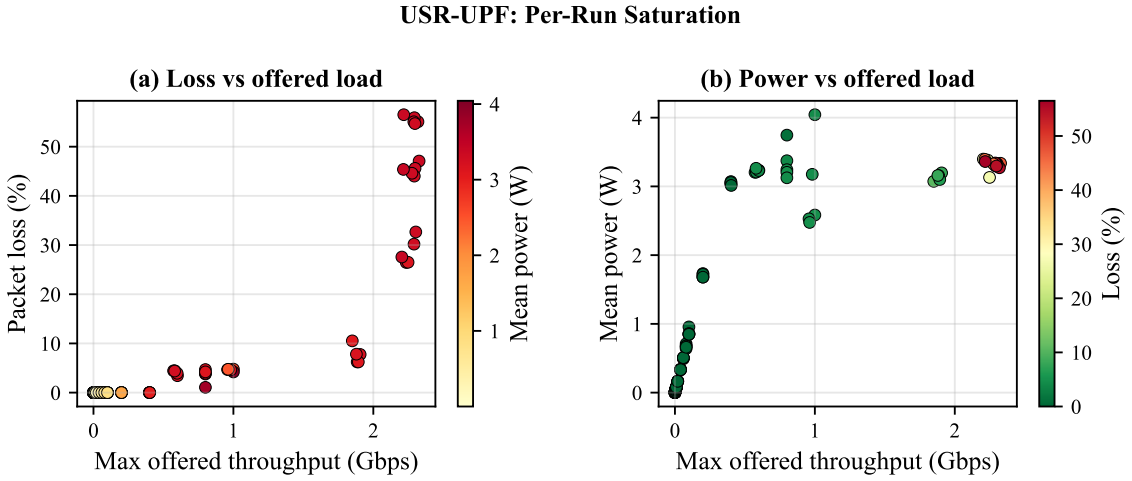


Figure 3.5: USR-UPF per-run saturation summary. Each point represents one measurement run. (a) Overall packet loss versus maximum offered throughput, coloured by mean power; (b) mean power versus maximum offered throughput, coloured by packet loss fraction.

3.5 Specific Energy Consumption Analysis

3.5.1 Definitions

Specific Energy Consumption (SEC) is defined as the ratio of power consumed to useful throughput delivered. Two variants are considered:

$$\mathrm{SEC}_{\mathrm{total}} = \frac{P_{\mathrm{total}}}{\lambda} \quad [J/Gb] \tag{3.1}$$

$$\mathrm{SEC}_{\mathrm{net}} = \frac{P_{\mathrm{net}}}{\lambda} = \frac{P_{\mathrm{total}} - P_{\mathrm{idle}}}{\lambda} \quad [J/Gb] \tag{3.2}$$

3.5.2 The Poll-Mode Measurement Paradox

The energy-per-bit form used here is the reciprocal of the data energy efficiency standardised for telecommunications equipment and mobile networks, which is defined as delivered data divided by consumed energy in bit/J [46, 47]; 3GPP TS 28.554 [2] applies the same construction at network-function scope, and defines the energy of a virtualised function as an estimate. The reciprocal is valid only when numerator and denominator share a boundary and an interval, so the boundary here is stated explicitly: the power attributed to the UPF process, the delivered downlink throughput, over one measurement interval. Equation (3.2) then isolates the *marginal* energy cost of one additional gigabit by removing the fixed idle overhead from the numerator. While this definition is appropriate for interrupt-driven systems (where $P_{\mathrm{idle}} \ll P_{\mathrm{total}}$), it produces a misleading result for poll-mode drivers.

In SD-CORE DPDK, the CPU polls continuously regardless of load, so $P_{\mathrm{idle}} \approx P_{\mathrm{total}} \approx 0.821\,\mathrm{W}$ at all times. Consequently, $P_{\mathrm{net}} = P_{\mathrm{total}} - P_{\mathrm{idle}} \approx 0$, and $\mathrm{SEC}_{\mathrm{net}}$ approaches zero as throughput increases, superficially suggesting near-perfect efficiency. In reality, SD-CORE DPDK consumes 0.82 W regardless of whether it forwards 0 or 5 Gbps.

Table 3.3 and Figure 3.6 quantify this discrepancy at high throughput (> 1 Gbps).

Table 3.3: Specific Energy Consumption at high throughput (> 1 Gbps): median values.

Metric	**SD-CORE DPDK**	**USR-UPF**
Total power at >1 Gbps (W)	0.824	3.478
$\mathrm{SEC}_{\mathrm{total}}$ (J/Gb)	0.274	1.615
$\mathrm{SEC}_{\mathrm{net}}$ (J/Gb)	0.002	1.607

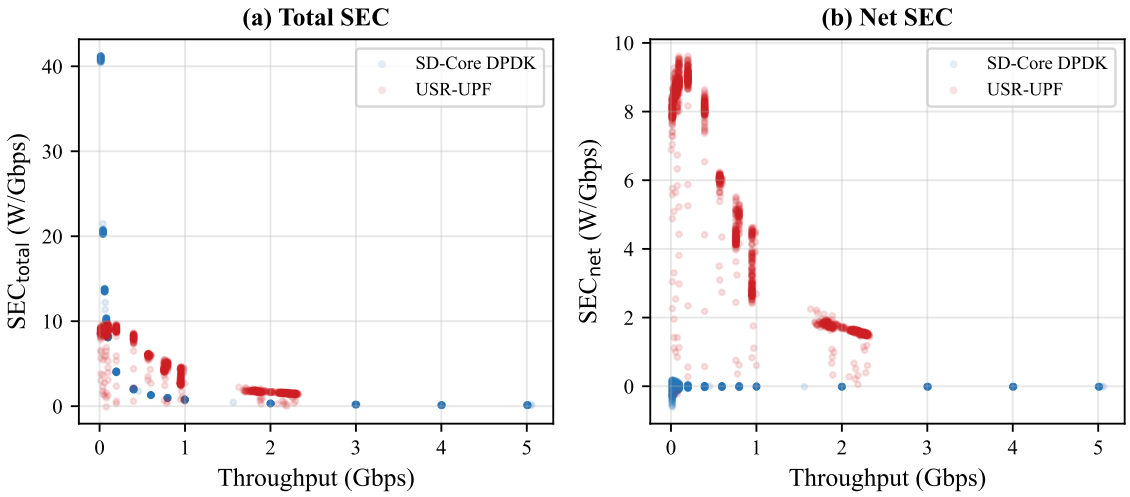


Figure 3.6: Specific Energy Consumption versus throughput. (a) SEC_{total}: DPDK appears efficient due to low power denominator relative to flat numerator. (b) SEC_{net}: SD-CORE DPDK's SEC_{net} collapses to near zero because its incremental power $P_{net} \approx 0$. Both metrics understate the operational energy cost of running a poll-mode UPF. Extreme low-throughput outliers are clipped to the 98th percentile for clarity.

3.5.3 Implications for Energy Metric Selection

For a fair cross-variant comparison, SEC_{total} is the more appropriate metric: it reflects the full operational cost of sustaining throughput λ.

One row of Table 3.3 must be read with care, and it is the row the headline comparison would otherwise rest on. Above λ_{50} the USR-UPF throughput counter reports the load *offered* by the generator rather than the rate actually forwarded (Section 3.4), so at > 1 Gbps the USR-UPF figure of 1.615 J/Gb divides power by traffic that was largely dropped. That violates the boundary condition stated in Section 3.5.1: numerator and denominator must cover the same delivered data. It is therefore *not* an efficiency measurement. It is reported here only to make the failure visible, and no cross-variant efficiency ratio is drawn from it. The defensible comparison is confined to loads below λ_{onset}, where USR-UPF delivers what is offered; there SD-CORE DPDK remains the more efficient variant per delivered bit, but by a margin that narrows sharply as load falls, which is precisely the regime the controllers exploit.

Total power must also be considered in absolute terms: at idle SD-CORE DPDK draws 0.82 W whereas USR-UPF draws a median of 0.000 W (Table 3.1 and its footnote). The preferable variant therefore depends on the traffic load distribution in deployment.

3.6 Feature Correlation Analysis

3.6.1 Correlates of Power Consumption

Table 3.4 lists the ten strongest Pearson correlations with `power_watts` in the pooled dataset (excluding features derived directly from the target).

Table 3.4: Top feature correlations with power consumption (pooled dataset). Rolling features are marked † to indicate data-leakage risk when used as predictors.

Feature	Pearson r	Note
`power_watts_roll_max_30s`†	+0.998	Derived from target
`cpu_pct`	+0.991	Legitimate predictor; see below
`power_watts_roll_mean_30s`†	+0.982	Derived from target
`cpu_per_watt`	−0.936	Derived from target
`dl_delay_power_product`	+0.797	Interaction feature
`high_delay_frac`	+0.768	QoS indicator
`weighted_mean_delay_us`	+0.735	Latency feature
`ul_delay_power_product`	+0.614	Interaction feature
`l2l3_overhead_ratio`	−0.561	Regime indicator†
`packets_lost_delta`	+0.535	Loss indicator

A pooled correlation computed across two variants with very different power levels can be an artefact of the grouping rather than evidence of a relationship within either group, so the CPU–power coefficient was recomputed per variant. It is not an artefact here: within USR-UPF alone the coefficient is $r = 0.993$, marginally *higher* than the pooled 0.991, while within SD-CORE DPDK it falls to $r = 0.642$. The relationship therefore holds inside the variant that carries the variance, and the low SD-CORE DPDK figure reflects that variant's near-constant power rather than a failure of the predictor.

Figure 3.7 shows the correlation structure separately for each variant. The two variants exhibit markedly different correlation patterns: in SD-CORE DPDK, power is weakly correlated with all traffic metrics ($r < 0.5$) because the power signal is nearly flat. The throughput–power coefficient of $r = 0.45$ should not be read as a weak-but-usable relationship: it is computed over a total power span of 52 mW, so even perfect correlation

†The measured median of this ratio is 2.10, i.e. the L2/L3 device counts roughly twice the bytes the GTP-U layer does. That is far above the ≈ 1.03 that per-packet framing overhead alone would produce on 1250 B payloads, so it is not an encapsulation-overhead figure in the literal sense: the two counters are taken at different layers and over different scopes, and their ratio should be read as a dimensionless regime indicator rather than as a physical overhead. It is retained because it is predictive, not because it is interpretable.

would carry almost no actionable information, and the operationally correct summary is that SD-CORE DPDK power does not vary with load. In USR-UPF, the throughput–power correlation is $r = 0.81$ and CPU–power is $r = 0.99$, making power highly predictable from traffic observables.

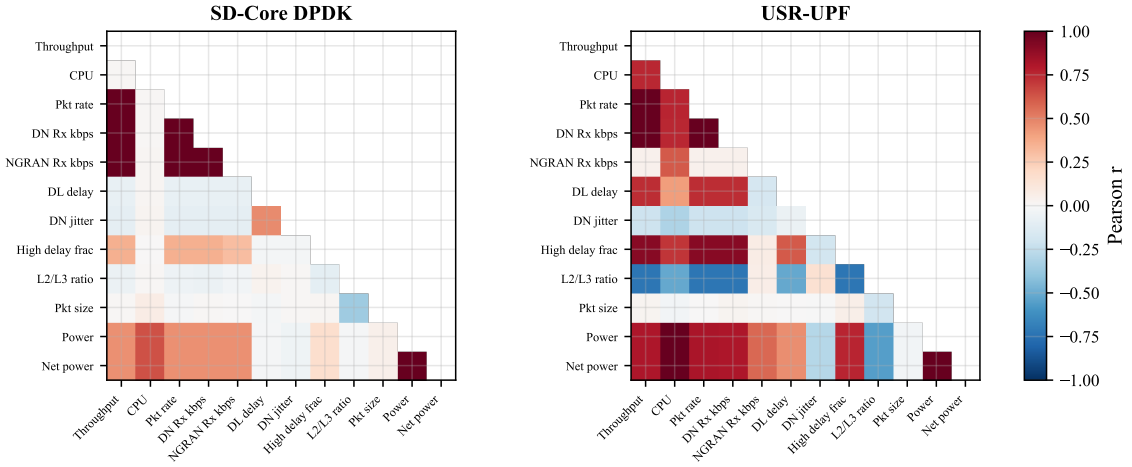


Figure 3.7: Pearson correlation heatmaps for a subset of key features, computed separately for each variant. The DPDK heatmap (left) shows uniformly low correlations with power, consistent with the near-constant power profile. The USR heatmap (right) reveals strong positive correlations between throughput, CPU, and power.

3.6.2 Temporal Dynamics

Figure 3.8 shows representative power and throughput traces within a single high-load run for each variant. SD-CORE DPDK exhibits essentially no power variation despite large throughput swings; USR-UPF tracks load closely until saturation, then power plateaus while throughput (offered load) continues to increase.

3.7 Discussion and Implications

3.7.1 Architectural Trade-offs

The measurements reveal a fundamental trade-off between the two I/O paradigms:

Poll mode (DPDK) delivers superior throughput (5.06 vs 2.33 Gbps), zero packet loss, and deterministically low latency, at the cost of a fixed power floor of 0.82 W regard-

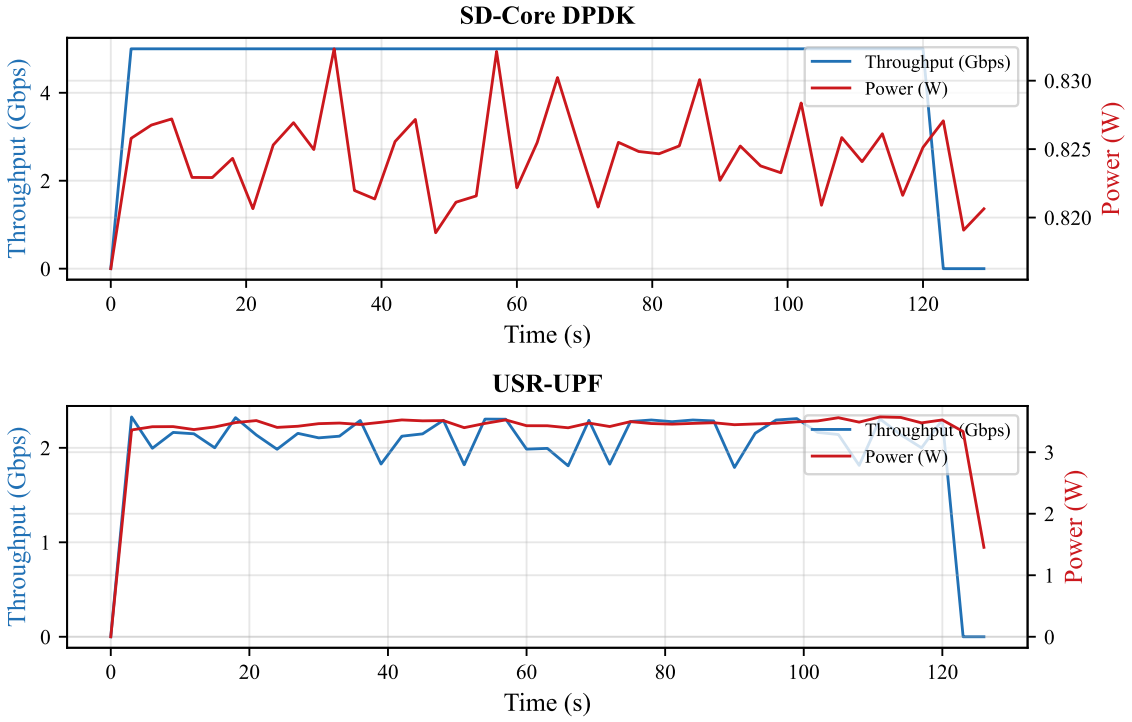


Figure 3.8: Temporal power and throughput traces for representative high-load runs. SD-CORE DPDK (top): power remains flat despite throughput variations. USR-UPF (bottom): power rises with load until the saturation plateau.

less of load. This is appropriate for high-throughput, latency-sensitive deployments where the UPF is expected to be consistently busy.

Interrupt mode (USR) scales power proportionally to load, drawing near-zero power at idle, but saturates at λ_{env} and incurs severe delay and loss in the saturated regime. It is preferable only in low-to-medium load scenarios where energy-proportionality is prioritised over peak throughput.

3.7.2 Empirical Operating Regions

The per-run analysis (Section 3.4, Figure 3.5) identifies a low-load region in which USR often operates without loss, but outcomes vary between runs even at similar loads. The reference $\lambda_{\text{safe}} = 0.10$ Gbps is therefore descriptive rather than a certified safe limit. The twin derives its own decision threshold from the power and QoS predictions, and reports the residual error of the safety verdict (Section 5.7.1). DPDK exhibited no packet loss in the campaign, although its measured delay was variable.

3.7.3 Implications for Predictive Energy Modelling

The near-constant power of SD-CORE DPDK presents a challenge for regression-based energy models: the target variable has a dynamic range of only 52 mW, approaching the noise floor of Scaphandre. Models trained on DPDK data are expected to predict 0.82 W for almost any input; the information content of the traffic features is minimal. Per-variant models are therefore strongly recommended over a single pooled model.

For USR-UPF, the strong linear relationships ($r > 0.8$) between throughput, CPU, and power suggest that simple linear or low-complexity tree models may suffice in the unsaturated regime. Training should be restricted to the modelling envelope $\lambda < \lambda_{\text{env}}$ unless the objective explicitly includes the saturated regime. The modelling envelope λ_{env} and empirical low-load reference λ_{safe} describe different aspects of the data; the controller's decision threshold is λ_{dec} (Table 3.2).

The rolling power features (`power_watts_roll_mean_30s`, `power_watts_roll_max_30s`) achieve very high correlations with the power target ($r > 0.98$) and must be excluded as predictors when the modelling goal is to predict power from non-power observables.

3.8 Variant Performance Summary

Before constructing the digital twin, the three deployment variants, DPDK, USR (full range), and USR (restricted training range), are compared across five key performance indicators (KPIs): downlink throughput, CPU utilisation, packet loss, one-way delay, and power consumption. Figure 3.9 shows the empirical distributions of each KPI as violin plots, with individual measurements overlaid as semi-transparent scatter points.
Three structural differences are relevant:

1. **DPDK power is load-independent.** The poll-mode driver keeps all cores spinning at full speed irrespective of traffic; the power distribution is narrow and flat across all load steps. This is the primary reason the DPDK Layer-2 power model achieves only $R^2 = 0.29$.

2. **Restricting the USR training range reduces target variance.** The `usr_safe` subset contains 2997 samples with recorded throughput below 0.5 Gbps, of which 58 report packet loss. The name identifies a restricted training subset, not a loss-

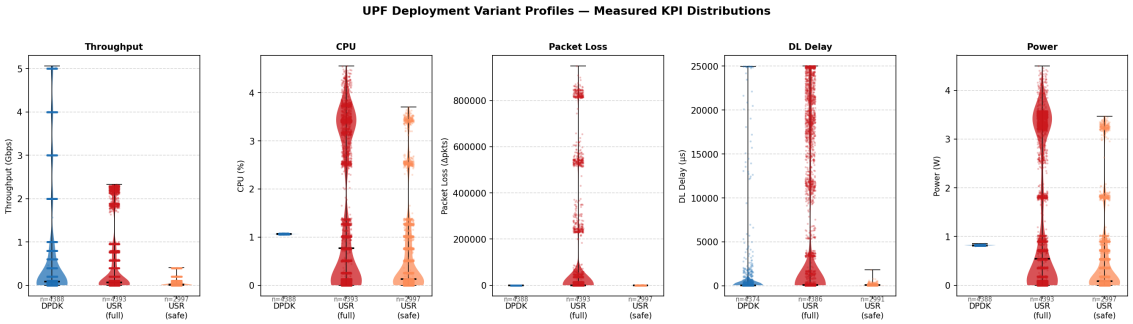


Figure 3.9: Measured KPI distributions for each deployment variant. Each violin combines a kernel-density estimate with the observed range; horizontal bars mark the median. Points are jittered for visibility. DPDK power exhibits near-constant behaviour (poll-mode driver draws fixed power regardless of load), explaining its low model R^2. USR (restricted range, `usr_safe`) loss and delay have lower variance than their full-range counterparts, which limits their model predictability compared with the full-range USR variant.

free regime. Its lower loss and delay variance limits the interpretation of their fits ($R^2 = 0.86$ and 0.52 respectively), despite high power and CPU scores ($R^2 > 0.999$).

3. **USR (full range) covers saturation.** Including the high-load regime exposes the non-linear onset of loss and delay, yielding $R^2 = 0.998$ and 0.968 for those targets at the cost of a slightly noisier power model ($R^2 = 0.988$).

Two caveats on these figures. The delay target is a histogram summary rather than an exact continuous measurement. The preprocessing assigns representatives to each LoadCore bin and computes their packet-weighted mean. In particular, the open-ended 20 ms-and-above bin is represented by 25 ms; this value is a coding convention, not a measured maximum delay. The model's $R^2 = 0.968$ therefore describes agreement with the encoded summary, not recovery of the true delay in the high-delay tail. Finer bins or packet-level measurements would be needed to assess that tail. For the 200 μs decision budget, samples dominated by that tail are clearly unacceptable, but the exact delay magnitude remains unresolved.

The throughput target carries the opposite caveat. Below saturation the UPF delivers essentially what is offered, so predicting delivered throughput from offered load is close to reproducing an identity; the near-unity R^2 for that slot is therefore a consistency check that the pipeline is wired correctly, not a measure of modelling skill. The informative throughput result is the one at and above λ_{onset}, where delivered and offered diverge.

The choice between USR safe and USR full therefore depends on the inference context: the restricted-range model may be used when loads remain within its training support, while the full-range model is preferable when the full operating envelope, including congestion, must be captured.

3.9 Surrogate Models for the Digital Twin

The empirical findings of the preceding sections motivate a set of *surrogate models*: given only an offered traffic load, they predict the UPF's power consumption, CPU utilisation, delivered throughput, packet loss and one-way delay, without requiring a live deployment. This section reports how those models were fitted and how accurate they are; the twin that composes them into a simulator, including its architecture, switching-cost model, safety semantics, and operating thresholds, is the subject of Chapter 5.

Two terms are needed to read the results below. The models are fitted as a two-layer stack per variant, mirroring the causal pathway load → performance → power. **Layer 1** comprises four independent regressors mapping offered load to delivered throughput, CPU utilisation, packet loss and downlink delay, using only TX-side controlled quantities as inputs. **Layer 2** is a single regressor predicting `power_watts` from those same inputs *plus* the four Layer 1 outputs, which is what allows power to be predicted at inference time without circularity, because the Layer 1 predictions are available before Layer 2 runs. Section 5.2 develops the structure in full, including why each layer's model family is selected independently and what that implies for extrapolation.

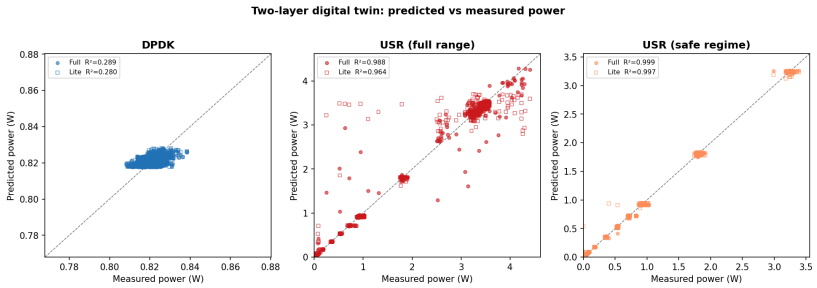


Figure 3.10: Predicted versus measured power for the three trained variants (SD-CORE DPDK, USR-UPF full range, USR-UPF restricted training range). Solid markers: full model; open markers: lite (NetMob-compatible) model. The dashed line is the identity.

3.9.1 Training Strategy

Three variant-specific models are trained to reflect the distinct operating regimes identified in Section 3.3:

> **`dpdk`**: all 4 388 DPDK samples. Included as an evidence baseline even though the near-constant power (52 mW range) makes the regression R^2 inherently low.
>
> **`usr_full`**: all 4 393 USR samples, including the saturated regime above λ_{50}.
>
> **`usr_safe`**: USR samples with delivered throughput below the modelling envelope $\lambda_{\text{env}} = 0.5$ Gbps.

The λ_{env} cut defines a restricted training envelope rather than a safe operating guarantee. The `usr_safe` label is an implementation name for that subset, which includes samples with loss. The full-range model is used for control evaluation because it includes the wider loss and delay responses; λ_{safe} remains an empirical reference and λ_{dec} is the separate controller threshold.

For each variant and each prediction slot (Layer 1 target or Layer 2 target), three model types are compared: Ridge regression, Random Forest, and Gradient Boosted Trees. Hyperparameters are selected by 5-fold cross-validation on the training split, and the model *type* is then chosen by test-set R^2.

Validity of the split. Two potential sources of optimistic bias were evaluated. The train/test division is made at the level of individual samples, so measurements taken 3 s apart within one sustained load step can fall on opposite sides of it; and choosing the model type by test-set R^2 means the reported score is also the selection criterion. Either could inflate the numbers in Table 3.5.

To measure the first, the entire pipeline was re-run with the sample-level split replaced by a grouped split in which no measurement run may appear on both sides, while every other element, including features, model family, hyperparameter search, and random seed, was held fixed. Across 220 runs and 8,781 samples:

Variant	Sample-level split	Grouped by run	Change
`usr_full`	0.988	0.993	+0.005
`usr_safe`	0.999	0.996	−0.003
`dpdk`	0.286	0.241	−0.045

The USR-UPF models are unaffected; `usr_full` is marginally *better* when runs are held out whole. These full-feature power scores show little sensitivity to this particular grouped split; they do not establish held-out accuracy of the deployed lite models or safety classification near the decision boundary. For this power comparison, Layer 1 features are TX-side controlled quantities that are near-constant within a load step, so a held-out run at a given offered load is not meaningfully harder to predict than a held-out sample from a run already seen. The SD-CORE DPDK model does lose accuracy, which is consistent with its already-poor fit: its target varies over only 52 mW, so it has little signal to generalise in the first place. The appropriate reading of both columns is that SD-CORE DPDK power is essentially load-independent rather than predictable.

To assess bias from model selection, a third evaluation used a fully nested protocol: the split is grouped by run; the model family *and* its hyperparameters are chosen by `GroupKFold` cross-validation *within the training runs only*; and the selected model is then scored once on the untouched test runs, with no further selection. The earlier grouped run had fixed the split but not the selection: it still compared families by test-set R^2, and its inner hyperparameter search used plain k-fold, so folds cut through runs.

Nested selection changes which family wins in 15 of the 30 slots. This does not materially alter the accuracy results, because in almost every case the families were effectively tied. The median absolute change in test R^2 across all 30 slots is 0.0004, and 25 of 30 change by less than 0.05. Every model the twin actually consumes is stable. For `usr_full`, the lite loss model moves from 0.9951 to 0.9950 and the lite delay model from 0.962 to 0.959, and among the six Layer 2 power slots the two criteria still agree on five.

Layer 2 power, full features	Sample split	Grouped	Grouped + nested CV
`usr_full`	0.988	0.993	0.991
`usr_safe`	0.999	0.996	0.996
`dpdk`	0.286	0.241	0.191

One slot changes substantially and should be reported because it reverses a conclusion rather than merely adjusting it. The SD-CORE DPDK *delay* model scored $R^2 \approx 0.28$–0.31 under both earlier protocols; under nested selection it scores -0.06 (lite) and -1.69 (full), and its cross-validated score is -86.8, far worse than predicting the training mean. The earlier figure should not be interpreted as a reliable fit. It was the maximum of three families evaluated on the set that then reported the score, and SD-CORE DPDK delay, with median 62.8 μs and a heavy tail reaching the 25 ms representative of the open-ended delay

bin discussed in Section 3.7, is not predictable from offered load at all. The nested protocol also selects a better-behaved model: the family it discards (Ridge) extrapolates to $-51.8\ \mu$s at low load, a negative delay, while the one it selects floors at $62.5\ \mu$s, exactly the measured minimum. The nested protocol therefore yields a lower estimate of predictive accuracy while selecting a model with physically admissible delay predictions.

Two consequences follow. Table 3.5 reports the originally deployed models, which are the ones inside the twin and therefore behind every downstream result in this thesis; their scores are the optimistic end of the range on both counts, and should be read as such. And the quantity that actually propagates downstream is not R^2 but the derived QoS limit, which is stable across all three protocols: 149 Mbps deployed, 150 grouped, and 153 fully nested. This spread is far narrower than the uncertainty in the limit itself (Section 5.7.1).

Layer 2 trains on *out-of-fold* Layer 1 predictions produced by a second grouped pass, so it never observes ground-truth Layer 1 values during training. This matches the distribution it will encounter at inference time (stacking). In the nested protocol those folds are grouped by run as well, and the family used inside each fold is chosen by an inner cross-validation on that fold's training part rather than by score on the part it is about to predict.

3.9.2 Deployed models and scope of validation

The models used by the twin and by every downstream experiment are the ones fitted with the row-level split; their metrics are those of Table 3.5. The grouped and nested protocols above refit the same pipeline to estimate how optimistic those scores are. For full-feature Layer 2 power, the sample-level refit gives $R^2 = 0.286$, 0.988 and 0.999 for DPDK, USR full-range and USR restricted-range, and the grouped-run refit gives 0.241, 0.993 and 0.996. The refitted models are not substituted into the downstream experiments, so that every controller result is computed with one consistent set of models.

A complementary study withholds complete measurement sweeps and load levels, selects models within the remaining runs, and evaluates the lite models through the twin's input adapter (Section 5.7.1). Because the deployed models were trained on rows from every measurement run, this study validates the modelling procedure on unseen runs rather than the deployed models themselves. Whole-range power accuracy and decision-region safety accuracy are therefore reported separately.

3.9.3 Model Performance

Table 3.5 summarises Layer 2 power model performance. Figure 3.10 shows predicted versus measured power for all variants and both model variants.

Table 3.5: Layer 2 power metrics of the deployed models (row-level split; model family selected on that test split). Grouped and held-out-sweep scores are reported in Sections 3.9.2 and 5.7.1.

Variant	Mode	R^2	RMSE (W)	MAE (W)	Best model
DPDK	Full	0.289	0.004	0.003	Ridge
DPDK	Lite	0.280	0.004	0.003	Gradient Boosting
USR (full)	Full	0.988	0.172	0.050	Random Forest
USR (full)	Lite	0.964	0.245	0.070	Random Forest
USR (safe)	Full	0.999	0.026	0.012	Ridge
USR (safe)	Lite	0.997	0.038	0.017	Ridge

The `usr_safe` variant achieves $R^2 = 0.999$ with the full feature set and $R^2 = 0.997$ with only DL/UL throughput inputs, confirming that power scales nearly linearly with load in the unsaturated regime. The `usr_full` model degrades gracefully ($\Delta R^2 \approx 0.024$) in lite mode, demonstrating that packet-count and overhead features add moderate but non-negligible predictive value. The DPDK $R^2 = 0.289$ confirms the analytical expectation: a 52 mW signal range is indistinguishable from noise at the model's operating scale.

Figure 3.11 provides the full R^2 picture across all prediction slots. Figure 3.12 shows the three-way algorithm comparison for the L2 power model, directly justifying the winning selection per variant. Figure 3.13 extends this comparison to all L1 targets. Figure 3.14 presents side-by-side interpretability plots (Ridge coefficients and tree importances) on the same feature order, making the two model families comparable regardless of which was selected.

3.10 NetMob 2023 Compatibility

The NetMob 2023 dataset provides per-cell normalised UL/DL traffic loads $\hat{d} \in [0, 1]$ but no packet-level detail. To drive the digital twin from such data a thin adapter layer translates normalised loads into the model's input space:

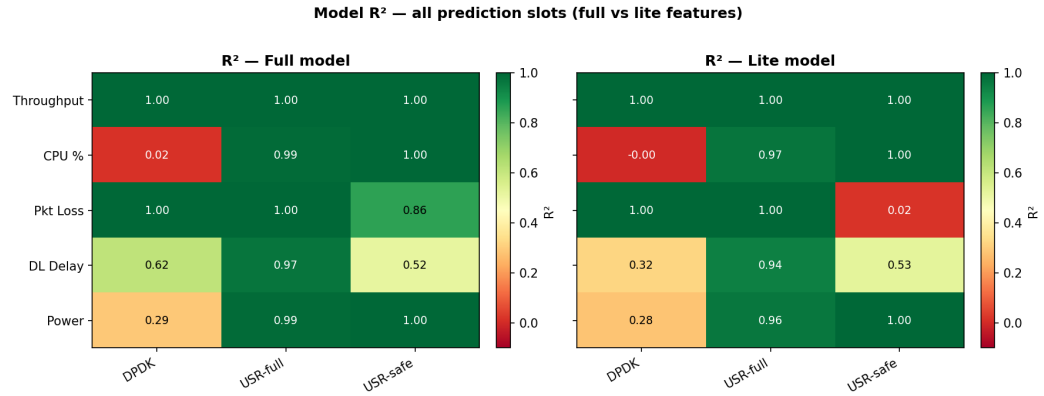


Figure 3.11: R^2 for all prediction slots (variant × target), full model (left) and lite model (right). Green = high accuracy; red = low. DPDK CPU and DPDK delay are structurally unpredictable (poll-mode artefacts).

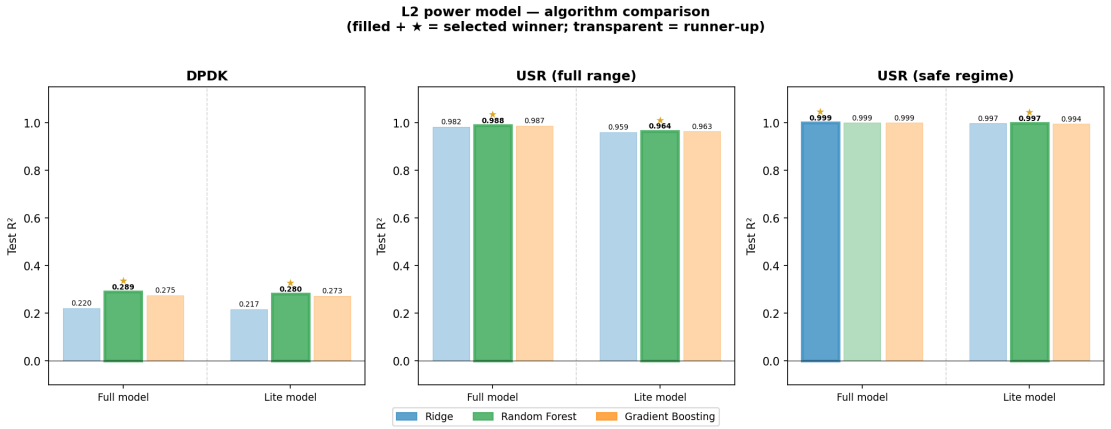


Figure 3.12: Algorithm selection for the L2 power model. Each group shows Ridge, Random Forest, and Gradient Boosting test R^2 for the full and lite feature sets. Filled bars and gold stars (★) mark the selected winner per group; transparent bars are runners-up. Ridge wins for `usr_safe` (linear regime); Random Forest wins for `usr_full` (non-linear saturation effects).

$$B_{\mathrm{DL}} = \hat{d}_{\mathrm{DL}} \cdot C_{\mathrm{max,DL}} \cdot 10^6 \quad [\mathrm{kbits/s}] \tag{3.3}$$

$$N_{\mathrm{pkt}} = \frac{B_{\mathrm{DL}} \cdot 10^3 \cdot \Delta t}{8 \cdot \bar{s}_{\mathrm{pkt}}} \tag{3.4}$$

where $C_{\max}$ is the rated UPF link capacity (default 10 Gbps), $\Delta t = 3$ s is the sampling interval, and $\bar{s}_{\mathrm{pkt}}$ is the assumed average packet size (default 1250 B). Missing features

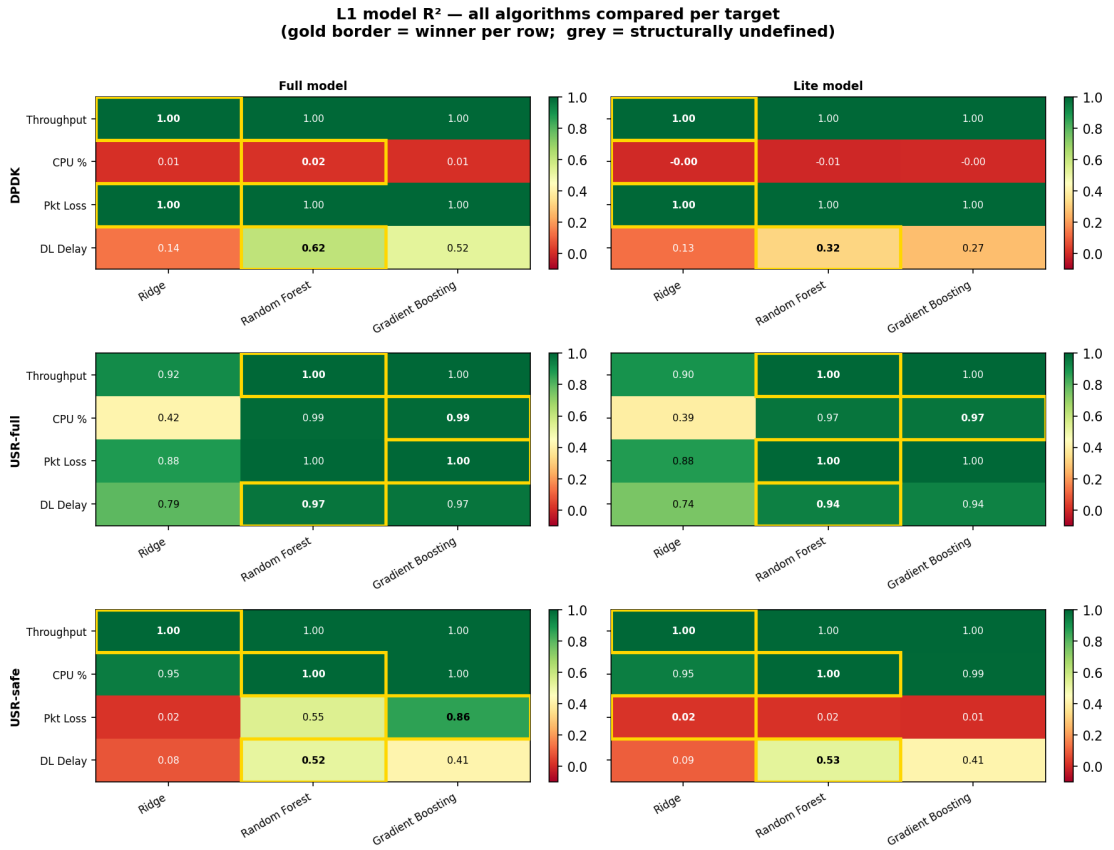


Figure 3.13: Algorithm comparison for all L1 prediction slots. Rows: prediction targets; columns: algorithm types. Gold borders mark the winning algorithm per target row. Ridge dominates the linear throughput–power relationship; tree models offer marginal gains only for CPU and delay targets.

(overhead ratio, etc.) are imputed from deployment constants stored in the twin configuration.

Full versus lite models. The *full* model accepts all seven offered-load features and serves laboratory or high-fidelity simulation scenarios. The *lite* companion model accepts only B_{DL} and B_{UL}, making it directly compatible with NetMob and any other dataset where only load volume is known. The lite model quantifies how much accuracy is sacrificed for this interoperability: for `usr_safe`, the power-model R^2 drops by only 0.002 (from 0.999 to 0.997).

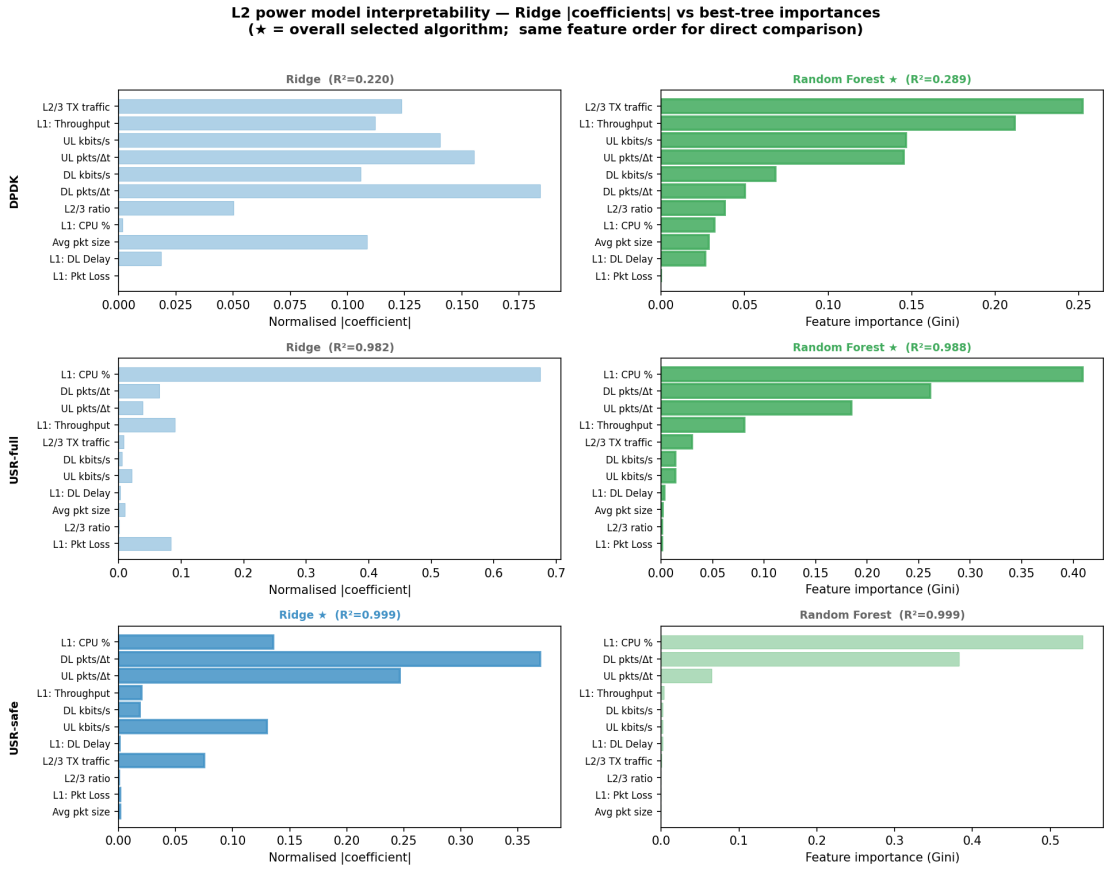


Figure 3.14: L2 power model interpretability for each variant. *Left panels*: Ridge regression normalised absolute coefficients. *Right panels*: feature importances from the best tree-based model (Random Forest or Gradient Boosting). Both panels share the same feature ordering (sorted by tree importance) to allow direct comparison regardless of which algorithm was selected (marked ★). The dominance of L1-predicted CPU and throughput confirms the causal pathway load → CPU → power.

3.11 Summary

This chapter characterised two 5G UPF implementations under controlled load conditions. The principal findings are:

1. **DPDK is power-constant**: total power varies by only 52 mW over the full 0–5 Gbps load range due to the CPU polling loop. This makes power prediction from traffic features statistically intractable.

2. **USR is power-proportional but capacity-limited**: power scales linearly from ≈ 0 to ≈ 3.4 W, but packet loss becomes near-universal above 0.57 Gbps. The empirical low-load reference is $\approx$0.1 Gbps and does not guarantee loss-free operation.

3. **The SEC measurement paradox**: the SEC_{net} metric misleadingly reports near-zero energy efficiency for DPDK because the idle power is almost equal to the load power. Total SEC (SEC_{total}) is the more operationally meaningful metric.

4. **CPU is the strongest legitimate predictor of power** ($r = 0.99$ pooled), followed by delay-related features. Rolling power features are informative but constitute data leakage when used as predictors of power.

5. **Separate per-variant models are necessary**: the two variants occupy distinct regimes with different predictive structures and cannot be modelled jointly without a variant indicator.

6. **The two-layer surrogate achieves high accuracy for USR-UPF**: the `usr_safe` power model reaches $R^2 = 0.999$ (full) and $R^2 = 0.997$ (lite), enabling power prediction from offered load alone without a live UPF.

7. **NetMob compatibility incurs minimal cost**: the lite model, driven by normalised UL/DL loads only, sacrifices $\Delta R^2 < 0.003$ for `usr_safe`, making the twin directly usable with real-world traffic traces.

Not all of these results travel equally far into the rest of the thesis. The digital twin of Chapter 5 uses the lite `dpdk` and `usr_full` surrogates, because a safety-aware evaluator must represent the saturated regime that `usr_safe` omits, together with the USR-UPF operating points of Table 3.2, from which it derives its own decision thresholds. The SEC analysis and the correlation study characterise the two deployments and justify the choice of total attributed power and per-variant models; they are not used again as evaluation metrics. The restricted `usr_safe` model is retained as a low-load power reference.

CHAPTER 4

The Spatiotemporal Traffic Forecaster

4.1 Introduction

In the augmented-observability pipeline, this chapter is the stage that makes the *future* observable: the load a site will face in the next control interval, which is otherwise known only after the decision has been taken. The digital twin of Chapter 5 requires an *offered load* for each candidate UPF evaluation. This chapter supplies that input through the spatiotemporal traffic forecasting stage of the pipeline: a forecaster that predicts near-future traffic for each of the K geographic service areas into which a city's base stations are grouped.

The interface between forecasting, calibration, and control is fixed explicitly. The forecaster emits a *dimensionless* per-cluster load $\hat{d}$. Per base station the normalised demand lies in $[0, 1]$, but a cluster signal is the *sum* over its member base stations, so $\hat{d}$ is bounded by cluster size rather than by unity. Over the deployed $K = 10$ partition, the observed maximum is $\hat{d} \approx 5.8$. The mapping to physical Gbps is the scenario constant α owned by the twin (Chapter 5, Eq. 5.1); it is applied *after* forecasting and never enters training. Control decisions are made downstream, where controllers consume the forecast and its error bar (Chapter 6). Forecasting is independent of physical load calibration, allowing the same predictions to be evaluated under different deployment scenarios through the interface in Section 4.5.

The contribution of this chapter is an evidence-based answer to two design choices that must be settled before a forecaster can be connected to the twin. The first is spatial granularity: whether to aggregate base stations into K service areas and forecast the cluster signal directly (*cluster-first*), or to forecast every base station and sum the results. Section 4.11 answers this with a controlled ablation and finds the cluster-first route substantially better, because aggregation denoises sparse per-station traffic and creates predictability. The second is model selection. Section 4.7 benchmarks six models, from

a linear baseline to established spatiotemporal graph networks, under a common protocol and finds that the field is tight: a simple, inexpensive model loses little to the more complex alternatives. The reusable substrate behind both results is the cluster-first pipeline itself. On this problem, the relevant predictability improves chiefly through aggregation, with the inter-cluster graph adding a smaller further gain. Section 4.12 shows that geography supplies deployable service areas rather than a forecasting advantage over random groups. Forecast accuracy is therefore part of the system contract rather than a private modelling metric: the twin sizes its anti-flapping band directly from the forecast error, so forecast quality propagates into control behaviour (Section 4.5).

The chapter proceeds as follows. Section 4.2 builds the forecasting graph from raw NetMob traffic. Section 4.4 introduces the forecasting models. Section 4.5 states the output contract consumed by the twin. Section 4.6 defines the evaluation methodology and the skill metric. Section 4.7 benchmarks the forecasting models and selects one; Sections 4.9–4.10 characterise the chosen model across cluster counts and traffic types; and Section 4.11 validates the cluster-first decision itself as the capstone. Section 4.14 summarises.

4.2 Data and Spatial Substrate

Before any forecasting model is involved, three steps turn raw operator measurements into a signal on a graph: the measurements are normalised, attributed to base stations, and the base stations are connected into an adjacency graph. This section documents those steps and characterises the result, because the properties established here — how sparse a single station is, how fast correlation decays in space — are what the design decisions of the following sections rest on.

4.2.1 Dataset and normalisation

Dataset. The forecaster is driven by NetMob 2023, the same source the twin's NetMob adapter targets: per-tile downlink traffic for Lyon over 77 days (2019-03-16 to 2019-05-31) at 15-minute resolution (96 slots/day; $T = 7388$ slots, one hour being absent at the daylight-saving change), for the services Netflix, YouTube, and DailyMotion. The spatial unit is a 100 m × 100 m tile; the Lyon footprint comprises 54 013 tiles covering 539 km^2. The raw values are privacy-preserving aggregate indicators with *no physical unit*; this is the structural reason the pipeline is normalised and the twin needs α.

Normalisation. Tile traffic is aggregated to base stations and divided by a single global maximum computed over all services,

$$\hat{d} = \frac{\text{bs_tile_sum}}{\text{global_max}}, \qquad \text{global_max} = 1.482 \times 10^8, \tag{4.1}$$

so that $\hat{d} \in [0, 1]$ for every base station. A *single* normaliser (rather than one per service) is used precisely so that the cross-service ratios are preserved and cluster signals can be formed by direct summation: the mean city-wide load of Netflix (4.39) is close to that of YouTube (4.28) and $\approx 4.7\times$ that of DailyMotion (0.94), differences the normalisation retains. Note that only the *per-station* load is bounded by one; a cluster signal is a sum over member stations and routinely exceeds it (Section 4.3).

Data quality. The trace contains one collection outage, on 12 May 2019, during which the recorded load is near zero from 01:00 to 18:00. The series is used as recorded. Under the chronological split of Section 4.6 the outage falls entirely inside the validation window: it touches neither the training data nor any reported test metric, and it perturbs the early-stopping criterion identically for every model.

4.2.2 From tiles to base stations: Voronoi attribution

NetMob reports traffic per tile, whereas the network is operated per site. Tiles are attributed to sites with a Voronoi tessellation built from the 965 distinct gNodeB locations inside the Lyon footprint (ANFR Cartoradio; co-located antennas are merged). The tessellation is computed in a local metric projection and each cell is clipped to the city footprint; a tile is assigned to the cell that contains its centroid, and a station's load is the sum over its tiles. The Voronoi cell is the standard zero-information approximation of a coverage area: in the absence of antenna azimuths, powers and propagation data, it assigns every location to its nearest site.

Figure 4.1(a) shows the resulting cells. Their size varies by nearly three orders of magnitude, following the site density: the median cell covers 0.34 km^2 (34 tiles), the mean 0.56 km^2, and the largest peripheral cell 7.5 km^2 (Figure 4.2(a)). Traffic is concentrated where the cells are smallest: the load density of the central cells exceeds that of the periphery by up to two orders of magnitude.

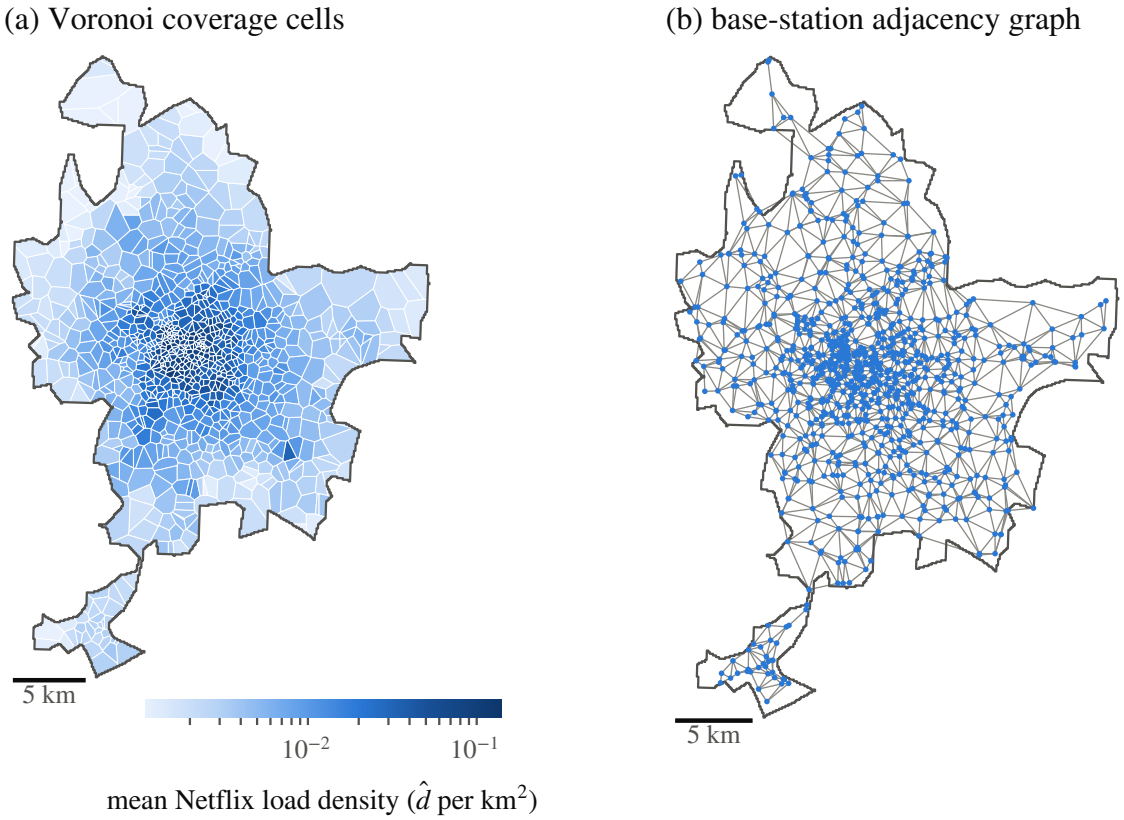


Figure 4.1: Spatial substrate. (a) Voronoi coverage cells of the 965 base stations, clipped to the Lyon footprint and shaded by mean Netflix load density (logarithmic scale). (b) The base-station adjacency graph: two stations are connected when their cells share a border (2 772 edges).

4.2.3 The base-station graph

Two base stations are adjacent when their clipped Voronoi cells share a border of at least 1 m; the edge is weighted by the inverse of the distance between the two sites. The resulting graph (Figure 4.1(b)) has 965 nodes and 2 772 undirected edges and is connected. As expected of a planar tessellation its degree distribution is narrow (mean 5.7, median 6, range 1–11; Figure 4.2(b)), while edge lengths are right-skewed (median 0.60 km, 90th percentile 1.48 km, maximum 4.1 km; Figure 4.2(c)), again reflecting the dense centre and sparse periphery. This graph plays two roles: it is the contiguity constraint for the clustering of Section 4.3, and it is the fine graph from which every coarse forecasting graph is derived.

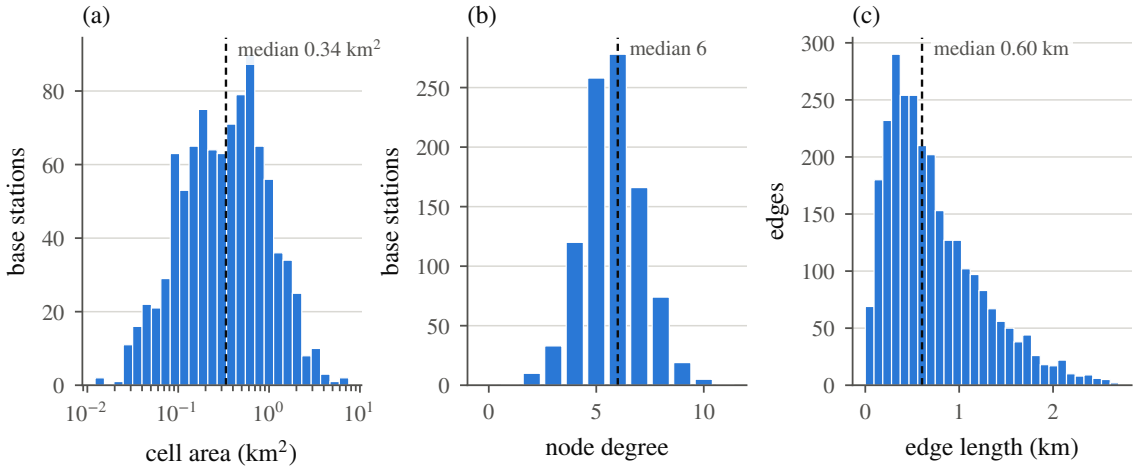


Figure 4.2: Statistics of the spatial substrate: (a) Voronoi cell area (logarithmic axis), (b) node degree, and (c) edge length of the base-station adjacency graph. Dashed lines mark medians.

4.2.4 Traffic characterisation

Four properties of the signal, summarised in Figure 4.3, shape everything that follows.

The city-level signal is strongly diurnal. All three services share a night-time trough near 05:00 and differ in their daytime shape (Figure 4.3(a)): Netflix peaks sharply in the late evening at roughly twice its mean, YouTube plateaus through the afternoon and evening, and DailyMotion shows a pronounced weekday midday peak. Weekend profiles rise later in the morning.

A single base station is sparse. For Netflix, 88% of all station-slots carry $\hat{d} < 0.01$, and the median station is below that level 93% of the time; for DailyMotion the figure is 99% (Figure 4.3(b)). Per-station traffic is thus a sequence of isolated bursts on a near-zero floor.

Aggregation restores temporal structure. The autocorrelation of a typical station drops to 0.58 after one 15-minute slot and reaches only 0.24 at the 24-hour lag; for the ten-cluster signals of Section 4.3 the corresponding values are 0.80 and 0.60, and for the city aggregate 0.96 and 0.86 (Figure 4.3(c)). The daily rhythm that makes traffic forecastable is barely present at a single station and emerges only as stations are summed — the observation that the ablation of Section 4.11 turns into a controlled result.

Correlation is local. The median correlation between two stations is 0.74 when they are adjacent, 0.44 at two hops and 0.31 at three, and reaches the city-wide background level of ≈ 0.23 by about six hops (Figure 4.3(d)). Spatial dependence therefore exists, is

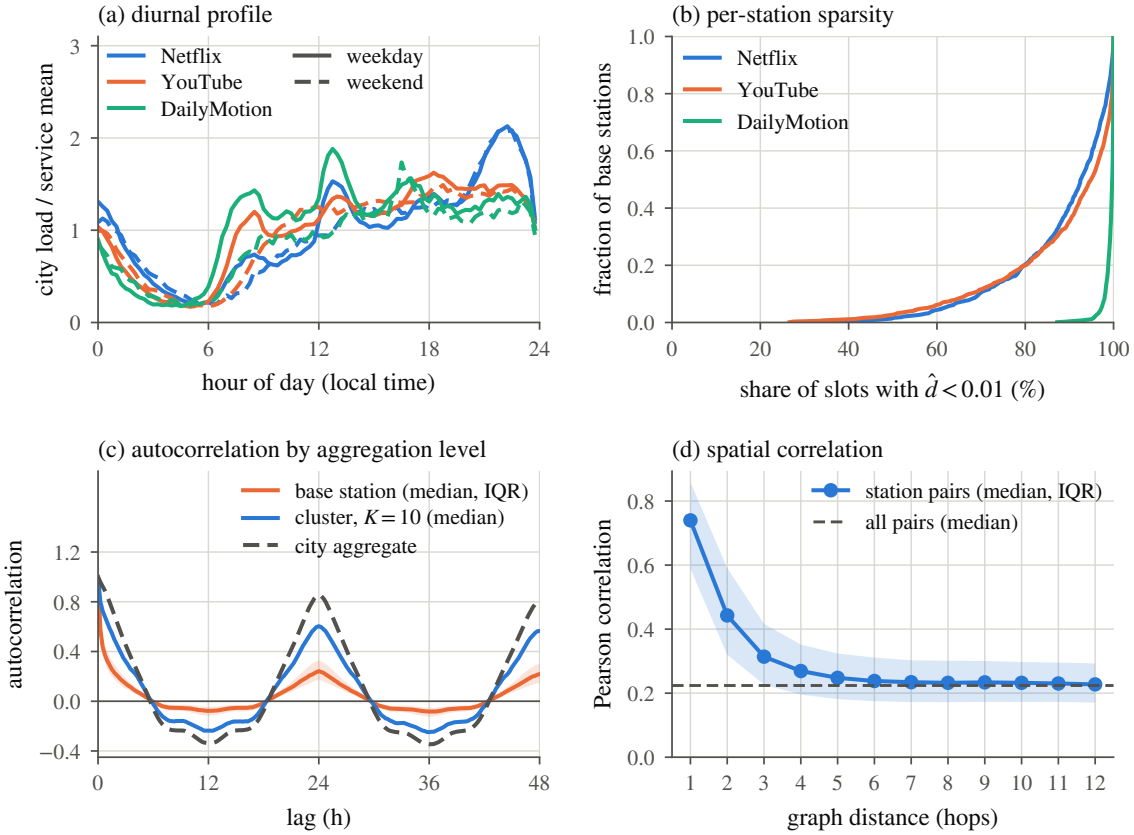


Figure 4.3: Traffic characterisation. (a) Mean diurnal profile of the city-wide load per service, relative to the service mean (solid: weekdays, dashed: weekends). (b) Empirical distribution over base stations of the share of slots with $\hat{d} < 0.01$. (c) Autocorrelation of the Netflix signal at three aggregation levels. (d) Pearson correlation between the Netflix series of two base stations as a function of their hop distance in the adjacency graph (median and inter-quartile range).

short-ranged, and is aligned with the Voronoi adjacency — which motivates both grouping *neighbouring* stations and giving the forecaster a graph.

4.3 Service-Area Clustering

With 965 base stations, neither per-station control nor — as Section 4.11 will show — per-station forecasting is viable. The stations are therefore grouped into K service areas, each of which is the catchment of one edge UPF site and one node of the forecasting graph.

4.3.1 Requirements and method

A partition into service areas must satisfy three requirements that follow from the deployment rather than from forecasting. (i) *Contiguity*: a service area is the set of cells anchored to one user plane function, so it must be a single connected region. (ii) *Compactness*: the distance from a site to its UPF bounds the transport latency, so areas should be geographically tight. (iii) *Service independence*: the partition describes infrastructure and must not change with the traffic type being forecast.

These requirements are met by SKATER [48], a divisive regionalisation method based on pruning a spanning tree of the adjacency graph. A minimum spanning tree of the base-station graph is built, with edges weighted by inter-site distance; removing an edge from a tree splits it into two connected subtrees, so any set of $K - 1$ edge removals yields K contiguous regions by construction. The edges are removed greedily: at each step the method cuts the edge that most reduces the within-region sum of squared deviations of the site coordinates from their region centroid,

$$\mathrm{SSD} \;=\; \sum_{k=1}^{K} \sum_{i \in C_k} \left\| \mathbf{x}_i - \bar{\mathbf{x}}_k \right\|^2, \tag{4.2}$$

subject to both resulting regions containing at least three base stations. The procedure is deterministic and uses coordinates only, so requirement (iii) holds trivially. Because each step splits exactly one existing region, the partitions are *nested* across K: the service areas at $K{+}1$ refine those at K, which makes results at different K directly comparable. One implementation detail should be noted: distances and deviations are evaluated in geographic coordinates (degrees) rather than in a metric projection, which at Lyon's latitude under-weights east–west separations by a factor of about 0.70 relative to north–south ones.

Two alternatives were considered. Plain k-means on site coordinates yields compact areas but ignores the adjacency graph, so contiguity is not guaranteed; it is kept as a comparison in Section 4.12. An attributed spectral clustering that mixed geographic affinity with load-magnitude and temporal-shape features was explored in an earlier iteration and abandoned: the traffic-feature terms produced spatially scattered areas and singletons, and made the partition depend on the service, violating requirements (i) and (iii).

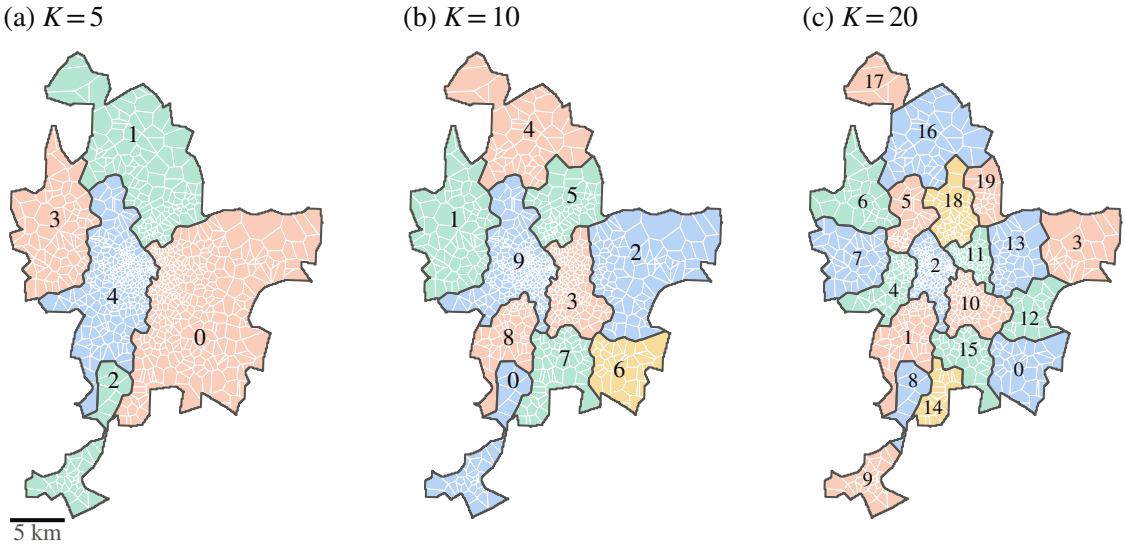


Figure 4.4: SKATER service areas for $K = 5$, 10 and 20. Thin white lines are Voronoi cells, dark lines service-area boundaries; numbers are cluster identifiers. Colours only separate neighbouring areas. The partitions are nested: each area at a larger K lies inside one area at a smaller K.

4.3.2 Resulting service areas

Figure 4.4 shows the service areas for three values of K, and Table 4.1 summarises their structure over the full range studied. Every cluster is connected at every K, and compactness improves steadily, the mean distance from a station to its cluster centroid falling from 4.1 km at K=4 to 0.7 km at K=100.

Two properties of the partition are relevant downstream, because they propagate to the twin and the controllers.

The service areas are compact but not balanced. SKATER minimises geographic dispersion, not load or membership imbalance, and Lyon's site density is strongly peaked at the centre. At K=10 (Figure 4.5) the central cluster contains 297 of the 965 stations and carries 36% of the load, with a mean of $\hat{d} = 1.57$ and a 99th percentile of 4.3, while the two smallest peripheral clusters (36 and 42 stations) average 0.14 and 0.10; the coefficient of variation of cluster load is close to one at every K. This imbalance is a feature of the city, not an artefact to be removed: it is what makes the downstream configuration decision non-trivial, since clusters differ by an order of magnitude in the capacity they require. It also means that pooled error metrics are dominated by the few heavy clusters, which is why Section 4.8 reports errors per cluster as well.

Table 4.1: Structure of the SKATER partition as a function of K (Netflix load). Radius is the mean distance of a station to its cluster centroid; load CV is the coefficient of variation of mean cluster load; "largest" columns give the share of all base stations and of all load held by the largest cluster; $\bar{\rho}_{\text{in}}$ is the mean pairwise correlation between stations of the same cluster (0.24 for a random partition with the same cluster sizes, at every K).

	stations per cluster	largest (%)		radius	load	coarse	
K	min / median / max	stations	load	(km)	CV	edges	$\bar{\rho}_{\text{in}}$
4	42 / 244 / 436	45	49	4.06	0.75	5	0.31
6	42 / 117 / 357	37	43	3.64	0.86	9	0.30
8	42 / 88 / 357	37	43	3.06	1.02	14	0.32
10	36 / 70 / 297	31	36	2.73	0.99	18	0.33
15	10 / 55 / 191	20	26	2.19	1.04	28	0.37
20	6 / 39 / 191	20	26	1.84	1.12	40	0.40
30	6 / 24 / 101	10	12	1.50	0.88	67	0.44
50	6 / 17 / 70	7	8	1.11	0.89	121	0.50
100	3 / 8 / 34	4	5	0.70	0.92	258	0.61

Geographic clusters are more coherent than chance, increasingly so as they shrink. The mean correlation between two stations of the same cluster rises from 0.31 at K=4 to 0.61 at K=100, against 0.24 for size-matched random groupings at every K (Table 4.1). This is the short-range correlation of Figure 4.3(d) seen through the partition: the tighter the area, the more its members move together.

4.3.3 Coarsening to the forecasting graph

Each cluster becomes one graph node. Two clusters are connected when any member base station of one is Voronoi-adjacent to any member of the other, and the edge weight is the mean geographic affinity (a Gaussian kernel on site coordinates) over those cross-border station pairs; self-loops of unit weight are added. The node signal at each slot is the *sum* of the member stations' $\hat{d}$,

$$\hat{d}_k(t) \;=\; \sum_{i \in C_k} \hat{d}_i(t), \tag{4.3}$$

so traffic is conserved exactly: the cluster signals sum to the city total at every slot, which the pipeline verifies numerically. The result is a K-node graph and a $(K \times T)$ signal matrix. The coarse graph is sparse — 18 edges at K=10, a mean degree of 3.6 — and remains so as K grows (Table 4.1). The complete pipeline is summarised in Figure 4.6.

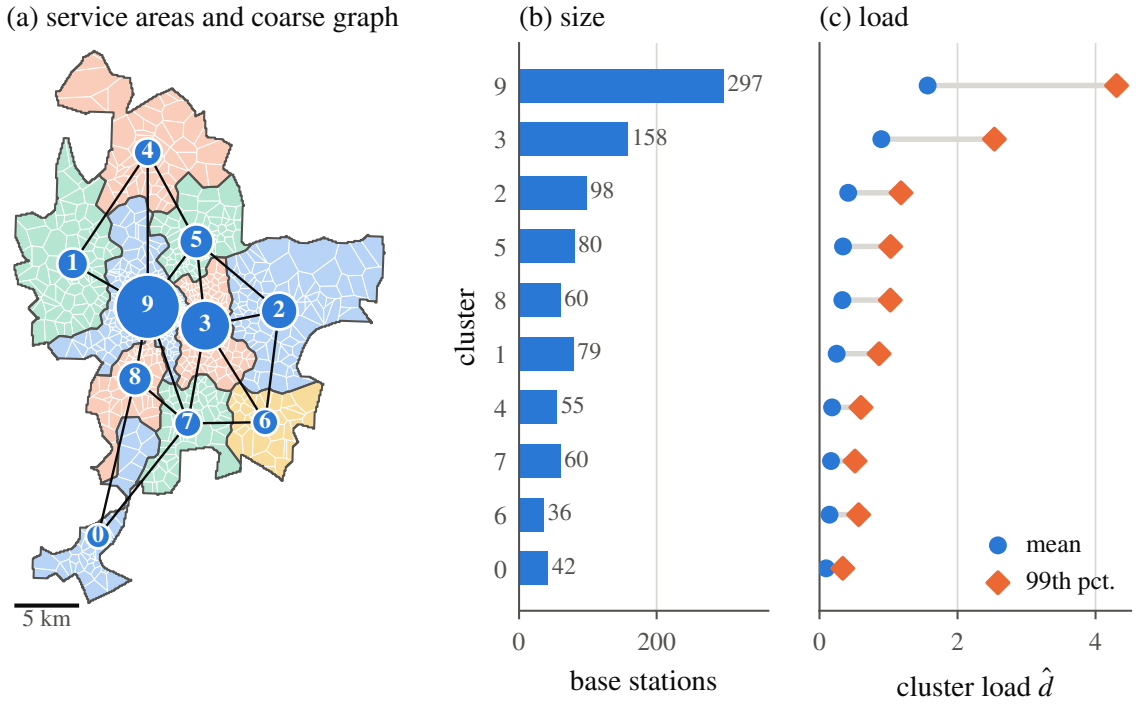


Figure 4.5: The K=10 partition used as the reference configuration. (a) Service areas with the coarse forecasting graph overlaid; node area is proportional to mean cluster load, and an edge joins two clusters that share a border. (b) Number of base stations and (c) mean and 99th-percentile Netflix load per cluster, ordered by decreasing mean load.

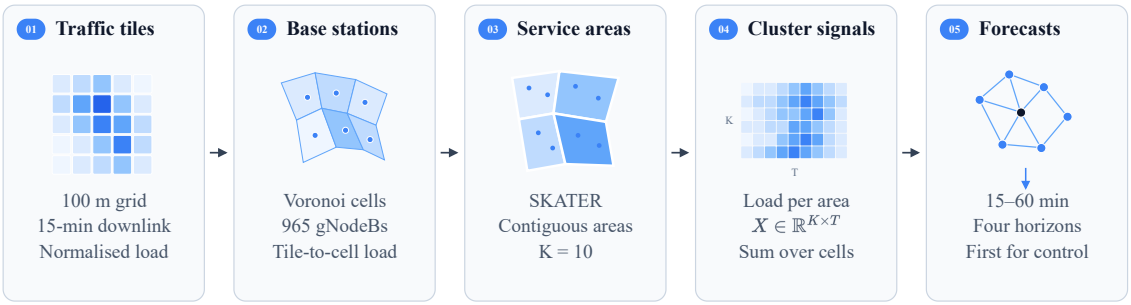


Figure 4.6: Cluster-first forecasting pipeline. Traffic tiles are assigned to base stations, partitioned into spatially contiguous service areas, and summed into cluster time series. The coarse graph and these signals are used to predict four 15-minute horizons; the first horizon supplies the aligned forecast used by the controller.

4.4 Forecasting Models

Every model predicts, for all nodes simultaneously, the next $H = 4$ slots (1 h) from the previous $L = 96$ slots (24 h): input (B, L, N) of normalised loads (one feature per node) plus, for graph models, the static graph (edge_index, edge_attr); output (B, H, N),

with $N = K$ clusters at the cluster level. To compare forecasters without favouring any paradigm, the benchmark of Section 4.7 spans, under one training protocol:

Naive baselines. *Persistence* repeats the last value, and *seasonal-naive* repeats the same slot 24 h earlier. These are the references against which the skill metric is measured.

DLinear [28]: a channel-independent trend/seasonal decomposition with two linear maps, used as a simple baseline for testing whether a deep architecture is needed.

GRU (no graph) : a per-node gated recurrent encoder. It is the purely temporal model and the natural graph ablation, since any graph model's edge over it estimates the value of spatial information.

DCRNN [25]: diffusion-convolutional recurrence, with spatial mixing fused into a GRU cell over the given graph.

Graph WaveNet [26]: dilated temporal convolutions with a learnable adjacency.

AGCRN [27]: a node-adaptive GRU over an adjacency the model *learns* from data, ignoring the supplied graph. This tests whether a better graph than the Voronoi construction can be learned from the data.

GAT–GRU (in-house) : a spatial-first encoder applies a linear input projection and two graph-attention layers (4 averaged heads) at every historical time step. A shared per-node GRU then encodes the resulting sequence, and a linear head maps its final state to the four forecast horizons. The GAT layers use ELU activations, residual connections and layer normalisation. This is the `ClusterSTGNN` implementation selected by the benchmark harness; it is included as an in-house baseline, not as a proposed method.

The graph models share the same coarse K-node graph; the GRU and DLinear ignore it. All are *compact reimplementations on a common harness*: results reflect each architecture's inductive bias under identical data, budget, and protocol rather than its best-tuned reference performance. This is the appropriate basis for a controlled comparison.

4.5 Outputs and the Twin Data Contract

For a chosen (service, K), the forecasting stage provides the twin with predictions, reference loads, spatial assignments, and an error estimate. These comprise the test-window

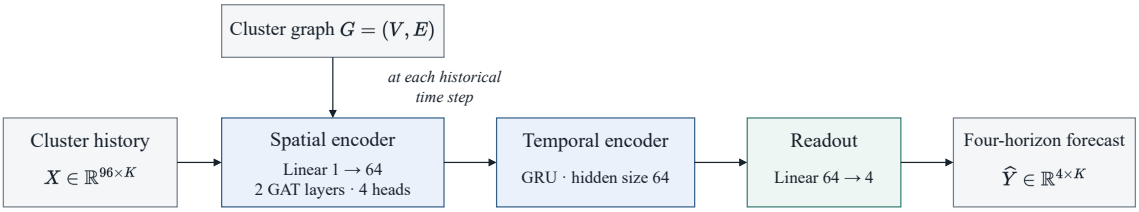


Figure 4.7: In-house spatial-first GAT–GRU forecaster, shown for one input window with the batch dimension omitted. The input contains 96 historical samples per cluster. A shared linear projection and two GAT layers encode the cluster graph at each time step; a per-cluster GRU integrates the resulting sequence, and a linear readout predicts the next four 15-minute samples. Hidden representations have width 64. This diagram describes the in-house baseline, not the Graph WaveNet architecture used in the model comparison and downstream forecasting experiments.

predictions and targets, the full cluster signal, the cluster→base-station map and assignment table, the base-station coordinates, and an evaluation summary. The arrays carry the summed normalised demand $\hat{d}$ (per-base-station values lie in $[0, 1]$; cluster sums exceed it); the twin applies α. The reported error is carried with the forecast so that its effect on decisions can be assessed; Section 4.13 shows that the relevant measure is the decision-flip rate near the threshold rather than the aggregate MAE.

4.6 Evaluation Methodology

Protocol. The data is split chronologically (70/15/15 train/val/test; no shuffling) to prevent leakage. Models train with Adam, an L1 (MAE) objective, cosine learning-rate decay, and early stopping on validation MAE (patience 15). Unless stated, results are averaged over three seeds.

Metrics. Forecasts are scored in $\hat{d}$ units with MAE, RMSE, and WAPE,

$$\mathrm{WAPE} = 100 \cdot \frac{\sum |\, y - \hat{y} \,|}{\sum |y|}, \tag{4.4}$$

which is used instead of MAPE because traffic is sparse and MAPE explodes on near-zero denominators.

Skill, not raw error. Absolute error is misleading across graph sizes: as K grows, each cluster aggregates fewer base stations, so its signal is burstier and *intrinsically* harder

Table 4.2: Model comparison at K=10 and K=50 (Netflix, seed-averaged over 3 seeds). Skill is over persistence (higher is better); WAPE in %. Best skill per column in bold; training time ranges over the two graph sizes and seeds. All models are compact reimplementations under a common protocol.

		K=10		K=50		
model	params	WAPE	skill	WAPE	skill	train (s)
persistence	n/a	29.5	0.000	44.4	0.000	n/a
DLinear	0.8k	23.6	0.199	36.2	0.185	4–7
GRU (no graph)	13k	22.6	0.234	34.8	0.217	15–60
Graph WaveNet	78k	22.0	0.253	34.2	0.229	50–170
DCRNN	75k	21.9	**0.258**	33.9	**0.236**	650–1040
AGCRN	252k	22.3	0.243	34.1	0.233	1670–1790
GAT–GRU (in-house)	60k	23.0	0.219	35.0	0.211	120–680

in relative terms even though its absolute magnitude shrinks. The model-independent quantity is therefore *skill* over a baseline,

$$\text{skill} = 1 - \frac{\text{WAPE}_{\text{model}}}{\text{WAPE}_{\text{baseline}}}, \tag{4.5}$$

reported against persistence (last value) and seasonal-naive (same slot 24 h earlier); a historical-mean baseline is also computed. Skill isolates what the model contributes from how hard the target intrinsically is.

4.7 Forecaster Benchmark

The system places one forecaster per MEC cluster (Section 4.2), so the first operative question is: which model best forecasts at the cluster level? Table 4.2 and Figure 4.8 benchmark six learned forecasters (Section 4.4) under one protocol at $K \in \{10, 50\}$ on Netflix.

Three findings follow. First, spatial information helps, but the gain is modest. Every model that uses the given graph or learns one, namely Graph WaveNet, DCRNN, and AGCRN, beats the graph-free GRU at both sizes (0.253/0.258/0.243 vs. 0.234 at K=10), corresponding to a repeatable ∼3% WAPE reduction. The gain is small, does not grow with K, and *learning* the adjacency with AGCRN does not beat the given Voronoi graph. The most economical explanation is that the residual spatial signal at cluster level is weak, rather than that the Voronoi graph is optimal. A single adaptive-adjacency model failing

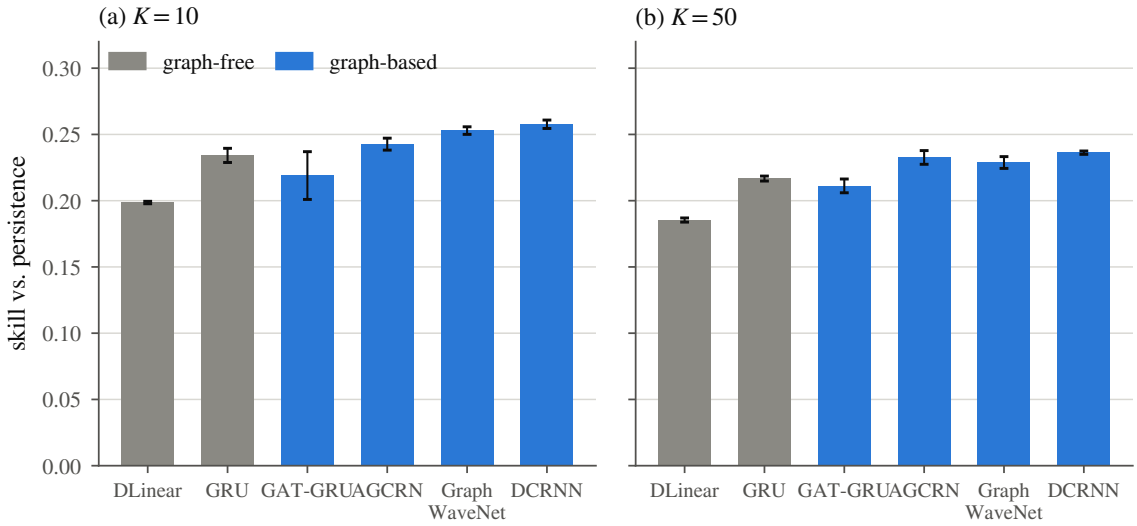


Figure 4.8: Skill over persistence of the six learned forecasters at K=10 (left) and K=50 (right); error bars are ±1 seed std. The whole field spans only ~0.06 skill: the graph models lead the graph-free GRU by a small margin, and even a linear model is close behind.

to improve on it is evidence against an *easily learned* better graph, not proof that none exists. Aggregation already smooths station-level traffic, leaving a modest further gain for the inter-cluster graph. The partition control in Section 4.12 distinguishes this smoothing from geographic locality.

Second, the benchmark is tight and simple models lose little. From DLinear (0.199) to the best model (0.258), the entire spread is ~0.06 skill, and the no-graph GRU (0.234) is within ~0.02 of the best. Model choice therefore has limited effect at this granularity.

Third, the accuracy–cost comparison favours simpler models. DCRNN leads but is among the slowest; Graph WaveNet matches it to within 0.005 skill at roughly a tenth of the cost, because its dilated convolutions parallelise where DCRNN's recurrence does not. AGCRN is the costliest model (252k parameters, ~28 min/run) and brings no gain, and the in-house GAT–GRU does not lead. Graph WaveNet is therefore adopted as the deployed forecaster because it offers the best accuracy–cost trade-off; a plain GRU would be a defensible simpler choice. The rest of the chapter characterises Graph WaveNet, and Section 4.11 shows that forecasting granularity dominates every model difference here.

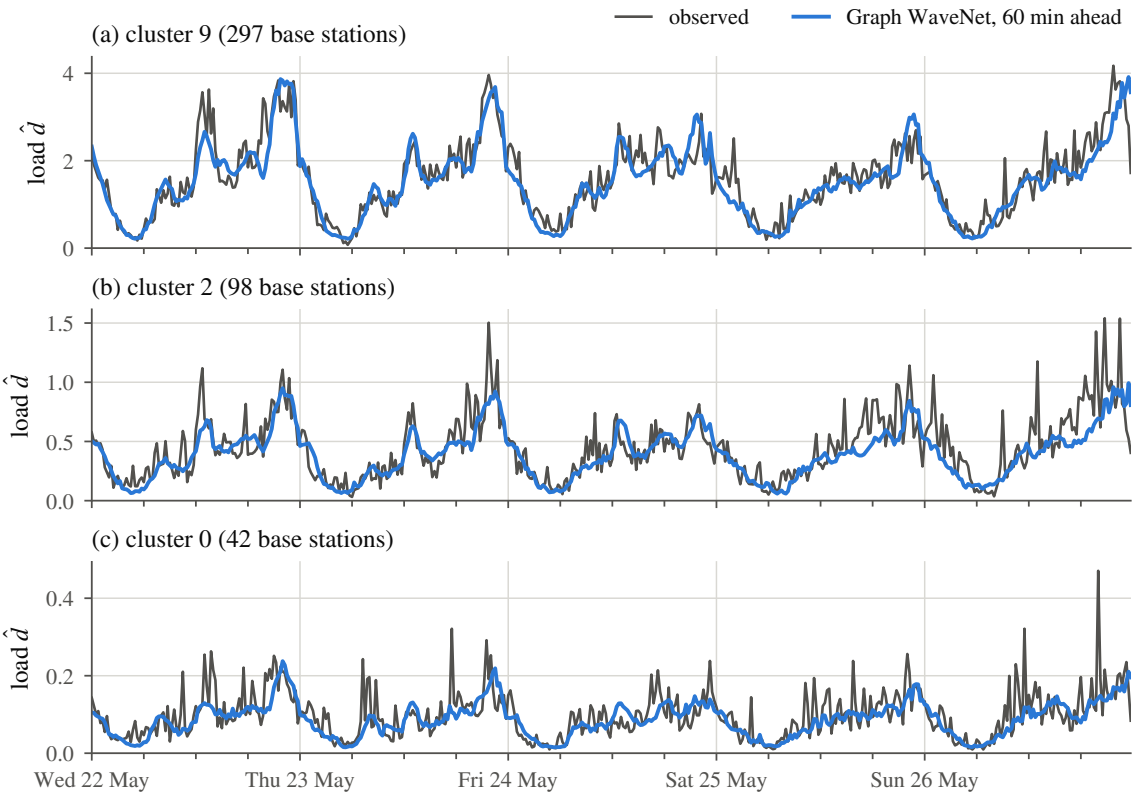


Figure 4.9: Observed and forecast load, 60 minutes ahead, over five days of the test window for the largest, a mid-sized and the smallest-load cluster of the K=10 partition (Graph WaveNet, one seed). The forecast tracks the diurnal envelope at every scale; short spikes, increasingly prominent as the cluster shrinks, are not anticipated.

4.8 Anatomy of the Forecast Error

The benchmark compares models through one pooled number. A controller, however, consumes one forecast per cluster, at a given lead time, and is most sensitive to errors when load is high. This section therefore opens up the pooled figure for the selected model at the reference configuration (Graph WaveNet, K=10, Netflix; three seeds, independently retrained with test predictions retained: pooled WAPE 22.1%, skill 0.249, consistent with Table 4.2). Figure 4.9 shows representative forecasts and Figure 4.10 the three breakdowns.

The advantage is uniform across clusters. Raw WAPE varies widely between clusters, from 16% for the central cluster to 40% for the smallest-membership one, mirroring the size effect of Section 4.9. Skill does not: it lies between 0.22 and 0.29 for all ten clusters (Figure 4.10(a)), with no cluster on which the model fails to improve on persistence and no

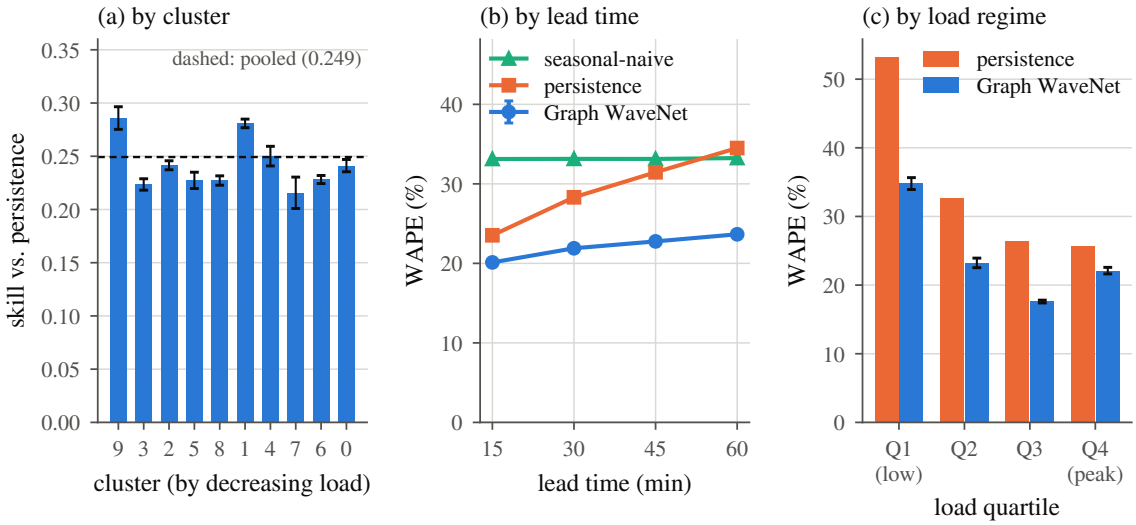


Figure 4.10: Breakdown of the Graph WaveNet test error at $K=10$ (Netflix; mean over three seeds, error bars ±1 std). (a) Skill over persistence per cluster, clusters ordered by decreasing mean load. (b) WAPE by lead time, against both naive baselines. (c) WAPE by quartile of the realised load, computed per cluster.

systematic dependence on cluster load. The pooled figure is therefore representative, not an average over good and bad areas, despite the load imbalance documented in Section 4.3.

The advantage grows with lead time. Persistence degrades quickly as the horizon lengthens (WAPE 23.5% at 15 minutes, 34.5% at 60), whereas the model degrades slowly (20.1% to 23.7%), so skill rises from 0.14 at 15 minutes to 0.31 at 60 (Figure 4.10(b)). At 60 minutes persistence is already worse than the seasonal-naive forecast. The model is thus most valuable exactly where a controller needs it: a configuration change has to be decided ahead of the load it will serve, so it is the far end of the horizon, not the next slot, that the decision depends on.

Peaks are under-forecast. Relative error is highest in the lowest load quartile (34.8%), where absolute loads are small, and lowest in the third (17.6%); in the top quartile it is 22.1%, against 25.6% for persistence, so the margin over persistence narrows from 0.29–0.35 in the lower three quartiles to 0.14 at peak (Figure 4.10(c)). More important than the size of the error is its sign. The forecast is biased upward at low load (+11% in the bottom quartile) and downward at high load (−17% in the top quartile, −35% over the top 1% of slots): it regresses toward the diurnal envelope and misses the short spikes visible in Figure 4.9. This is the expected behaviour of a model trained with an absolute-error objective, which targets the conditional median of a right-skewed quantity. It matters downstream, because under-forecasting a peak is the costly direction for a

Table 4.3: Graph WaveNet forecast quality vs. cluster count K (Netflix, seed-averaged). Raw WAPE rises and MAE falls with K, both intrinsic effects of shrinking clusters, while *skill* over the baselines stays flat.

K	test WAPE	test MAE	skill vs. persist.	skill vs. seasonal
4	16.4	0.192	0.266	0.333
6	18.9	0.147	0.250	0.328
8	20.5	0.120	0.248	0.329
10	22.0	0.103	0.253	0.337
15	24.8	0.078	0.239	0.333
20	26.7	0.062	0.238	0.334
30	30.4	0.048	0.231	0.335
50	34.2	0.032	0.229	0.338
100	38.8	0.018	0.232	0.347

capacity decision. The twin's hysteresis band, sized from the forecast MAE (Section 4.5), absorbs part of it; a quantile or asymmetric training objective would address it at the source and is the most direct improvement left open by this chapter.

4.9 Effect of Cluster Count K

The number of MEC clusters K is a deployment parameter that determines how finely the operator partitions the city. Table 4.3 reports how the chosen forecaster, Graph WaveNet, behaves as K grows from a handful of clusters to one hundred (Netflix, 3 seeds).

Across a 25× change in K, Graph WaveNet holds a near-constant ~0.23–0.27 skill over persistence and ~0.33 over seasonal-naive, even as raw WAPE climbs from 16.4 to 38.8 and MAE falls from 0.192 to 0.018. The rising WAPE and falling MAE are two views of one intrinsic effect: smaller clusters aggregate fewer stations, so their signal is burstier in *relative* terms but smaller in *absolute* terms. The flat skill curve shows that the model keeps the same advantage regardless of granularity, which is why skill rather than raw WAPE is the headline metric. For the operator, the practical consequence is that K can be chosen on deployment grounds, such as the number of MEC sites and controller tractability, without paying a forecasting penalty.

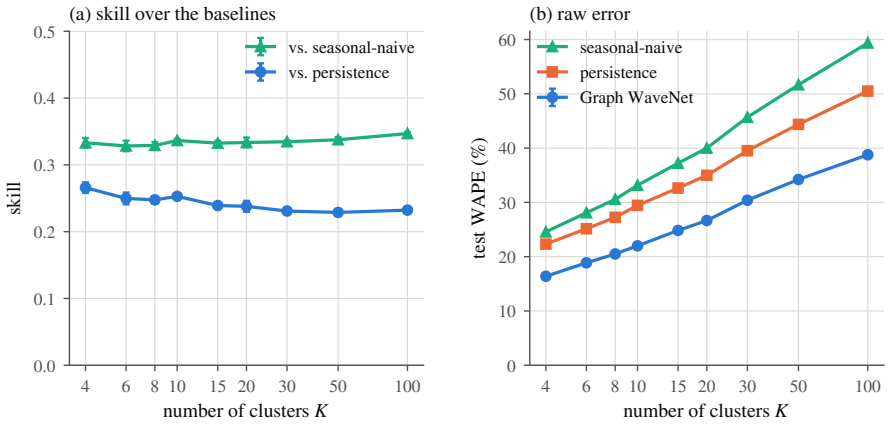


Figure 4.11: Graph WaveNet preserves its skill over the naive baselines from 4 to 100 clusters even as the raw error grows with the intrinsically harder small-cluster signal. The dashed red curve (raw WAPE, right axis) rises while the solid skill curves (left axis) stay flat.

Table 4.4: Graph WaveNet skill over persistence by traffic type (seed-averaged). Skill is positive for every service, and the margin tracks predictability: largest on smooth aggregate traffic and smallest on sparse DailyMotion.

service	K=8	K=10	K=15
total (aggregate)	0.305	0.287	0.281
Netflix	0.248	0.253	0.239
DailyMotion (sparse)	0.140	0.160	0.153

4.10 Generalisation Across Traffic Types

Because SKATER clustering is geographic, the *same* cluster structure can be re-used for a different service, changing only the signal on the nodes. This provides an external-validity check within the same city: whether Graph WaveNet maintains its performance for traffic with a different spatial and temporal profile. Table 4.4 reports skill over persistence for the aggregate of all services (`total`) and the sparse `DailyMotion`, alongside Netflix, at $K \in \{8, 10, 15\}$ (3 seeds, frozen architecture).

Skill is positive for every service and every K, so the result is not Netflix-specific: it holds for aggregate and sparse traffic on the same geographic graph. The margin behaves as predictability would suggest. It is widest on the smooth `total` signal (~0.29), narrower on Netflix, and smallest on the heavily sparse DailyMotion (~0.15 vs. persistence). DailyMotion is the instructive case. Its persistence-skill is low because its traffic is bursty, yet its *seasonal*-skill is the highest of all (~0.37), because the sparse signal is highly repetitive

day to day. The model therefore beats persistence only modestly but beats seasonal-naive strongly. The supported scope is robustness *across traffic types within Lyon*: the services share city, base stations, and period, so this is not evidence of cross-city transfer.

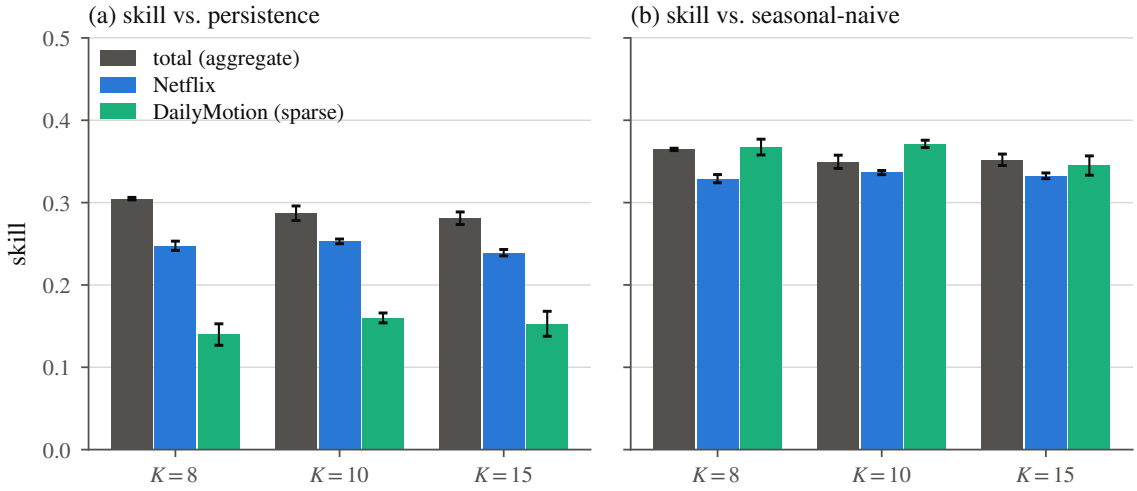


Figure 4.12: Graph WaveNet's advantage over both baselines holds for aggregate and sparse traffic, scaling with the predictability of each service. The seasonal-naive panel (right) shows DailyMotion's skill rising to ~0.37, above Netflix, despite its sparsity.

4.11 Cluster-First versus Per-Station Forecasting

The preceding sections chose a model (Graph WaveNet) and showed it forecasts cluster load robustly across graph sizes and traffic types. These results rest on the decision to forecast at the *cluster* level. The MEC deployment already supplies clusters, but it remains necessary to test whether aggregating before forecasting improves accuracy or merely reduces computational cost. This closing section tests that decision directly.

Section 4.9 cannot answer that question because it scores each K against its own changing target. A controlled test instead fixes the delivered target (the K=10 cluster signal) and compares two routes to it:

Route A (cluster-first). Aggregate to K=10 clusters; forecast the cluster signals directly.

Route B (per-base-station). Forecast all 965 base stations; *sum* the predictions into the same 10 clusters.

Both routes are scored on the identical K=10 target. The 965-station series sums exactly to the cluster series, verified to floating-point error, so the comparison isolates *when*

aggregation happens. Both routes are given the same model capacity, so Route B spreads it over 965 nodes and Route A over 10. The comparison therefore answers the practical question "at a fixed budget, where should aggregation happen?" rather than the idealised one "is per-station forecasting worse in principle?"; a capacity-matched-per-node Route B is not tested here. The route is run with the graph-free GRU and a sparse graph-attention STGNN. The dense-adjacency models (Graph WaveNet, DCRNN, AGCRN) were not run at 965 nodes: their $O(N^2)$ adjacency did not fit the training budget used throughout this chapter. This is a statement about the budget, not about the architectures. The 965-node setting is modest by current STGNN standards, and published work routinely handles graphs of this size with more memory or a sparsified adjacency.

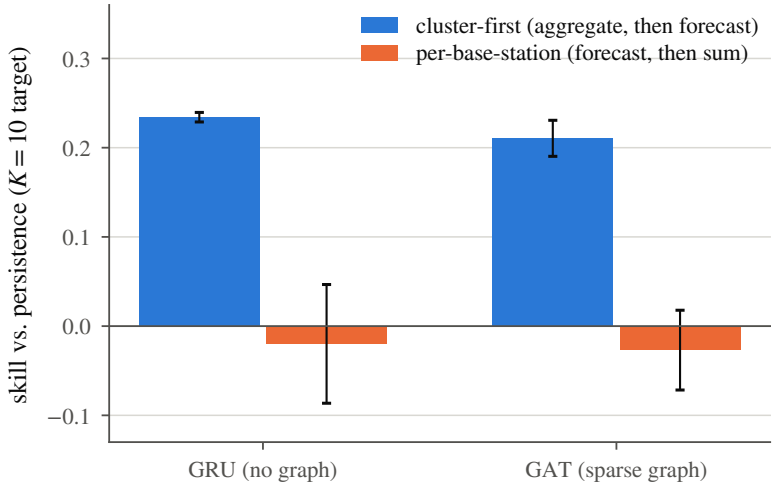


Figure 4.13: Forecasting the aggregated cluster signal directly (cluster-first) versus forecasting 965 base stations and summing (per-base-station), scored on the same $K=10$ target. The per-station route collapses to persistence-level skill (≈ 0); cluster-first reaches 0.21–0.23.

Figure 4.13 shows that forecasting at the base-station level and summing yields a cluster forecast *no better than persistence* (skill −0.02 to −0.03), whereas forecasting the cluster signal directly reaches 0.21–0.23, consistently across both models and all three seeds. The mechanism is *denoising by aggregation*: per-station traffic is extremely sparse (for Netflix, 85–99% of station-slots are below 0.01), so 965 such series are nearly unpredictable individually, and a sum of persistence-level forecasts is itself persistence-level. The *sum*, by contrast, is a smooth, strongly diurnal signal that is easy to forecast.

Aggregation therefore does most of the work: it turns sparse per-station traffic into a predictable signal, which is why the inter-cluster graph adds only ∼3% in Section 4.7 and why no model choice comes close to the gap seen here. Whether *geographic* grouping is

what creates that predictability is a separate question, tested in Section 4.12. Cluster-first forecasting is therefore a forecasting advantage, not merely a tractability convenience. It is the design decision this chapter tests most directly. A far larger per-station model might recover some skill, but the observed gap, 0.22 versus ≈ 0, is too large to assume it would disappear without evidence, and it is consistent with the per-station sparsity.

4.12 Geographic versus Random Aggregation

The ablation shows that stations must be pooled before forecasting. It is tempting to conclude that the *geographic* pooling of Section 4.3 is what creates the predictable signal — that grouping neighbouring, correlated stations is how spatial structure is exploited. That is a separate claim, and it can be tested by holding everything fixed (the 965 station series, K, the models, the protocol) and changing only how the stations are partitioned:

SKATER. The contiguous service areas used throughout this chapter.

k-means. k-means on projected site coordinates: geographic and compact, but with no contiguity constraint.

Random. The SKATER labels permuted over the stations: the same K and exactly the same cluster sizes, but no geography. Three independent permutations are drawn.

Each partition is coarsened by the rule of Section 4.3 and forecast with the graph-free GRU and with Graph WaveNet under the protocol of Section 4.6, at K=10 and K=50 (three seeds per partition; 60 training runs). As a check on the harness, the re-derived SKATER coarsening reproduces the pipeline's stored cluster signals and coarse graph exactly. Table 4.5 and Figure 4.14 report the outcome.

Geography is not what makes the signal predictable. The random partition is not harder to forecast; it is markedly easier. At K=10 Graph WaveNet reaches a WAPE of 14.6% on random groups against 22.2% on the SKATER areas, and 20.8% against 34.2% at K=50. The targets themselves are smoother — persistence alone scores 20.3% on random groups and 29.5% on SKATER areas at K=10 — and the model's skill over persistence is, if anything, higher (0.285 against 0.248). The predictability documented in Section 4.11 therefore comes from *pooling many stations*, not from pooling *neighbouring* ones. The explanation is statistical. A sum of weakly correlated bursty series averages its fluctuations out; a sum of strongly correlated ones does not. Stations in a geographic

Table 4.5: Partition control (Netflix). Structure: number of contiguous clusters, mean station-to-centroid radius, mean correlation between stations of a cluster ($\bar{\rho}_{\text{in}}$) and between cluster signals ($\bar{\rho}_{\text{btw}}$). Forecast: WAPE in %, skill over persistence; medians over runs (3 seeds; 3×3 for the random partitions).

		structure				WAPE		skill	
K	partition	contiguous	radius	$\bar{\rho}_{\text{in}}$	$\bar{\rho}_{\text{btw}}$	persist.	GWN	GRU	GWN
10	SKATER	10/10	2.73	0.33	0.65	29.5	22.2	0.238	0.248
	k-means	10/10	2.58	0.33	0.66	30.1	22.8	0.233	0.244
	random	0/10	6.40	0.24	0.93	20.3	14.6	0.282	0.285
50	SKATER	50/50	1.11	0.50	0.43	44.4	34.2	0.217	0.229
	k-means	50/50	1.06	0.51	0.43	44.8	34.3	0.199	0.235
	random	0/50	6.39	0.24	0.76	27.9	20.8	0.246	0.255

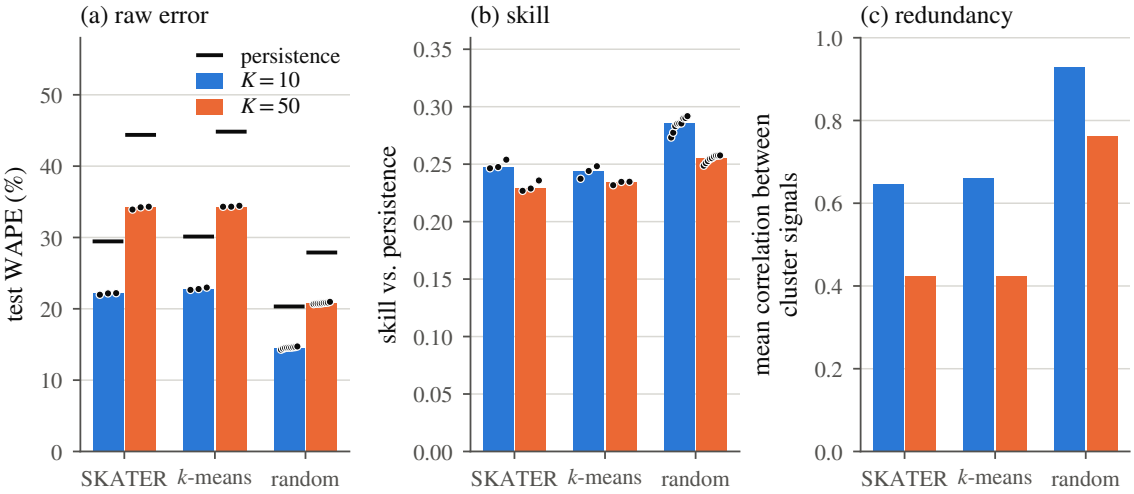


Figure 4.14: Partition control with Graph WaveNet at K=10 and K=50 (Netflix). Bars are medians and dots individual training runs. (a) Raw test WAPE, with the persistence WAPE of the same target as a black tick. (b) Skill over persistence. (c) Mean pairwise correlation between the K cluster signals. Random groupings of the same sizes are easier to forecast and slightly *more* skilful, but their signals are nearly interchangeable.

cluster are positively correlated ($\bar{\rho}_{\text{in}}$ = 0.33 at K=10, 0.50 at K=50, against 0.24 for random groups; recall that adjacent stations correlate at 0.74), so a contiguous area retains local fluctuations that a city-wide sample of the same size cancels. It would therefore be wrong to say that aggregation "exploits" spatial structure: spatial correlation works *against* the smoothing that aggregation provides.

But a random partition forecasts nothing useful. The reason random groups are so smooth is that each one is a scaled-down replica of the whole city: their signals correlate at 0.93 with one another at K=10 (0.76 at K=50), against 0.65 (0.43) for the service areas

(Figure 4.14(c)). Ten such forecasts carry little more than one forecast of the city total, and they do not correspond to anything a user plane function can serve: none of the random clusters is contiguous, and their members lie on average 6.4 km from the centroid, more than twice the 2.7 km of a SKATER area. The quantity the system needs is the load *of a place*, and that is intrinsically harder to predict than the load of a random sample. The difference — about 8 WAPE points at K=10 and 13 at K=50 — is the cost of forecasting a geographically defined area. It is a property of the problem, not a deficiency of the pipeline, and it also accounts for the rise of raw error with K in Section 4.9: smaller areas pool fewer and more strongly correlated stations.

Among geographic partitions the method matters little. k-means and SKATER produce areas of similar compactness and coherence, and their forecast quality differs by no more than 0.006 skill at either size, in opposite directions (Graph WaveNet 0.244 against 0.248 at K=10, 0.235 against 0.229 at K=50). On this site layout k-means also happened to return contiguous clusters in every case. The choice of SKATER therefore rests on what it guarantees by construction — contiguity, and partitions that are nested across K — and not on a forecasting advantage. Finally, the graph is only useful when it means something: Graph WaveNet improves on the graph-free GRU by about 0.01 skill on the two geographic partitions and by 0.003 on the random one at K=10, where the coarse graph is complete and carries no information.

Two caveats apply. The control uses one service and two values of K; and one of the nine GRU runs on a random partition stopped after four epochs (skill 0.03), which is why medians rather than means are reported — every run is shown in Figure 4.14.

4.13 From Forecast Error to Decision Error

Reporting the forecaster's skill in normalised MAE leaves a practical question unanswered: the controller does not consume the forecast as a number, it consumes it as a *decision*, by comparing it with the threshold $\lambda_{\mathrm{dec}} = 81$ Mbps derived in Chapter 5. Under the declared calibration $\alpha = 1$ the horizon-averaged MAE of 110.7 Mbps is *larger* than that threshold, which invites the conclusion that the forecaster cannot resolve the decision it feeds. This section shows why that conclusion does not follow, and reports the metric that does matter.

Which horizon is scored. The model is trained on $H = 4$ steps, but the controller consumes only the first. The two are not interchangeable:

Horizon	Lead time	MAE at $\alpha = 1$
$h = 0$	+15 min	97.6 Mbps
$h = 1$	+30 min	110.1 Mbps
$h = 2$	+45 min	115.0 Mbps
$h = 3$	+60 min	120.0 Mbps
mean over H	n/a	110.7 Mbps

The 110.7 Mbps figure quoted downstream is the mean over all four horizons. The quantity that governs control is the one-step error, 97.6 Mbps.

Decision error. Aggregate MAE is a poor proxy here because the forecast error scales with load, whereas the decision depends only on which side of a *low* threshold the load falls. Applying the threshold rule to the predicted and to the actual load over the 10,090 test cluster-steps:

Actual load band	Share of steps	MAE	Decisions flipped
$[0,\ \lambda_{\text{dec}})$	13.7 %	21.9 Mbps	22.7 %
$[\lambda_{\text{dec}},\ 0.2)$	25.1 %	42.7 Mbps	12.7 %
$[0.2,\ 0.5)$	34.4 %	78.6 Mbps	0.8 %
$[0.5,\ \infty)$	26.8 %	212.2 Mbps	0.0 %
overall	100 %	97.6 Mbps	**6.6** %

Only 6.6 % of decisions differ from those an oracle forecaster would take, split into 3.5 % that select USR-UPF when the actual load did not warrant it, the direction that costs QoS, and 3.1 % that select SD-CORE DPDK unnecessarily, which costs only energy.

The band table explains the discrepancy. The MAE is dominated by high-load steps, where an error of 212 Mbps is large in absolute terms but flips no decisions at all, because both the prediction and the actual load sit far above the threshold. Near the threshold, where a flip is possible, the error is an order of magnitude smaller (21.9 Mbps). Comparing a load-averaged error with a fixed low threshold therefore overstates the difficulty of the decision by roughly a factor of four.

What this does and does not license. It does not make the forecaster accurate: 6.6 % of steps are decided on a wrong premise, and the 3.5 % unsafe-direction share is the mechanism behind the residual QoS violations of the threshold controller in Chapter 6. It

does establish that the appropriate accuracy target for this application is the decision-flip rate near λ_{dec}, not the aggregate MAE, and that hysteresis bands sized as a multiple of the aggregate MAE (Chapter 5) are calibrated against the wrong statistic. This is why the automatic band degenerates.

What the spatial control adds. Aggregating stations before forecasting improves predictability, but the partition control in Section 4.12 shows that geography is not the source of that smoothing advantage. Random groups forecast more easily because they resemble the city total. Geographic service areas are retained because they correspond to a deployable place, and SKATER guarantees their contiguity. The spatial pipeline therefore defines the quantity needed by control rather than maximising forecasting accuracy alone.

4.14 Summary

This chapter built the spatiotemporal forecaster that supplies the digital twin's offered load. The principal points are:

1. **Scope.** The forecaster emits a dimensionless per-cluster load $\hat{d}$: a *sum* of per-station values that each lie in $[0, 1]$, so $\hat{d}$ is bounded by cluster size rather than by unity (observed maximum ≈ 5.8). Physical calibration (α) and control are external. One trained model serves every capacity scenario.

2. **Cluster-first is a forecasting advantage at equal model budget.** A controlled ablation on a fixed target shows that forecasting the aggregated cluster signal directly reaches 0.21–0.23 skill, while forecasting all 965 base stations and summing is no better than persistence (≈ 0). The aggregation *denoises* sparse per-station traffic into a predictable signal. Geographic partitions define local service demand; random partitions are even smoother but do not correspond to deployable areas (Section 4.12).

3. **Spatial structure helps modestly beyond aggregation.** On top of the clusters, the inter-cluster graph adds only a small gain (~3% WAPE; graph models > no-graph GRU), and a model that *learns* its adjacency (AGCRN) does not beat the given Voronoi graph, indicating that the residual spatial signal at cluster level is genuinely small.

4. **Compact models offer competitive accuracy.** Six forecasters from linear to learned-graph span only ~0.06 skill; Graph WaveNet gives the best accuracy–cost trade-off and a plain GRU is near-best, so the system is insensitive to forecaster sophistication. Graph WaveNet is recommended for the twin.

5. **The problem is well-behaved.** Skill over persistence is near-constant across graph sizes ($K = 4 \rightarrow 100$; rising raw WAPE reflects intrinsic small-cluster difficulty, not model decay) and holds across aggregate and sparse traffic types, so the findings are not specific to the Netflix trace; they remain evidence from a single city.

CHAPTER 5

The UPF Digital Twin

5.1 Introduction

Chapter 3 established, from controlled laboratory measurements, *how* two open-source UPF implementations behave under load: SD-CORE DPDK draws a near-constant 0.82 W regardless of traffic, whereas USR-UPF scales its power proportionally to load but begins to lose packets from about 0.17 Gbps and is fully saturated above roughly 0.5 Gbps. Those measurements are the raw material; on their own they do not yet constitute a tool a network controller can query.

This chapter describes the construction of the *digital twin* used for counterfactual UPF evaluation. It is a measurement-grounded, controller-agnostic software component that answers a single well-posed question,

> *Given an offered traffic load and a candidate UPF configuration, what would happen in terms of power, throughput, CPU, packet loss, delay, and QoS safety?*

The twin receives load values from the forecaster and candidate actions from a controller, then returns the predicted consequences of those actions. This interface keeps the pipeline modular: the forecaster owns the load prediction, the twin owns the physics and switching-cost evaluation, and the controllers own the policy that selects a realisation. The same boundary is reflected in the implementation through a reusable software library with separate interfaces for evaluating candidate actions and simulating sequential decisions.

As the counterfactual evaluation stage, the twin combines the UPF-level surrogates fitted in Chapter 3 with the calibrated load interface used by Chapter 4. Runtime control between two UPF realisations requires exactly this comparison: the controller observes the outcome under the running realisation and needs an estimate of the outcome that would follow from switching to the other one. For illustration within this chapter, the

rollout examples use a transparent hysteresis rule (Section 5.8); the learned controllers and clairvoyant reference are developed in Chapter 6.

The remainder of the chapter proceeds as follows. Section 5.2 bridges the profiling results to a predictive surrogate and justifies which trained variants the twin actually uses. Section 5.3 presents the two evaluation modes (stateless physics and stateful rollout) and the data contract. Section 5.4 describes load calibration and the NetMob-compatible input adapter. Section 5.5 defines the predicted KPIs and the safety/efficiency semantics. Section 5.6 formalises the switching-cost model. Section 5.7 shows how operating thresholds are *derived* from the surrogate rather than hardcoded. Section 5.8 defines the evaluation methodology and metrics and demonstrates the assembled twin under the hysteresis controller. Section 5.10 summarises.

5.2 From Profiling to a Predictive Surrogate

Using a learned surrogate as a cheap evaluator inside a decision loop is established methodology [18, 19], and its network-scale form is specified as the digital twin network [5, 20]; Chapter 2 positions this twin against both. The twin's predictive core is the two-layer surrogate fitted in Chapter 3, whose accuracy is reported in Section 3.9. This section develops the structure those results assume, then turns to the design decisions specific to embedding it in an online evaluator.

5.2.1 Two-layer structure

For each UPF variant the surrogate is organised as a stack that mirrors the causal pathway *load* → *performance* → *power*:

Layer 1 (performance). Four independent single-target regressors map the offered load to delivered throughput (Gbps), CPU utilisation (%), downlink packet loss (packets per interval), and downlink one-way delay (μs).

Layer 2 (power). A single regressor maps the offered load *plus* the four Layer 1 outputs to `power_watts`.

Because each Layer 1 slot is selected independently by cross-validated model search, the chosen algorithm differs by target: the near-linear throughput–load relationship is best captured by Ridge regression, whereas the strongly non-linear delay and loss responses

near saturation are best captured by tree ensembles (Random Forest or Gradient Boosting). This heterogeneity is a feature, not an inconsistency: forcing a single algorithm across all slots would underfit the non-linear targets. One practical consequence, visible later in Figure 5.2, is that tree-based slots produce *piecewise-constant* predictions and do not extrapolate beyond the measured envelope, a property the twin must respect when it is queried.

5.2.2 Which variants the twin uses

The profiling campaign trained three variants: `dpdk`, `usr_full` (all USR samples, including the saturated regime), and `usr_safe` (USR samples restricted to the unsaturated regime below λ_{env}). The twin exposes exactly two *actions* to a controller, `DPDK` and `USR`, and maps them to trained variants as follows:

$$\texttt{DPDK} \longmapsto \texttt{dpdk}, \qquad \texttt{USR} \longmapsto \texttt{usr_full}.$$

The choice of `usr_full` over `usr_safe` to represent `USR` is deliberate and central to the twin's purpose. A digital twin whose remit explicitly includes *packet loss, delay, and QoS safety* must be able to represent what happens when USR-UPF is pushed past its safe operating point. `usr_safe` was fitted to a restricted throughput range below λ_{env}, which includes some samples with loss but omits much of the saturated response. Predictions beyond that subset's support are insufficient to characterise the full safety transition. `usr_full` includes the wider observed loss and delay behaviour; its full-feature Layer 1 scores (Table 3.5 protocol) are $R^2 = 0.998$ and 0.968, compared with 0.86 and 0.52 for `usr_safe`. The power score is slightly lower ($R^2 = 0.988$ versus 0.999), but the full-range model is better suited to evaluating both acceptable and unacceptable operating conditions. The restricted-range models are retained as a low-load power reference rather than used in the action map.

5.2.3 Full and lite feature sets

Each variant was trained in two feature regimes. The *full* model accepts all seven offered-load features (DL/UL kbits/s, TX packet counts, average packet size, and the L2/L3 overhead ratio) and targets laboratory or high-fidelity simulation. The *lite* model accepts only the DL/UL load volume and is therefore drivable from datasets such as NetMob 2023 that expose only aggregate per-cell load with no packet-level detail. The twin loads the *lite*

variant exclusively, because its online use case is driven by cluster-level load forecasts that carry no per-packet observability. On the row-level power test of Section 3.9, the accuracy difference is small: for the power model the lite–full gap is $\Delta R^2 \approx 0.024$ for `usr_full`.

5.3 Twin Architecture

The twin offers two evaluation modes over the same surrogate core, reflecting the two questions a consumer may ask (Figure 5.1).

Stateless physics. `evaluate_action(action, load)` returns the predicted KPIs for a single (action, load) point. It is a pure function: no memory, no side effects. This is the mode used by parameter sweeps, the threshold derivation of Section 5.7, and interactive inspection.

Stateful rollout. A *session* wraps the stateless core with the previous action and applies the switching-cost accounting of Section 5.6. `session.step( requested_action, load)` returns the realised KPIs for one time step, including any energy paid for activation. This is the mode used to drive a sequential simulation (and, in Chapter 6, the `step()` of a reinforcement-learning environment).

The twin combines the fitted Layer 1 and Layer 2 surrogate models with the activation-cost parameters. A model registry identifies the fitted models, while a separate configuration specifies the QoS budget, switching-accounting policy, and time-step length. This separation allows the same physical model to support different deployment scenarios and controller comparisons. Controller logic remains external to both evaluation modes.

5.3.1 Interactive front-end

Both modes are also exposed through an interactive dashboard, a thin presentation layer that calls the same twin interface as every other consumer and adds no modelling of its own. Its *Twin Explorer* view is the stateless mode made interactive: the user sets an offered load and reads the predicted KPIs and safety flags of both UPFs side by side. Its *Overview* view is the stateful mode: a rollout of a selected controller over one cluster's test windows is replayed step by step against the always-SD-CORE DPDK baseline, showing the serving UPF, the offered load relative to the decision threshold of Section 5.7, and the

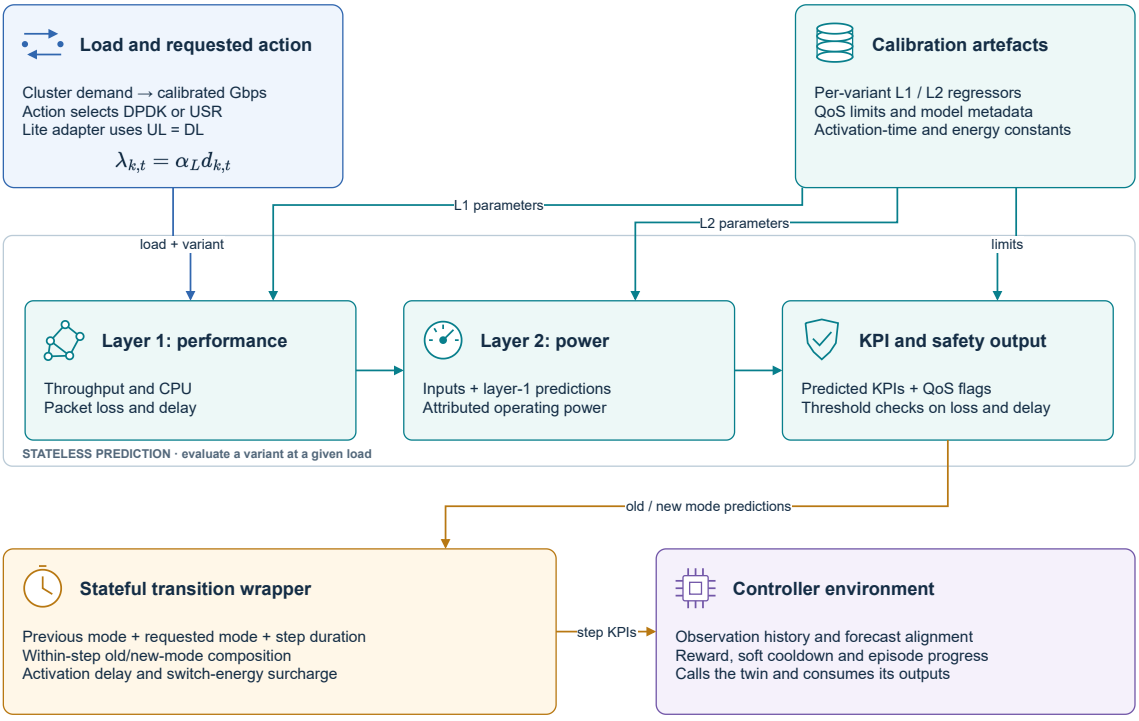


Figure 5.1: Digital-twin architecture. Layer 1 predicts performance KPIs and layer 2 predicts attributed operating power from the load and layer-1 outputs. Stateless evaluation returns these predictions and QoS flags. The stateful wrapper combines old- and new-mode predictions and adds activation and switching-energy accounting. The controller environment remains external and maintains observation history, rewards, cooldown and episode progress. The load scale α_L shown in the diagram denotes the calibration factor α defined in this chapter.

resulting power and QoS traces. The replay runs in *shadow mode*: it shows what the twin predicts a controller would have done on recorded traffic, and actuates nothing.

The dashboard is included as an illustration of the augmented observability described in Section 5.1, not as a source of results. Every quantitative statement in this chapter is taken from the scripted rollouts of Section 5.8, which aggregate over all clusters; the dashboard shows one cluster at a time and its headline figures are not comparable to the metrics defined there.

5.4 Inputs and Load Calibration

5.4.1 Normalised load to physical load

The forecasting stage emits, for each cluster, the *sum* of the normalised per-base-station demand over the cluster's members. Each per-station value lies in $[0, 1]$, so the cluster signal $\hat{d}$ is bounded by the cluster size, not by unity: over the deployed $K = 10$ partition it ranges up to $\hat{d} \approx 5.8$. A single global calibration constant α converts it to a physical offered load in Gbps:

$$\lambda_{\text{Gbps}} = \alpha \cdot \hat{d}, \qquad \alpha = 1.0 \text{ Gbps per normalised unit.} \tag{5.1}$$

The value of α is a scenario parameter; it sets the physical scale at which the (otherwise dimensionless) forecast is interpreted, and hence where a given normalised load falls relative to the USR-UPF operating points of Table 3.2.

The calibration α is a *declared modelling assumption, not a recovered calibration*. The NetMob traces are dimensionless by construction, for privacy reasons, and the forecaster sums per-base-station normalised demand across each cluster; there is consequently no physical scaler that could be looked up or fitted. The value was therefore chosen by examining a range of scales (Section 5.9) and selecting the one that yields the most useful mix of operating regimes across sites. At $\alpha = 1$, one unit of summed normalised demand corresponds to 1 Gbps of offered UPF load, placing the $K = 10$ clusters at mean loads of 0.11–1.66 Gbps with peaks to 5.8 Gbps. At this scale the fleet contains both sites dominated by SD-CORE DPDK (the central, heavily loaded clusters) and sites that spend substantial periods in the USR-UPF-favourable region (the peripheral clusters), so the selection problem is non-trivial across the fleet. Smaller values push almost every site into the USR-UPF regime, where the best policy is close to static USR-UPF; larger values push almost every site into the SD-CORE DPDK regime, where it is close to static SD-CORE DPDK. The same value is used by the controller work of Chapter 6, so that both chapters describe one physical regime. Section 5.8 returns to the consequences of this choice, which are substantial: because the profiled USR-UPF data path saturates one to two orders of magnitude below these loads, α largely determines how much scope for energy-aware switching exists at all.

5.4.2 Lite input adapter

The lite surrogate consumes load volume in kbits/s. The twin converts the calibrated load directly,

$$B = \lambda_{\text{Gbps}} \times 10^6 \text{ [kbits/s]}, \tag{5.2}$$

and supplies B as both the downlink and uplink lite features, matching the NetMob compatibility adapter of Section 3.10. This is the only transformation between a controller-facing load value and the model input; all remaining (full-model) features are absent by construction in the lite regime and are never required.

5.5 Predicted KPIs and Safety Semantics

A single evaluation returns a structured result containing the offered load, predicted `power_watts`, `delay_us`, `throughput_gbps`, and `predicted_loss`, together with two boolean flags that encode the twin's two distinct concerns:

`is_safe` (QoS preserved). True iff the predicted downlink delay is within the configured budget *and* the predicted packet loss does not exceed its tolerance:

$$\texttt{is_safe} \;=\; \big(\text{loss} \le L_{\max}\big) \;\wedge\; \big(\text{delay} \le D_{\max}\big), \tag{5.3}$$

with the default budget $D_{\max} = 200$ μs and loss tolerance $L_{\max} = 5$ packets per interval, where the interval is the 3 s profiling sample on which the loss surrogate was fitted. A control step is 15 minutes, so the flag asks whether the *instantaneous* loss rate at the step's offered load would breach the tolerance, not how many packets the step loses in total; no rescaling between the two time bases is applied, and none is implied. The 3 s sample is treated as a steady-state representative of the step, which is the assumption the profiling campaign supports: each load step was held long enough to reach steady state before sampling. Expressing the budget as a packet *count* rather than a ratio has one consequence. At the campaign's fixed 1250 B packet size, five packets per 3 s interval is a loss ratio of 0.17 % at 10 Mbps, 0.021 % at $\lambda_{\text{dec}} = 81$ Mbps and 0.011 % at $\lambda_{\text{qos}} = 149$ Mbps. A fixed count is thus a budget that tightens in inverse proportion to load, by about fifteen-fold across that range, which is the opposite of how a ratio-based service-level objective behaves. This is one reason $L_{\max}$ exerts the leverage on λ_{qos} documented in Section 5.7.1.

USR-UPF is safe only below its knee; SD-CORE DPDK is safe across the entire *profiled* range, with two qualifications set out below. The tolerance is deliberately non-zero: it is shared with the controller work of Chapter 6, where it additionally serves as the normalisation base of a graded QoS score. Its value is consequential: as Section 5.7 shows, it moves the derived QoS limit and thereby determines which constraint binds the decision threshold.

`is_efficient` (locally cheaper). True iff this action's predicted power is lower than the *other* action's predicted power at the same load. It is a relative, per-load comparison, not an absolute threshold, and is the signal a controller uses to decide whether the energy-cheaper action is still the safe one.

Interpretation of SD-CORE DPDK delay and safety. The delay surrogate predicts the same LoadCore downlink delay summary reported in Table 3.1, not a separately measured UPF-only delay. The preprocessing computes this target as a packet-count-weighted average of delay histogram representatives; no subtraction of RAN or other path components is performed. The measured mean of 311 μs summarises all DPDK samples, whereas the predictions of approximately 63–66 μs characterise the low-delay portion of the fitted response. These are a dataset-wide mean and conditional model predictions of the same target. The 200 μs budget is applied to the model prediction, so a predicted safe step does not guarantee that every physical measurement at that load meets the budget.

In the reported sweep, the predicted DPDK delay rises from 64.4 to 791 μs around 3.58 Gbps, changing the model's safety verdict. This load is within the marginal input ranges of the profiling data: the DPDK DN-side TX feature reaches approximately 5.00 Gbps and the NGRAN-side TX feature 5.16 Gbps, while the reported throughput reaches 5.06 Gbps. Consequently, the jump cannot be classified as out-of-range extrapolation on load alone. A tree-based predictor can produce such discontinuities, but the physical significance of this crossing requires validation against runs near that load and against the joint input distribution. In particular, the twin supplies equal values to its two lite features, whereas the measured features need not be equal (Section 5.4).

Under the declared calibration, 0.377 % of test cluster-steps exceed 3.58 Gbps, corresponding to the 0.38 % model-evaluated violation rate of static DPDK in Chapter 6. The reported correspondence at selected calibration values is 0.377, 0.050, 0.000 and 0.000 % against static-DPDK unsafe rates of 0.38, 0.05, 0.00 and 0.00 %. These rates describe the

surrogate verdicts in replay; they neither establish physical DPDK failure at that crossing nor justify dismissing it as an extrapolation artefact.

Safety across a switch. When a controller changes realisation within a step, the composite result is safe only if *both* the outgoing and the incoming realisation are safe at that load: $\texttt{is_safe} = \texttt{is_safe}_{\text{old}} \wedge \texttt{is_safe}_{\text{new}}$. This is deliberate, because during activation the step is served partly by each realisation, but it means switching steps carry their own violation risk, and it is the largest single contributor to the violation rate of the flip-prone controllers. For the derived threshold rule at $\alpha = 1$, the all-steps rate of 1.42 % decomposes into three disjoint buckets: 0.38 % on USR-UPF-selected steps that were *not* switches, 0.38 % on SD-CORE DPDK-selected steps that were not switches, and 0.66 % on switching steps of either kind. Counting instead by selected realisation and ignoring whether a switch occurred, as the *unsafe USR-UPF* column of Table 5.3 does, gives 0.58 % of all steps on USR-UPF-selected steps, equivalently 4.2 % of the 14.0 % of steps that select USR-UPF, and 0.83 % on SD-CORE DPDK-selected steps. The two presentations are consistent; they differ only in whether switching steps are separated out. This is why the two violation metrics used in this thesis do not reconcile by simple multiplication, and it is a further argument for anti-flapping on QoS grounds (Section 5.10).

The two flags are intentionally orthogonal. A USR-UPF result with `is_safe`=True and `is_efficient`=False denotes a load at which USR-UPF still meets QoS but already costs more than SD-CORE DPDK: the regime in which a controller should prefer SD-CORE DPDK on energy grounds even though USR-UPF would not violate QoS.

Figure 5.2 shows the surrogate's predicted responses over the deployable load range. The contrast from Chapter 3 is reproduced by the twin: SD-CORE DPDK power and CPU are essentially flat, while USR-UPF rises with load and its delay and loss turn sharply upward past the knee. The piecewise-constant "staircase" in the USR-UPF curves is the signature of the tree-based lite models. Beyond their training support, tree predictions remain at fitted leaf values; this numerical behaviour does not guarantee physical validity or conservative safety estimates.

5.6 Switching-Cost Model

Switching from one UPF to the other incurs setup work: the new data plane must be activated, which takes time and energy. Modelling this cost is essential, both to penalise

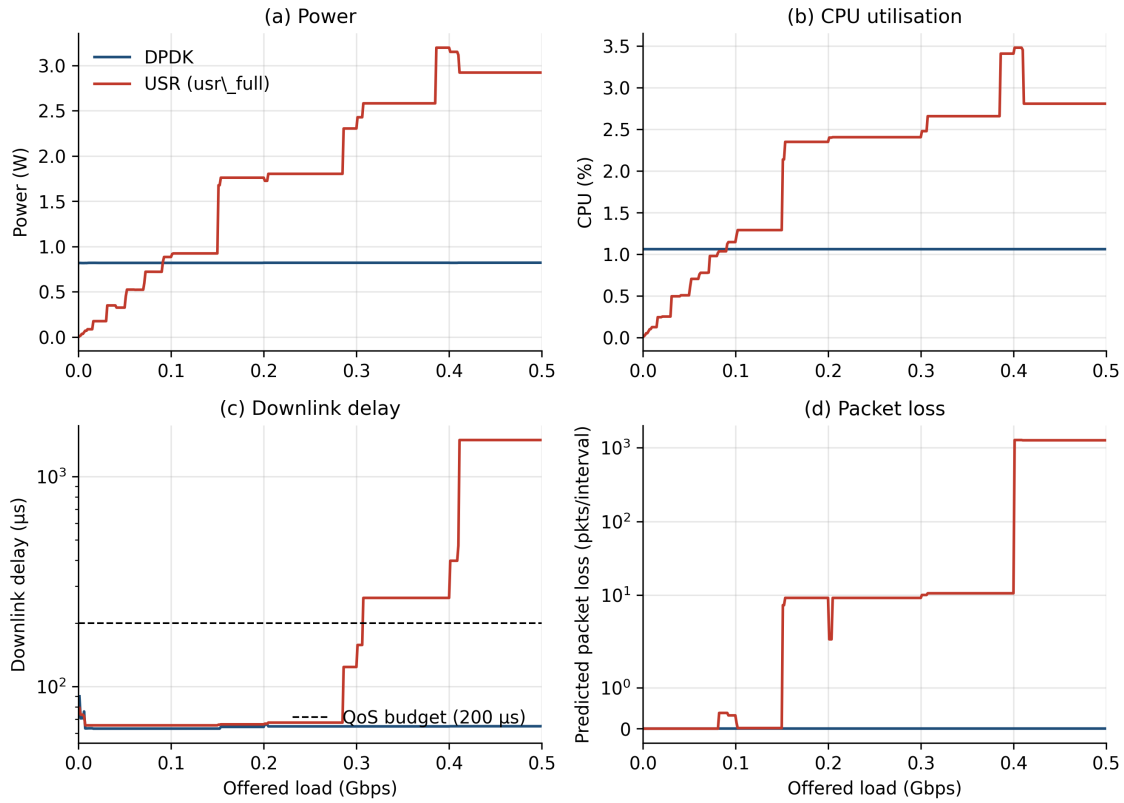


Figure 5.2: Twin-predicted UPF responses versus offered load (lite surrogate). (a) Power: SD-CORE DPDK flat near 0.82 W, USR-UPF rising and crossing SD-CORE DPDK at low load; (b) CPU; (c) downlink delay (log scale) with the 200 μs QoS budget marked; (d) predicted packet loss (symmetric-log scale). The USR-UPF staircase reflects the piecewise-constant nature of the tree-based lite models.

controllers that flip excessively and to represent the brief window during which the new UPF is not yet serving traffic.

5.6.1 Activation duration and the energy-spike rule

The activation *duration* is a property of the software stack, not of the host: bringing up the DPDK poll-mode driver takes substantially longer than starting the interrupt-driven USR-UPF path. The twin uses measured durations of 24.0 s for SD-CORE DPDK and 3.3 s for USR-UPF.

Absolute switching *energy* measured on one machine is not portable to another, and activation occurs at *zero traffic*, before the new UPF is forwarding, so the spike cannot depend on the serving load λ. It is a per-variant constant. The portable quantity is the

dimensionless *burst ratio*

$$k_v \;=\; \left.\frac{P_v^{\text{act}}}{P_v^{\text{idle}}}\right|_{\text{source}} \tag{5.4}$$

the ratio of the activation-transient power to the source machine's own idle draw (source measurements: $k_{\text{SD-CORE DPDK}} = 1.73$, $k_{\text{USR-UPF}} = 0.37$). Note that k_v is a ratio, not an increment: $k_{\text{USR-UPF}} < 1$ means the USR-UPF activation transient was measured *below* that machine's idle baseline, so Eq. 5.5 attributes a smaller spike to USR-UPF than its idle draw would imply. Rebasing onto this deployment's per-process attributed idle power P_v^{idle} and the software-stack activation duration $t_{\text{act},v}$ gives

$$E_{\text{spike},v} \;=\; \frac{k_v\, P_v^{\text{idle}}\, t_{\text{act},v}}{3600} \quad [\text{Wh}] \tag{5.5}$$

which is independent of λ. With $P^{\text{idle}}_{\text{SD-CORE DPDK}} = 0.82\,\text{W}$ and $P^{\text{idle}}_{\text{USR-UPF}} = 0.015\,\text{W}$, this yields $E_{\text{spike,SD-CORE DPDK}} \approx 9.5 \times 10^{-3}\,\text{Wh}$ and $E_{\text{spike,USR-UPF}} \approx 5.2 \times 10^{-6}\,\text{Wh}$. The two-order-of-magnitude asymmetry is preserved: SD-CORE DPDK activation is a long, CPU-heavy driver and hugepage initialisation, whereas USR-UPF activation is a short container start. Anchoring to loaded power would have flattened that distinction.

5.6.2 Within-step accounting

Because a simulation step (15 min at NetMob resolution) is far longer than any activation, the twin must decide how a switch that occurs *inside* a step is accounted for. Three policies are supported (Figure 5.3); the default is *sub-step*:

`sub_step` (default). The old UPF serves for t_{act} seconds and the new UPF for the remainder; the step's power is the time-weighted average, and the energy spike is paid once. KPIs are attributed to whichever UPF served the majority of the step, while safety is treated conservatively (a step is safe only if *both* the old and new UPF are safe at that load).

`round_down`. The switch is treated as instantaneous within the step; only the spike is paid.

`round_up`. The new UPF is treated as unavailable for the entire step (worst case); the requested action does not take effect until the next step.

An optional *prewarm* mode keeps SD-CORE DPDK in standby so that switching to it is effectively instantaneous, at the cost of a continuous standby power while it is not

the serving UPF. Prewarm is *disabled* in the scenario reported here, so SD-CORE DPDK cold-starts and the activation cost above applies in full; no standby draw is charged. This matches the deployment assumption of Chapter 6 and models cold-start switching without a resident standby realisation.

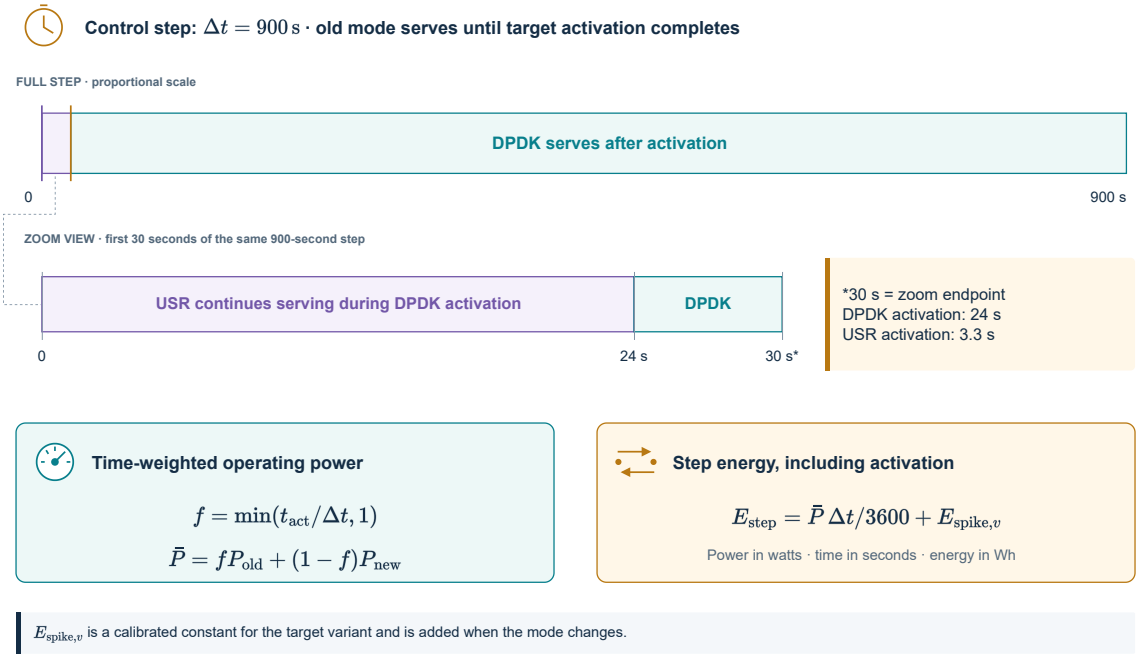


Figure 5.3: Within-step accounting under the `sub_step` policy for a USR-to-DPDK switch with prewarming disabled. The full bar represents a 900-second control step; the lower bar enlarges only its first 30 seconds. USR continues serving during the 24-second DPDK activation, after which DPDK serves the remainder. Thus 30 seconds is the endpoint of the zoom view, not an activation time. Operating power is time-weighted, and the calibrated activation energy is added once per switch.

5.7 Operating Thresholds Derived from the Surrogate

A naive controller might switch UPFs at a hand-tuned load threshold. The twin instead *derives* its operating points from the surrogate itself, so that they remain consistent with the measured physics and update automatically if the models are retrained. Two quantities are obtained by sweeping the stateless twin over load (Figure 5.4):

Energy break-even λ_{be}. The lowest load at which USR-UPF power meets or exceeds SD-CORE DPDK power. Below it, USR-UPF is the energy-cheaper action; above it, SD-CORE DPDK is.

QoS limit λ_{qos}. The highest load at which USR-UPF is still `is_safe`. This is the point the sweep *can only find because the twin uses* `usr_full`: it is where the surrogate's predicted loss or delay first crosses the budget.

For the scenario studied here the sweep yields $\lambda_{\text{be}} = 91$ Mbps and $\lambda_{\text{qos}} = 149$ Mbps. The decision threshold a controller should use is the more conservative of the two, minus a safety margin:

$$\lambda_{\text{dec}} = \min(\lambda_{\text{be}}, \lambda_{\text{qos}}) - m, \qquad m = 10 \text{ Mbps}, \tag{5.6}$$

giving $\lambda_{\text{dec}} = 81$ Mbps here (energy-limited: the break-even binds before the QoS limit). This ordering depends on the loss tolerance $L_{\max}$ of Equation (5.3). Under zero tolerance the QoS limit falls to 81 Mbps and binds first, giving $\lambda_{\text{dec}} = 71$ Mbps and a QoS-limited regime. The derivation machinery is unchanged either way, which is the point of deriving rather than hardcoding, but the reported operating point is sensitive to a budget that is a policy choice rather than a measured quantity.

5.7.1 Validating the boundary, not the fit

A held-out validation study evaluates the lite surrogate procedure by leaving one complete measurement sweep out (LOSO) or leaving groups of offered-load levels out (LOLO). Model family and hyperparameters are selected within the remaining runs. This is distinct from the original row-level split and from a single grouped-run split. Because the deployed models were trained on rows drawn from every campaign run, no run is held out from them; the following metrics therefore assess surrogates refitted under the same procedure and input convention (UL:=DL), and characterise the modelling procedure rather than the deployed models themselves.

Table 5.1: Held-out-sweep validation of lite surrogates with nested run-grouped selection. Whole-range fit does not measure accuracy at a QoS decision boundary.

Variant	Target	R^2	MAE
DPDK	Attributed power	0.175	0.0033 W
DPDK	Delay	−0.215	471 μs
USR	Attributed power	0.971	0.085 W
USR	Delay	0.913	1088 μs
USR	Loss	0.995	3765 packets/3 s

USR delay and loss have high whole-range R^2, but their errors exceed the 200 μs delay and five-packet loss budgets. Large saturated-regime values dominate these scores.

DPDK's low power R^2 accompanies a small absolute error because its attributed power varies little. Neither observation establishes reliable QoS classification in the operating band.

Table 5.2: USR safety classification under nested LOSO. False-safe is the share of measured-unsafe samples predicted safe; false-unsafe is the share of measured-safe samples predicted unsafe. These denominators differ from the probability of a violation conditional on a predicted-safe verdict.

Load band	Samples	False-safe (%)	False-unsafe (%)
All measured loads	4386	2.45	8.94
Decision band (50–447 Mbps)	996	26.67	19.08

The boundary is poorly resolved by the measurements. The nominal 149 Mbps limit lies just below a tree split between profiled levels at 100 and 200 Mbps; it is not a physical limit measured to one-megabit precision. The deployed safety map is also non-monotone: it marks 150–199 Mbps unsafe but 200 Mbps safe. A first crossing is therefore an operating rule for this model, not a guarantee that every lower or higher load shares its verdict. Approximately 60.5% of controller test steps occupy unprofiled gaps between 0.1 and 0.6 Gbps. An operating-region validation campaign is needed to resolve those gaps, with power measurement scope and hardware recorded explicitly.

Controller QoS violations elsewhere in this thesis are violations predicted inside the surrogate environment. They do not include the surrogate's own errors against physical measurement. The two error sources cannot be added without a joint evaluation. Section 6.9.2 tests how the controller comparisons change under alternative surrogate calibrations; that sensitivity study complements, but does not replace, physical validation.

5.7.2 Anti-flapping band

A single decision threshold applied to a noisy forecast would cause rapid, wasteful flipping near the boundary. A hysteresis band prevents this: the controller switches USR-UPF→SD-CORE DPDK only when the predicted load rises above $\lambda_{\uparrow}$, and back only when it falls below $\lambda_{\downarrow}$.

The earlier single-site prototype [49] sized this band from the forecaster's reported error,

$$\text{band} = 2 \cdot \text{MAE}_{\text{forecast}}, \qquad \lambda_{\uparrow} = \lambda_{\text{dec}}, \qquad \lambda_{\downarrow} = \lambda_{\text{dec}} - \text{band}. \tag{5.7}$$

Algorithm 1 Hysteresis controller with cooldown, per cluster. Thresholds $\lambda_\uparrow$, $\lambda_\downarrow$ are derived from the surrogate (Section 5.7); $\hat{\lambda}_t$ is the one-step load forecast. State is per cluster: the previous action and a cooldown counter.

Require: $\lambda_\uparrow > \lambda_\downarrow \geq 0$, cooldown length $c \geq 0$
$a_{\text{prev}} \leftarrow \varnothing$; $k \leftarrow 0$ ▷ no action yet; cooldown counter
for each control step t **do**
  observe forecast $\hat{\lambda}_t$
  if $a_{\text{prev}} = \varnothing$ **then** ▷ first step: decide on the band midpoint
    $a_t \leftarrow$ USR **if** $\hat{\lambda}_t < \frac{1}{2}(\lambda_\uparrow + \lambda_\downarrow)$ **else** DPDK
  else if $k > 0$ **then** ▷ locked in: thresholds are ignored while cooling down
    $k \leftarrow k - 1$; $a_t \leftarrow a_{\text{prev}}$
  else
    $a_t \leftarrow a_{\text{prev}}$
    if $a_{\text{prev}} =$ USR **and** $\hat{\lambda}_t \geq \lambda_\uparrow$ **then**
      $a_t \leftarrow$ DPDK ▷ load rose past the upper edge
    else if $a_{\text{prev}} =$ DPDK **and** $\hat{\lambda}_t \leq \lambda_\downarrow$ **then**
      $a_t \leftarrow$ USR ▷ load fell below the lower edge
    end if
    if $a_t \neq a_{\text{prev}}$ **then**
      $k \leftarrow c$ ▷ start cooldown
    end if
  end if
  $a_{\text{prev}} \leftarrow a_t$; **emit** a_t
end for

At the load scale studied here that rule no longer works. The forecaster's error is reported in normalised units, so the band inherits the scale factor α of Equation (5.1). At $\alpha = 1$ the mean absolute error for the operating cluster count is ≈ 110.7 Mbps, so Equation (5.7) would prescribe a band of ≈ 221 Mbps, wider than the decision threshold itself. The lower switch point would be clamped at $\lambda_\downarrow = 0$, and the controller could never return to USR-UPF once it had left. Section 4.13 explains why: the aggregate MAE is dominated by high-load steps far above the threshold, and is the wrong statistic for sizing a band around a low threshold.

With the optimised models, the band is therefore set directly rather than derived from the forecast error. The results of this chapter use

$$\text{band} = 20 \text{ Mbps}, \qquad \lambda_\uparrow = 81 \text{ Mbps}, \qquad \lambda_\downarrow = 61 \text{ Mbps}, \tag{5.8}$$

an energy-oriented setting that returns to USR-UPF as early as possible while preserving genuine two-threshold behaviour. Chapter 6 also uses a 50 Mbps band, the value that maximises the controllers' combined reward.

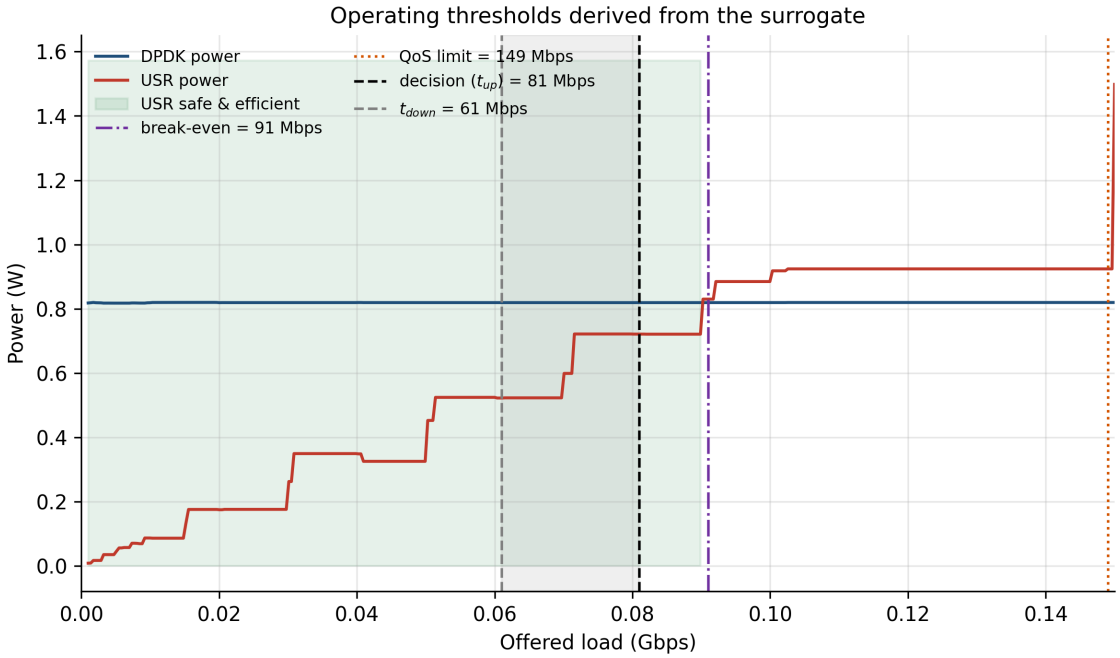


Figure 5.4: Operating thresholds derived from the surrogate. USR-UPF power (red) crosses SD-CORE DPDK power (blue) at the energy break-even (91 Mbps); the QoS limit (149 Mbps) is where USR-UPF first becomes unsafe. The decision threshold $\lambda_{\uparrow} = 81$ Mbps is the binding (energy break-even) limit minus a 10 Mbps margin; the shaded band down to $\lambda_{\downarrow} = 61$ Mbps is the hysteresis region. The green region marks loads where USR-UPF is both safe and cheaper than SD-CORE DPDK.

5.8 Evaluation Methodology and Demonstration

5.8.1 Rollout engine

To evaluate any controller, the twin is driven over the full test set of per-cluster load windows. For efficiency the rollout pre-evaluates the stateless twin once for both actions over the entire flattened array of loads, then performs the conceptually sequential per-cluster loop in pure Python: each step reads the controller's requested action, looks up the precomputed KPIs, and applies the switching-cost accounting of Section 5.6. The result

is one record per cluster-step, suitable for aggregation. In the scenario studied here this is $K = 10$ clusters $\times$ 1009 windows = 10,090 steps of 15 min each.

Algorithm 2 Rollout engine. Evaluates one controller over the full horizon. The controller sees only the *forecast*; the twin is always evaluated at the *actual* load, so that forecast error is reflected in the evaluated outcome.

Require: controller π, twin $\mathcal{T}$, forecasts $\hat{\lambda}$, actuals λ, calibration α, step length Δt

precompute $\mathcal{T}(\cdot)$ on the grid of actual loads ▷ shared across controllers, so all see one twin
for each cluster k **do**
  $a_{\text{prev}} \leftarrow \varnothing$
  for each step t **do**
    $a_t \leftarrow \pi_k(\alpha\hat{\lambda}_{t,k})$ ▷ decide on the forecast
    $r_t \leftarrow \mathcal{T}(a_t,\ \alpha\lambda_{t,k})$ ▷ but evaluate on the truth
    if $a_t \neq a_{\text{prev}}$ **and** $a_{\text{prev}} \neq \varnothing$ **then**
      charge switching energy $E_{\text{spike}}(a_{\text{prev}} \to a_t)$ and apply within-step accounting (Section 5.6)
    end if
    accumulate energy r_t.power $\cdot\ \Delta t$, safety flag r_t.`is_safe`, and usage
    $a_{\text{prev}} \leftarrow a_t$
  end for
end for
return per-cluster and aggregate energy, violation rate, usage, switch count

5.8.2 Metrics

From the per-step records the following metrics are computed. Energy includes both steady-state consumption (power $\times\ \Delta t$) and the switching spikes of Equation (5.5), so that flipping is penalised:

Total energy (Wh) and **average power (W)** over all steps.

Energy saving (%) relative to the always-SD-CORE DPDK baseline.

Unsafe USR rate the fraction of USR-UPF-serving steps that violated QoS, the key safety metric.

USR usage (%) the fraction of steps served by USR-UPF.

Decision flip rate the per-cluster fraction of steps at which the serving UPF changed, used as a proxy for churn.

5.8.3 Controllers used in this chapter

As stated in Section 5.1, this chapter exercises only non-learning controllers, so that the figures characterise the *twin* rather than any particular learned policy:

Static SD-CORE DPDK always selects SD-CORE DPDK (the safe, high-power reference).

Static USR-UPF always selects USR-UPF (maximally energy-greedy, and unsafe whenever load exceeds the knee).

Hysteresis the two-threshold rule of Algorithm 1, consuming the surrogate-derived decision point together with the fixed 20 Mbps band of Equation (5.8), rather than the forecast-derived band of Equation (5.7), which degenerates in this regime (Section 5.7), with a one-step cooldown after each switch.

Learned controllers are deferred to Chapter 6. The single-threshold policy and the informed oracle are evaluated here as well, the first as the simplest controller consuming a derived operating point and the second as a clairvoyant reference using realised rather than forecast load; both appear in Table 5.3.

5.8.4 Headroom for energy-aware switching

The case for switching at all rests on the load distribution. Figure 5.5 shows that the offered load is heavily right-skewed, but that under the calibration of Equation (5.1) only about 14 % of cluster-steps fall below the decision threshold. In other words, only on roughly one step in seven is USR-UPF both safe and cheaper than SD-CORE DPDK. The remaining steps exceed the decision threshold, so the rule selects DPDK; not all of them exceed the separate saturation onset λ_{onset}.

A clairvoyant reference evaluates realisation selection using the actual load at each step instead of a forecast. In the reported replay it saves 6.7 % relative to static DPDK. This provides a benchmark for the forecast-driven policies under the same calibration and accounting rules. It is not a proven upper bound on all causal controllers: the reference does not establish a globally optimal sequence under switching costs and within-step safety constraints. Accordingly, ratios to its saving are reported as relative performance, not as fractions of a certified optimum.

The reason is physical rather than algorithmic. The profiled USR-UPF data path first shows loss at $\lambda_{\text{loss}} = 60$ Mbps and saturates from $\lambda_{\text{onset}} \approx 170$ Mbps (Table 3.2); the derived break-even and QoS points, $\lambda_{\text{be}} = 91$ and $\lambda_{\text{qos}} = 149$ Mbps, sit inside that band.

Meanwhile $\alpha = 1$ places cluster loads at 0.11–1.66 Gbps mean with peaks to 5.8 Gbps, one to two orders of magnitude above any of them. A software UPF of this capacity is undersized relative to the offered traffic, and the twin's role here is precisely to make that mismatch quantitative rather than to disguise it.

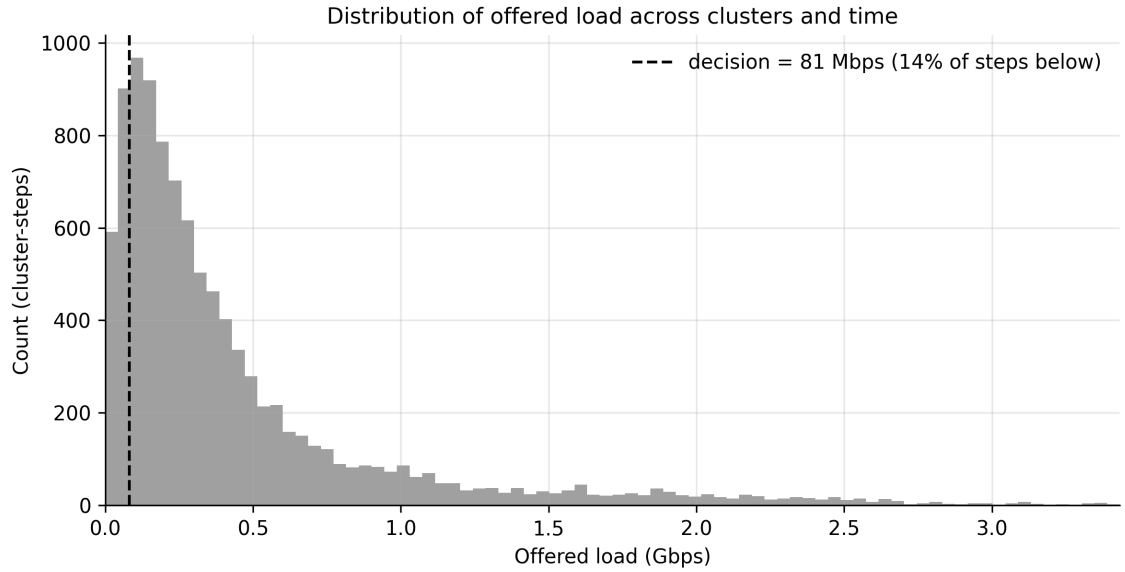


Figure 5.5: Distribution of calibrated offered load across all clusters and time windows. Only about 14 % of cluster-steps lie below the derived decision threshold (dashed), i.e. in the regime where USR-UPF is the safe and cheaper action; the long right tail extends well beyond the USR-UPF onset of saturation λ_{onset}, where SD-CORE DPDK is the only viable action.

5.8.5 The twin under a hysteresis controller

Figure 5.6 shows a representative window of one cluster's rollout. The controller holds USR-UPF while the predicted load stays within the hysteresis band, switches to SD-CORE DPDK when load rises above $\lambda_{\uparrow}$, and returns to USR-UPF only after load falls below $\lambda_{\downarrow}$; the power trace (lower panel) accordingly sits near SD-CORE DPDK's 0.82 W plateau during high-load excursions and drops into USR-UPF's low-power regime otherwise. The band visibly prevents single-sample flips around the threshold: the load crosses $\lambda_{\uparrow}$ repeatedly without provoking a switch until it also clears the band. Because prewarm is disabled, the cold-start activation cost is visible directly in the trace as short spikes above the SD-CORE DPDK plateau at switch instants.

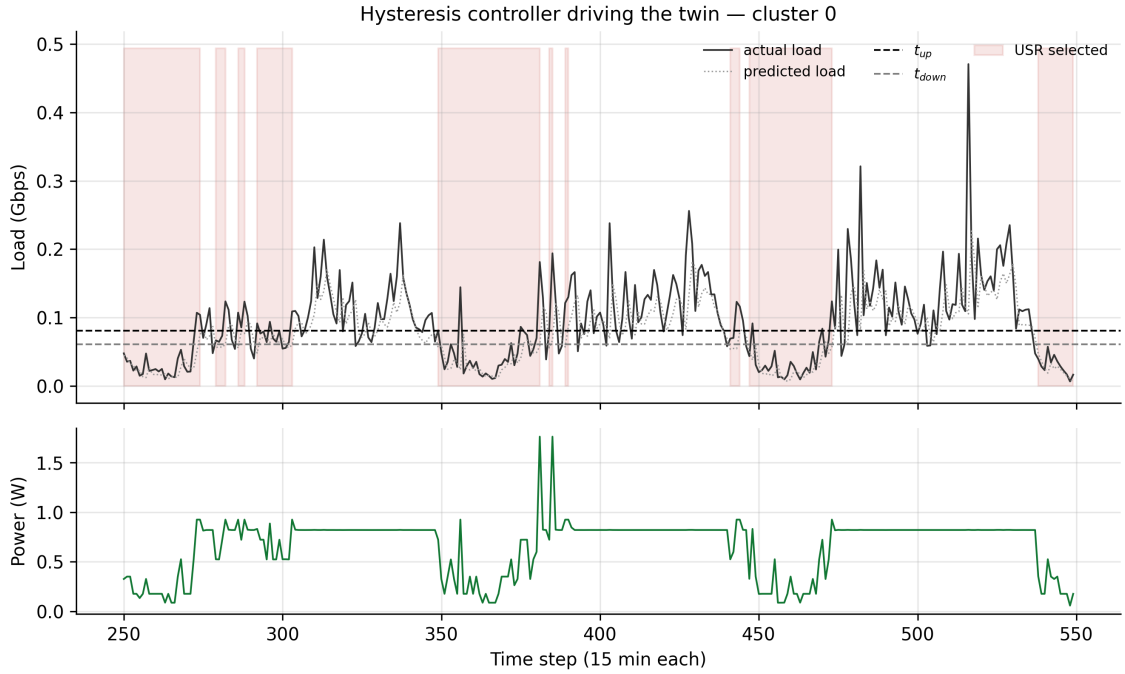


Figure 5.6: A representative window of the hysteresis controller driving the twin (one cluster). Upper: actual and predicted load with the switch points $\lambda_{\uparrow}$, $\lambda_{\downarrow}$; shaded spans mark USR-UPF-served steps. Lower: the twin's predicted power, near the SD-CORE DPDK floor when SD-CORE DPDK serves and low when USR-UPF serves.

5.8.6 Aggregate behaviour

Table 5.3 and Figure 5.7 summarise the controllers over all 10,090 steps. The headline observation is that static USR-UPF is not merely unsafe here but actively *more expensive*: it consumes 2.5× static SD-CORE DPDK's energy (−154.5 % "saving") while violating QoS on 70.3 % of its steps. This is the direct consequence of running a saturated data path: past its knee, USR-UPF loses throughput, inflates delay, and consumes more CPU power than the accelerated data path. Any intuition that the software UPF is the cheap option holds only below the knee.

The hysteresis rule recovers 4.8 % of static SD-CORE DPDK's energy while keeping the unsafe rate at 3.0 %, by serving USR-UPF on the 11.5 % of steps where it is genuinely both safe and cheaper and ceding to SD-CORE DPDK elsewhere. Against the 6.7 % clairvoyant-reference saving established above, it achieves roughly 72 % of that saving using only a forecast and two derived thresholds. The stateless threshold policy achieves a marginally higher saving (4.9 %) at a noticeably worse unsafe rate (4.2 %) and twice the switching churn, which is the trade the hysteresis band exists to make.

The absolute saving is modest, and it would be misleading to present it otherwise: the evaluated clairvoyant reference saves 6.7 % in this scenario. The demonstration this chapter offers is therefore not that switching saves a great deal of energy on this hardware, but that the twin *quantifies the modelled trade-off*: it identifies where the operating points lie, prices the switching cost, supplies a clairvoyant comparison, and exposes the energy–safety frontier that learned controllers will later be trained to navigate. The modest savings should be interpreted alongside the load distribution and calibration rather than extrapolated to other deployments.

Table 5.3: Aggregate metrics over 10,090 cluster-steps (non-learning controllers; learned controllers are deferred to Chapter 6). Energy includes switching spikes. The informed oracle sees the actual load at each step and is reported as a clairvoyant reference, not as a deployable controller or a proven optimum. *Unsafe* USR-UPF is the fraction of USR-UPF-selected steps that violated QoS, so it is zero by construction for a controller that never selects USR-UPF; it is therefore not comparable with the all-steps violation rate reported for the learned controllers in Chapter 6.

Controller	**Energy** (Wh)	**Avg** P (W)	**Saving** (%)	**Unsafe USR-UPF** (%)	**USR-UPF use** (%)
Static SD-CORE DPDK	2070.7	0.821	0.0	0.0	0.0
Static USR-UPF	5269.8	2.089	−154.5	70.3	100.0
Threshold	1969.6	0.780	4.9	4.2	14.0
Hysteresis	1971.0	0.781	4.8	3.0	11.5
Oracle (reference)	1931.0	0.765	6.7	0.0	13.7

5.9 Sensitivity to the Load Calibration

Every headline number in this chapter is conditioned on α, which Section 5.4 declares to be an assumption rather than a measured constant. That makes the sensitivity of the conclusions to α a first-order question, not an appendix item: if the ordering of controllers changed with the calibration, none of the comparisons would mean anything. This section sweeps it. Thresholds are re-derived from the surrogate at each value and all non-learning controllers are re-run over the full 10,090 cluster-steps.

The magnitudes move; the ordering does not. Across a twelve-fold range of α the achievable saving varies by a factor of fifteen, from 56 % to under 4 %. The *ranking* is unchanged at every value: oracle > threshold ≈ hysteresis > static USR-UPF. The conclusions

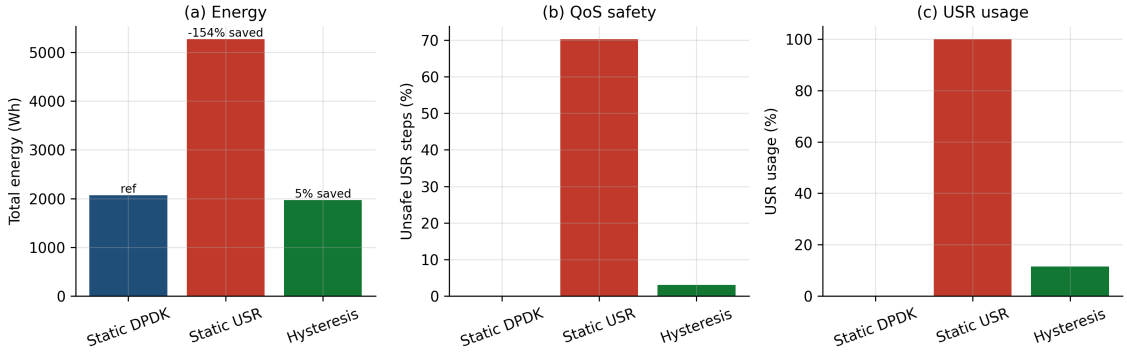


Figure 5.7: Aggregate comparison of the non-learning controllers. (a) Total energy (with saving vs. static SD-CORE DPDK annotated); (b) unsafe USR-UPF rate; (c) USR-UPF usage. Static USR-UPF is both the most expensive and by far the least safe option in this regime, since it runs a saturated data path; hysteresis achieves a modest saving while keeping the unsafe rate low.

Table 5.4: Sensitivity of the energy result to the load calibration α. Savings are against static SD-CORE DPDK. *Capture* is the hysteresis saving as a fraction of the clairvoyant-reference saving. The row $\alpha = 1.0$ is the configuration used throughout this thesis.

α	Mean load (Gbps)	Peak (Gbps)	Steps below λ_{dec}	Oracle (%)	Hyst. (%)	Thresh. (%)	Static USR-UPF (%)	Capture (%)
0.12	0.056	0.70	80.9 %	56.11	55.32	55.80	43.4	98.6
0.25	0.117	1.45	57.2 %	33.67	31.16	32.02	−7.1	92.5
0.50	0.234	2.90	31.7 %	16.41	13.43	14.20	−74.6	81.8
0.75	0.351	4.35	19.8 %	9.76	7.31	7.62	−120.6	74.8
1.00	0.468	5.80	13.7 %	6.68	4.80	4.85	−154.5	71.8
1.50	0.703	8.69	8.1 %	3.75	2.17	2.21	−200.7	57.9

this chapter draws are therefore properties of the system, while the headline percentage is a property of the assumption, and the two should not be quoted interchangeably.

Where the regime inverts. Static USR-UPF saves 43 % at $\alpha = 0.12$ and *costs* 7 % at $\alpha = 0.25$. Between those two values the correct default flips from USR-UPF to SD-CORE DPDK, which is the single number that characterises the operating regime most compactly. At $\alpha = 1$ the deployment sits well beyond that crossing: USR-UPF is the exception, not the default, and only 13.7 % of steps fall below λ_{dec}.

Comparison with the earlier single-site prototype. The single-site prototype of [49] reported 16.9 % saving, compared with 4.8 % for hysteresis here. The calibration sweep

illustrates why raw percentages from different scenarios are not directly comparable: at $\alpha = 0.5$ the clairvoyant reference saves 16.41 %, compared with 6.68 % at $\alpha = 1$. This does not identify the prototype's effective calibration or establish equivalent controller performance; its fitted models and evaluation conditions also differ. A direct comparison requires matched traffic, physical models, calibration, and switching accounting.

Caveat. The sweep covers the non-learning controllers. The learned controllers of Chapter 6 are trained at a specific α, so evaluating them at another is a distribution shift rather than a sensitivity test; establishing the same invariance for them requires retraining at each calibration.

5.10 Summary

This chapter constructed a measurement-grounded digital twin of the UPF data plane from the profiling results of Chapter 3. The principal points are:

1. **Scope.** The twin answers "what would happen" for a given (load, action) pair; it is neither a forecaster (Chapter 4) nor a controller (Chapter 6), and is kept strictly controller-agnostic.

2. **Variant choice.** `USR` is represented by `usr_full`, not `usr_safe`, precisely because a safety-aware twin must be able to represent the full observed response; the restricted `usr_safe` subset does not adequately cover the high-load regime.

3. **Two modes.** A stateless physics mode and a stateful rollout mode share one surrogate core; the latter adds a portable switching-cost model based on a measured activation duration and an energy-spike scaling rule.

4. **Derived thresholds, with one deliberate exception.** The energy break-even and QoS limit are swept from the surrogate rather than assumed, keeping the operating points consistent with the measured physics. The anti-flapping band is the exception. Deriving it from the forecaster's own error as $2 \cdot \mathrm{MAE}$, as the earlier prototype did, gives 221 Mbps at $\alpha = 1$, which is *wider* than the 81 Mbps decision threshold it is meant to surround; that clamps $\lambda_{\downarrow}$ to zero and collapses the controller into always-SD-CORE DPDK. This rule, used by the earlier prototype, is therefore replaced by a band set directly: 20 Mbps here, oriented to energy. Chapter 4 identifies why the

old rule fails: an aggregate MAE is the wrong statistic to size a band around a low threshold.

5. **Switching cost is small; hysteresis is a QoS argument.** The per-switch energy is 9.5 mWh against roughly 205 mWh for a SD-CORE DPDK step, some 4.6 %. Anti-flapping is therefore justified by QoS rather than by energy: a controller that flips on forecast jitter spends part of each transition on the realisation that is wrong for the load actually arriving. The sensitivity study of Section 5.9 bears this out: the plain threshold rule is marginally *cheaper* than hysteresis at every calibration examined, and consistently less safe.

6. **Demonstration.** Driven by a transparent hysteresis controller, the twin recovers 4.8 % of SD-CORE DPDK's energy, roughly 72 % of the 6.7 % saving of the clairvoyant reference, while remaining safe on 97.0 % of USR-UPF steps, quantitatively exposing the energy–safety trade-off that learned controllers will later be trained to navigate.

7. **Calibration-dependent opportunity.** Under the declared calibration $\alpha = 1$, the profiled USR-UPF path saturates one to two orders of magnitude below the offered cluster loads, so only ≈ 14 % of steps admit a USR-UPF decision at all and static USR-UPF costs 2.5× SD-CORE DPDK. The load distribution and reference replay quantify the limited opportunity for low-power operation in this scenario. They motivate evaluating QoS at comparable energy as well as reporting absolute saving.

CHAPTER 6

Learning Controllers for Energy-Aware UPF Orchestration

6.1 Introduction

The preceding chapters provide the signals needed for energy-aware UPF orchestration: attributed per-UPF power and QoS (Chapter 3), spatiotemporal load forecasts (Chapter 4), and counterfactual power/QoS estimates from the digital twin (Chapter 5). This chapter evaluates how those signals change the control problem. At each 15-minute interval and at each Mobile Edge Computing (MEC) site, a controller selects between a high-throughput SD-CORE DPDK data plane and a lower-power USR-UPF data plane. The evaluation compares the energy–QoS trade-offs obtained by rule-based, single-site, and cooperative learning controllers. It examines how forecast load, modelled power and QoS, temporal history, and shared training contribute to control performance. All policies are evaluated through the digital twin; the implications of this evaluation design are discussed in Section 6.10.

Within the augmented-observability pipeline, this chapter is where the inferred signals are put to use, and it adds the fourth stage. A controller at one site can observe only that site; sharing one policy across the fleet lets each site act on regularities learned from the operating experience of the others. This transfer is a property of training rather than an additional runtime signal, and the chapter tests whether it improves decisions.

The controller designs developed over the course of this research address successive limitations in modelling, temporal information, and cross-site learning. Section 6.2 describes the single-site prototype from which this work started, which combined telemetry, a fitted physical model, and an offline forecast for realisation selection at one site. Section 6.3 formalises the rule-based baselines used to assess the learning controllers. Section 6.4 introduces the single-site reinforcement learning (RL) formulation and its result. Section 6.5 compares recurrent policies with explicit load histories to assess how temporal information should enter the control decision. Section 6.6 examines the limitations of independent per-site learning, and Section 6.7 evaluates cooperative multi-agent RL, and

Section 6.8 tests the result across fleet sizes and load calibrations. Section 6.9 reports additional runs carried out for the conference paper, which focus on the combined reward and isolate the contribution of the centralised critic. Section 6.10 draws the observability argument together.

6.2 Starting point: a single-site prototype

The work began as a single, self-contained prototype [49] that combined, in one repository, a hand-fitted power model, an offline traffic predictor, and a PPO controller for one UPF site. It established the problem shape and the reward ingredients that survive into the present work, and its limitations are precisely what drove the modular pipeline developed in the thesis.

Three of its design choices matter because they motivate the design of the forecasting, modelling, and control components. First, its power and QoS model was fitted inside the control repository from a bespoke measurement set; there was no separation between the *evaluator* and the *controller*, so the physical model and policy were developed as a coupled system. Second, its traffic forecast was a Keras Long Short-Term Memory (LSTM) network trained offline on a single time series, its predictions exported to a column the environment read as static data, not a live model in the loop. Third, it was single-site by construction: one environment, one agent, one UPF location.

Quantitatively the prototype demonstrated the potential for energy-aware control: across five independently trained seeds a feed-forward PPO controller reduced mean attributed energy by 16.9% relative to always-on SD-CORE DPDK provisioning, at an equal QoS-violation rate to a hand-tuned rule-based threshold. It also showed, through an explicit ablation, that a *recurrent* PPO policy did not improve on the feed-forward one, a comparison repeated with the modular digital twin (Section 6.5).

The prototype established the feasibility of forecast-driven realisation selection at one site. The subsequent research separated physical modelling from policy learning, developed spatially aggregated traffic forecasts, and extended the control problem to a fleet of sites. The following sections evaluate the resulting controller designs.

6.3 Problem formulation and rule-based baselines

Decision problem. A fleet of $K = 10$ heterogeneous MEC clusters, each serving an aggregated traffic load derived from the NetMob Lyon traces at 15-minute resolution, must at every step select per site between SD-CORE DPDK and USR-UPF. SD-CORE DPDK sustains a near-constant 0.82 W power floor but tolerates high load; USR-UPF is far cheaper at low load but has a narrow safe operating region and violates the delay/loss budget above roughly 0.15 Gbps. Switching realisations incurs a measured activation energy and, optionally, a cooldown penalty. The controller sees, per site, the current load, an eight-step load history, a one-step-ahead forecast, and the recent QoS, energy, and switching state (a 14-dimensional observation).

What the action denotes. Selecting USR-UPF at a site means bringing the USR-UPF data plane up on that site's host and cutting the site's traffic over to it, tearing down the realisation that was serving. That is the reading the cost model encodes: the penalty of Section 5.6 is an *activation* cost, namely driver and hugepage initialisation for SD-CORE DPDK and container start for USR-UPF, incurred over a measured duration during which the incoming realisation is not yet forwarding. It is not the cost of steering traffic between two realisations that are both already resident.

Two limits follow, and every controller result in this chapter inherits them. First, the profiling campaign of Chapter 3 measured each realisation in steady state and never observed a live transition: the transition is therefore composed from a separate activation measurement rather than observed end to end, and because that measurement was taken on a different host at whole-socket scope, only its duration and its dimensionless burst ratio are carried across (Section 5.6). Second, an operator who instead kept both realisations resident and merely re-routed between them would face a smaller cost, with no activation energy and no activation outage, so the numbers below are conservative with respect to that deployment style rather than descriptive of it. In neither reading does a switch reconfigure one program between two I/O modes: the two realisations are different projects (Section 3.2), so the fleet is heterogeneous by construction.

Baselines from the twin. The rule-based controllers are not hand-tuned; they are *derived* from the digital twin's surrogate by sweeping load and reading off the physical operating points (Chapter 5):

Operating point	Value	Definition
Energy break-even λ_{be}	91 Mbps	first load where USR-UPF power ≥ SD-CORE DPDK power
QoS limit λ_{qos}	149 Mbps	last load where USR-UPF is safe
Decision threshold λ_{dec}	81 Mbps	$\min(\lambda_{\text{be}}, \lambda_{\text{qos}})$ less a 10 Mbps margin
Forecast MAE (K=10)	110.7 Mbps	from the forecaster's held-out summary

Two families follow. A *threshold* controller switches to USR-UPF whenever predicted load is below λ_{dec}. A *hysteresis* controller adds a band, $\lambda_{\uparrow} = 81$, $\lambda_{\downarrow} = \lambda_{\uparrow} - \text{band}$, to suppress flapping under forecast noise. As noted in Section 5.7.2, the automatic band defined by Equation (5.7) is $2 \times \text{MAE} = 221$ Mbps, which is *wider* than the decision point itself and collapses the controller to always-SD-CORE DPDK. The classical contenders are therefore tuned bands: a 20 Mbps band oriented to energy, used in this section, and a 50 Mbps band that maximises the combined reward (Section 6.9). This distinction between the derived-but-degenerate rule and the tuned rule is exactly the kind of brittleness that motivates learning.

Reward. All learning controllers optimise

$$r_t = -\big(\alpha\,\text{SEC}_t + L_t^{\text{QoS}} + L_t^{\text{SW}} + L_t^{\text{CD}}\big), \tag{6.1}$$

where SEC_t is the steady-state specific energy consumption (power per Mbps), $L_t^{\text{QoS}} = \lambda_{\text{QoS}} \max(0, \tau - Q_t)$ is a graded penalty on the twin's continuous QoS score Q_t, $L_t^{\text{SW}} = \lambda_{\text{sw}} \cdot$ sw_energy_wh charges the twin's measured switching energy only when the realisation actually changes, and L_t^{CD} is a soft cooldown surcharge. Crucially the switching energy is a *load-independent per-variant constant*: activation happens at zero traffic, so the switch penalty does not spuriously scale with the load the new UPF has not yet begun to serve (Chapter 5). The weights are fixed across every experiment in this chapter:

Symbol	Value	Role
α_r	100.0	scales SEC_t into the reward. Distinct from the load calibration α of Chapter 5; the two are unrelated and share a symbol only by convention.
λ_{QoS}	30.0	weight of the graded QoS penalty
τ	0.90	QoS compliance target below which the penalty engages
λ_{sw}	4.0	multiplies the twin's switching energy, in Wh
cooldown window	4 steps	minimum dwell before a switch is unpenalised
cooldown cost	0.5	surcharge for switching inside that window

A caveat belongs with this table. SEC_t is a ratio, power per unit throughput, while the three penalties are absolute, so the balance between them is load-dependent by construction. To quantify this balance, the terms were logged separately over a full test rollout of the trained cooperative controller:

Term	Total penalty	Share
$\alpha_r\,\mathrm{SEC}_t$ (energy)	4261.0	75.1 %
L_t^{QoS}	1342.6	23.7 %
L_t^{CD} (cooldown)	64.1	1.1 %
L_t^{SW} (switching)	5.9	0.1 %
Total	5673.6	100 %

Three things follow. First, the reward is overwhelmingly an energy objective with a QoS constraint attached: three quarters of the accumulated penalty is the SEC term and almost all of the remainder is QoS.

Second, the ratio does inflate at low load, as the form of SEC_t implies: the mean per-step energy penalty is 1.07 below the decision threshold against 0.32 above it, a factor of 3.3. It does not, however, grow without bound in this deployment: predicted power is floored by the idle draw of whichever realisation is active, so SEC_t is bounded wherever the offered load is bounded away from zero, which it is over the whole trace. The term is skewed towards low load, not divergent there.

Third, the switching penalty contributes 0.1 % of the total. The anti-flapping machinery is therefore inert as an *energy* incentive, which is consistent with the measured switching cost of Chapter 5 being a few per cent of a step. Its justification is the QoS term it protects, not the energy it saves.

Results are reported in absolute Wh alongside reward throughout, so that no conclusion rests on the reward scale alone.

Every term in Equation (6.1) is a reading from the upstream pipeline: SEC and Q from the twin's surrogates, the forecast inside the observation from the forecaster, the switching energy from the profiling campaign. Figure 6.1 shows the resulting agent–environment loop, the composition of the observation, and the reward.

Algorithm 3 MAPPO with centralised training and decentralised execution. One actor is shared by all K sites, so experience gathered anywhere updates the policy used everywhere; this parameter sharing is the mechanism behind the spatial-observability claim. The critic sees the joint state and is used only during training.

Require: shared actor π_θ, centralised critic V_ϕ, K sites, rollout length T

for each update **do**
  Collect: ▷ decentralised – each site acts on its own observation
  for $t = 1 \dots T$ **do**
    for each site k **do**
      $a_{t,k} \sim \pi_\theta(\cdot \mid o_{t,k})$ ▷ same θ for every k
    end for
    step the environment jointly; record $o_{t,k}, a_{t,k}, r_{t,k}$ and joint state s_t
  end for
  Estimate advantages: ▷ centralised – critic sees all sites
  $\hat{A}_{t,k} \leftarrow \textsc{Gae}(r_{\cdot,k},\ V_\phi(s_\cdot);\ \gamma, \lambda)$
  Update: for each epoch, over minibatches of the pooled $(K \times T)$ transitions
    $\theta \leftarrow \theta + \nabla_\theta \mathbb{E}\Big[\min\Big(\rho_{t,k}\hat{A}_{t,k},\ \mathrm{clip}(\rho_{t,k}, 1-\epsilon, 1+\epsilon)\hat{A}_{t,k}\Big) + \beta\,\mathcal{H}[\pi_\theta]\Big]$
    where $\rho_{t,k} = \pi_\theta(a_{t,k} \mid o_{t,k}) \,/\, \pi_{\theta_{\text{old}}}(a_{t,k} \mid o_{t,k})$
    $\phi \leftarrow \phi - \nabla_\phi \mathbb{E}\big[(V_\phi(s_t) - \hat{R}_t)^2\big]$
end for
return π_θ ▷ deployed per site; the critic is discarded

6.4 Single-site reinforcement learning

The first learning controller is a feed-forward PPO agent on a single cluster, trained on the training split, checkpoint-selected on the validation split, and evaluated once on the held-

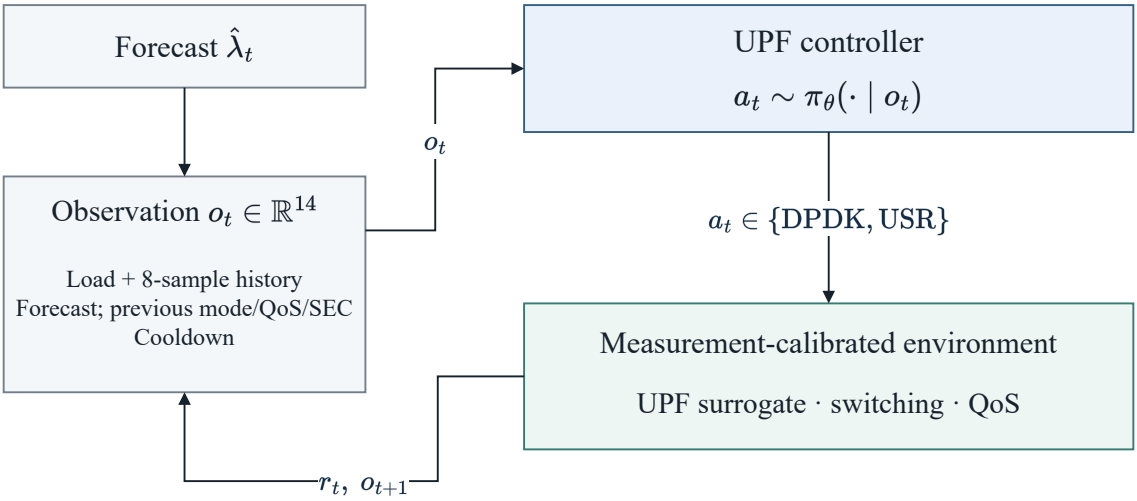


Figure 6.1: Controller–environment interaction at a 15-minute decision step. The local observation contains the current load, eight historical loads, an aligned forecast, the previous mode, QoS and specific-energy values, and cooldown progress. The controller requests DPDK or USR; the measurement-calibrated environment evaluates the transition and returns the reward and next observation. All transitions are evaluated offline through the digital twin.

out test split. On the representative low-load cluster it reduces energy and total penalty substantially relative to both a static policy and the derived threshold:

Policy	Total reward	Energy (Wh)	Unsafe %
Single-site PPO	**−880 ± 25**	**176.6 ± 1.2**	0.36 ± 0.13
Always SD-CORE DPDK	−1321.4	206.8	0.00
Threshold (derived 81 Mbps)	−1348.1	175.9	2.48

Reported over eight independently seeded runs, evaluated once each on the held-out test slice. The controller raises total reward from −1321.4 to −880 and cuts energy from 206.8 to 176.6 Wh (14.6 %) over always-SD-CORE DPDK, and improves reward and safety relative to the derived threshold (0.36 % against 2.48 % unsafe), at slightly higher mean energy consumption (176.6 versus 175.9 Wh). The seed spread is small relative to the margins (±25 reward units against a gap of 441, and ±1.2 Wh against a gap of 30 Wh), so the ordering does not depend on which seed is drawn. A plausible explanation is the learner's richer observation: it receives the eight-step history and model-derived QoS and energy signals, whereas the threshold rule uses the one-step forecast. However, the policies also differ in architecture and training. This experiment demonstrates an improvement in

reward and safety for the evaluated learning design, rather than isolating the effect of each observation component.

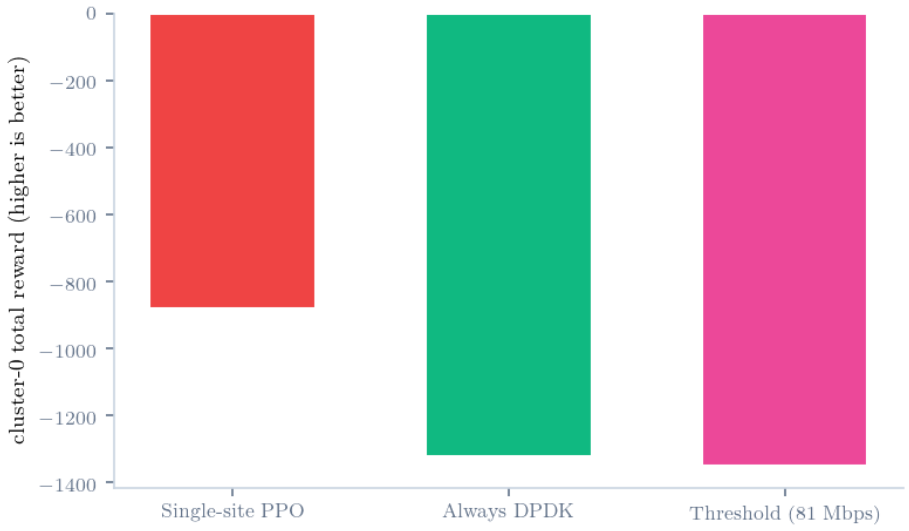


Figure 6.2: Single-site PPO against static and threshold baselines on the held-out test period for a low-load cluster. The learner improves total reward and safety relative to the threshold policy at similar energy, and reduces energy relative to static DPDK (cluster 0).

Results across clusters. The result above is for a single representative cluster, which is too narrow a base for a claim about single-site learning in general. The experiment was therefore repeated across the other nine clusters of the deployed partition, four seeds each, evaluated once per run on the held-out test slice. Cluster 0 above used eight seeds; the reduced budget here buys breadth across sites instead of depth at one, and the per-cluster spreads below are correspondingly wider.

The picture is more mixed than the single-cluster result suggests, and the distinction between the three outcomes matters.

Counting cluster 0, the learner is clearly better on six of the ten and marginally better on a seventh. On clusters 2 and 3 it converges to always-SD-CORE DPDK and its reward is identical to the static policy to the reported precision. That is not a failure: these are the highest-load sites, where USR-UPF is never the right choice, and a controller that discovers this and stops switching is behaving correctly. It does mean, however, that on a fifth of the fleet the learned controller offers no advantage over a static rule, which matters for deployment.

Table 6.1: Single-site PPO against static SD-CORE DPDK on every cluster, mean over four seeds. Energy saving is against static SD-CORE DPDK on the same cluster. Cluster 0 is omitted: it is the cluster reported above, at eight seeds rather than four, and remains the most favourable of the ten (14.5 % saving). †Static SD-CORE DPDK on cluster 9 violates on 3.67 % of steps, so almost all of that figure is shared with the static policy and reflects the model-evaluated DPDK safety response discussed in Section 5.5 rather than a consequence of the policy.

Cluster	PPO reward	Δ vs SD-CORE DPDK	Saving	Unsafe	Outcome
1	−539 ± 5	+330	7.2 %	0.30 %	better
4	−662 ± 33	+294	9.3 %	0.45 %	better
6	−723 ± 14	+233	7.4 %	0.22 %	better
7	−662 ± 65	+96	6.8 %	0.40 %	better
5	−415 ± 18	+29	2.0 %	0.12 %	better
8	−401 ± 9	+1	0.5 %	0.05 %	marginal
2	−285 ± 0	0	0.0 %	0.00 %	degenerates to SD-CORE DPDK
3	−165 ± 0	0	0.0 %	0.10 %	degenerates to SD-CORE DPDK
9	−1054 ± 15	−8	−0.7 %	3.69 %†	worse

Cluster 9 is a negative result. The learner is worse than static SD-CORE DPDK on reward, by 8 units, and consumes marginally more energy. Its violation rate of 3.69 % is higher than on the other clusters but comparable to that of static SD-CORE DPDK on the *same* cluster (3.67 %). Cluster 9 includes high-load steps at which the twin predicts DPDK delay above the budget (Section 5.5). The similar violation rates show that these verdicts occur under both policies, rather than being introduced mainly by the learner. Their correspondence to physical violations requires separate validation. The learner also fails to improve energy consumption and pays a small additional cost for switching.

The general lesson is not about safety but about heterogeneity: a controller trained on one site's load distribution is fitted to that distribution, and the site where it helps least is the one least like the others. That is an argument for the cooperative controller developed later in this chapter, whose shared policy is trained across all ten sites simultaneously.

The resulting summary is therefore narrower than the cluster 0 figure alone would support: single-site learning helps on most sites, is inert on the busiest, and can hurt on an unusual one.

6.5 Explicit versus Recurrent Temporal Information

The observation vector encodes time explicitly, through an eight-step load history. An alternative is to let the policy network carry temporal state implicitly, through recurrence: an LSTM policy that reads only the current observation and maintains its own hidden memory. Two experiments bear on the choice, and they support the same design decision with different strength.

The prototype's comparison. The single-site prototype [49] tuned both policy classes and compared five seeds each with a Wilcoxon signed-rank test. The feed-forward policy was numerically better on every metric: energy 122.6 ± 1.5 Wh against 125.3 Wh, mean SEC lower by 7 %, and violation rate lower by 45 %. However, *no difference reached significance* ($p = 0.63$, 0.38 and 0.50 respectively). With five paired samples the smallest achievable two-tailed p is 0.0625, so the test could not have declared significance for a difference of this size. Training cost was the one clear separation: roughly 690 s against 443 s.

This work's ablation. The comparison was repeated on the current environment, five seeds, 200 000 steps, holding every hyperparameter identical across the two policy classes:

	Feed-forward	Recurrent	Wilcoxon p
Test reward	−827 ± 26	−1238 ± 187	0.063
Energy (Wh)	174.6 ± 2.2	200.5 ± 14.0	0.125
Training time (s)	106 ± 4	598 ± 88	0.063

The separation here is much larger, but for a reason that qualifies it. Four of the five recurrent runs converged to always-SD-CORE DPDK, with reward exactly −1321.4, identical to the static policy, while no feed-forward run did. The recurrent policy did not lose the comparison so much as fail to leave the trivial local optimum within the budget, and required 5.6× the training time.

Interpretation. Hyperparameters were held fixed across policy classes to keep the comparison controlled, but that choice favours the architecture they were selected for; a recurrent policy given its own tuning budget might well escape the local optimum, as the prototype's tuned comparison suggests it can. The defensible statement is therefore not that recurrence cannot work here, but that it did not pay for itself: at matched budget

it was markedly harder to train and never better, and under the prototype's separate tuning it was still not better. Neither experiment provides a reason to prefer it.

Across both experiments the temporal observability the controller needs is at least as well supplied *as an explicit, inspectable feature of the observed state* as it is learned implicitly by an opaque recurrent policy, and it is far cheaper to train. This is the same design principle the whole stack embodies: make the decision-relevant signal an explicit, validated observable rather than an implicit internal quantity. The explicit history window is retained in every subsequent controller, and the recurrent variant is not pursued further.

6.6 Limitations of independent per-site learning

A per-site controller, whether rule-based or learned, sees only its own site. That is a real limitation once the deployment is a fleet of heterogeneous clusters. Some clusters carry dense, persistent demand and generate abundant experience around the switching boundary; others sit at low load for long periods and rarely enter the unsafe region at all. An independent learner at a low-load site may encounter relatively few boundary-crossing events, limiting the experience available for learning the switching decision. Sharing training experience is one way to address this imbalance.

Two multi-site designs represent the extremes. A single *centralised* PPO, observing the concatenated 140-dimensional fleet state and emitting a joint 10-dimensional action, has the information in principle but must learn one monolithic policy over a joint space; at a fixed training budget it is fragile. In this work the centralised controller collapsed: on one low-load cluster ($c6$ in the illustrated seed) it degenerated to near-always-USR-UPF, driving that cluster's QoS-violation rate well above every other controller's, and its across-seed reward variance was an order of magnitude larger than any other controller (Figure 6.3). The opposite design, an *independent* PPO per site (an IPPO ensemble), is stable and trivially parallel to train, but by construction cannot share what one site learns with another.

Independent per-site training cannot transfer experience between sites. Shared-policy learning provides that capability, although its practical value also depends on optimisation and the training budget. The following comparison evaluates whether a shared actor and centralised critic improve performance relative to the independent and monolithic alternatives.

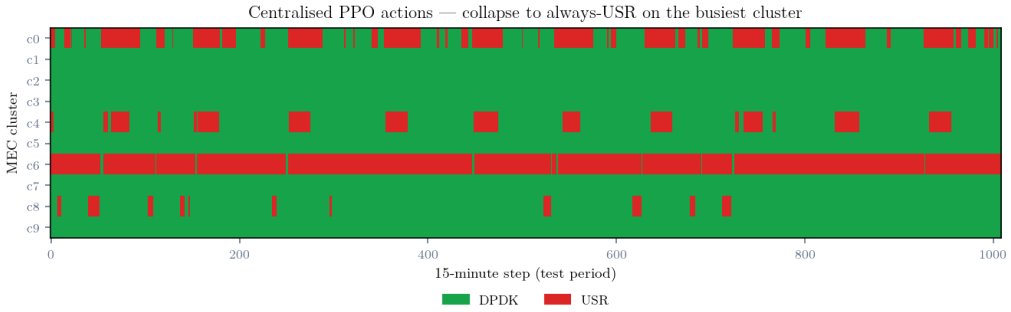


Figure 6.3: Per-cluster, per-step action of the centralised PPO on the test period (green = SD-CORE DPDK, red = USR-UPF). The solid red band (cluster c6 in this seed) is a low-load cluster collapsed to near-always-USR-UPF: the monolithic joint policy could not factor the per-site substructure at this training budget.

6.7 Cooperative multi-agent control (MAPPO)

The resolution is centralised training with decentralised execution (CTDE), realised as multi-agent PPO (MAPPO), shown in Figure 6.4. A single *shared* actor network maps one site's 14-dimensional observation to that site's action; the same parameters serve all $K = 10$ agents, so each gradient step aggregates experience across the fleet. A *centralised* critic sees the full 140-dimensional joint state during training and produces a value estimate per agent, so credit assignment respects the shared reward structure. Execution remains fully decentralised: at run time each site acts on its own observation alone.

Scope of this comparison. The learned controllers in this comparison were developed in stages and were not all trained under exactly the same reward settings; all are evaluated in the same twin, with the same reward and test period. Section 6.9 reports additional runs in which all learned controllers are retrained under one protocol, together with a shared-policy critic ablation.

Result. Across eight independent training seeds, evaluated once each on the held-out test period with the switching-cost accounting defined in Section 5.6:

MAPPO beats every baseline, including the best classical controller, whose thresholds are derived from the surrogate and whose band alone is tuned (Section 6.3). It does so at the lowest QoS-violation rate of any *trained* controller and with energy indistinguishable from the ensemble. Static SD-CORE DPDK is nonetheless safer still (0.38 % against MAPPO's 0.53 %), which is expected: it never runs USR-UPF and so cannot incur USR-UPF's QoS risk.

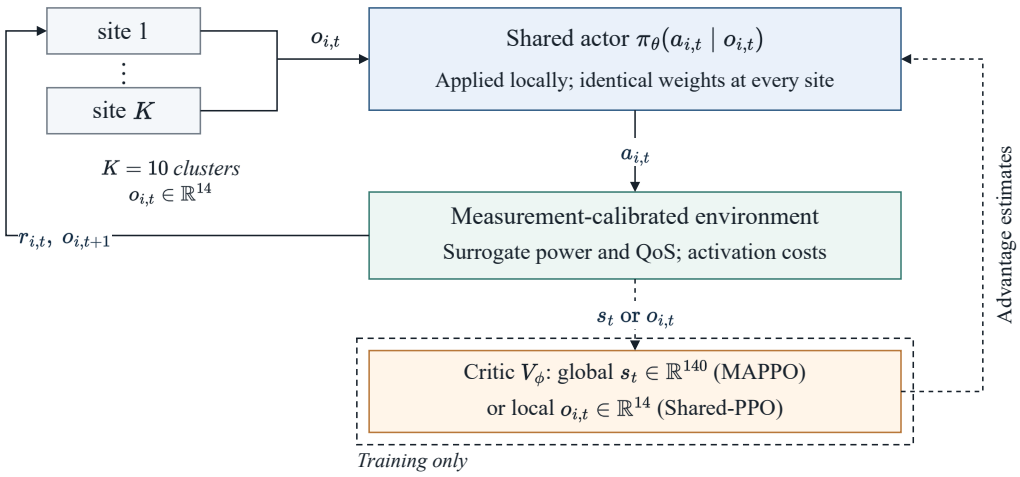


Figure 6.4: Shared-policy control with the two critic alternatives used in the MAPPO/Shared-PPO comparison. Each of the $K = 10$ sites supplies a local 14-dimensional observation to the same actor. MAPPO's training critic receives the concatenated 140-dimensional state and produces a value estimate per site; Shared-PPO uses a shared local critic. Dashed paths are used during training to form advantage estimates. Execution uses the shared actor and each site's local observation only.

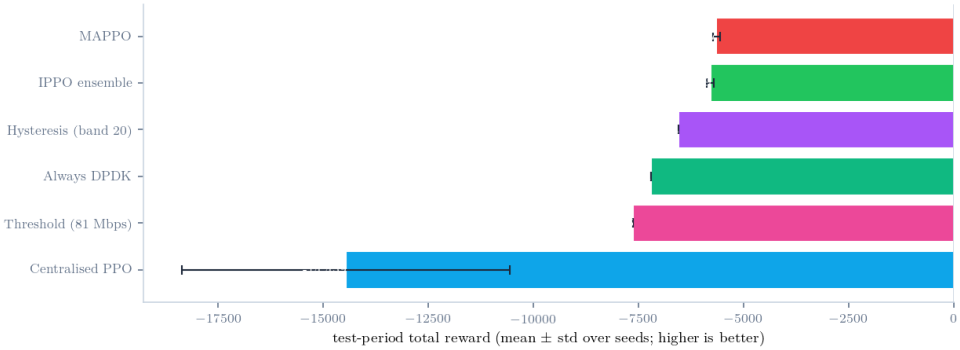


Figure 6.5: Test-period reward, mean over eight training seeds, with standard deviation. MAPPO is the best controller and the most stable trained one.

This safety comes at the cost of higher energy consumption than the threshold, hysteresis, and decentralised learning policies; centralised PPO consumes more energy still. The claim here is not that MAPPO is the safest possible policy, but that it occupies the best point on the energy–safety trade-off among policies that attempt the trade-off at all. The margin over the IPPO ensemble ($\approx$ 141 reward units) exceeds the combined seed standard deviation, and a bootstrap over the eight seeds confirms the ordering is significant.

Both paired and unpaired intervals are reported, since training seeds are not matched experimental units. Training seeds are not matched experimental conditions in the way

Table 6.2: Test-period comparison across controllers (mean ± population standard deviation over seeds where trained).

Controller	Reward	Energy (Wh)	Unsafe %	Seeds
MAPPO (CTDE)	**−5647 ± 86**	1967	**0.53**	8
IPPO ensemble	−5789 ± 83	1974	0.57	8
Hysteresis (band 20, cd 1)	−6544	1971	0.94	det.
Always SD-CORE DPDK	−7201	2071	0.38	det.
Threshold (derived 81 Mbps)	−7623	1970	1.42	det.
Centralised PPO	−20003 ± 6776	2185	6.22	8

that repeated measures on the same subject are; a seed fixes network initialisation and environment stochasticity, so pairing removes some shared variance, but it does not make the two ensembles paired observations of one underlying unit. The conclusion does not depend on the choice:

Procedure	Mean Δ	95% interval
Paired bootstrap (by seed)	141	[26, 238]
Unpaired bootstrap	142	[54, 220]
Welch t-test	n/a	$p = 0.008$
Mann–Whitney U (distribution-free)	n/a	$p = 0.002$

All four exclude the null, and the unpaired interval, which assumes nothing about correspondence between seeds, is the *narrower* of the two, so pairing is not doing the work. MAPPO exceeds the ensemble on seven of the eight seeds. The asymmetry of the paired interval, noticeable at $[26, 238]$ around a mean of 141, is traceable to the single discordant seed, on which the ensemble wins by 207 reward units; resampling that one observation in or out skews the bootstrap distribution. Reporting it alongside the unpaired interval is the more conservative presentation.

What the interval is over. The unit being resampled is the training seed, so all four procedures answer one question and not another: would a fresh training run of MAPPO beat a fresh training run of the ensemble, on this deployment and this test window? A significant result there establishes that the ordering is a stable property of the two algorithms rather than a favourable initialisation; in other words, it establishes training stability. It is not evidence that the ordering would survive a different city, a different traffic mix, or a different pair of UPF realisations. A seed is not a draw from the population

of deployments, and this thesis has one deployment; seed resampling cannot substitute for an independent deployment. The nearest available evidence on that other axis is weaker by construction: the per-cluster breakdown below and the fleet-size sweep of Section 6.8 both repartition a single city's trace rather than introducing an independent site.

Comparison with the clairvoyant reference. Chapter 5 reports a reference policy using actual load, which saves 6.7 % relative to static DPDK under the declared calibration. This benchmark helps interpret the saving of forecast-driven controllers, but does not establish a globally optimal action sequence under switching costs. The following ratios express performance relative to that reference.

Controller	Energy saving	Relative to reference	Unsafe %
Oracle (reference)	6.68 %	100 %	0.00
MAPPO (CTDE)	4.82 %	72 %	0.53
Hysteresis (band 20)	4.80 %	72 %	0.94
Threshold (derived 81)	4.85 %	73 %	1.42

These ratios show that on the *energy* axis the three controllers are indistinguishable: each achieves a little under three-quarters of the reference saving. The remaining gap may reflect forecast error, policy design, and switching behaviour; these comparisons do not establish that it is unattainable. Learning provides little additional energy saving in this regime.

What differs is the price paid in QoS to obtain that energy. The threshold rule gives up 1.42 % of steps, hysteresis 0.94 %, and MAPPO 0.53 %, a factor of nearly three between the extremes at equal energy. The contribution of the learned controller in this regime is therefore *efficiency of the trade-off*, not an improvement in the headline saving. The benefit is a reduction in QoS violations at comparable energy consumption.

The narrowness is itself a property of the operating regime rather than of the method: with only 13.7 % of steps below the decision threshold (Section 5.9), there is little room for any controller to distinguish itself on energy. The sensitivity table in that section shows the headroom widening rapidly as the calibration falls, which is where a larger separation between controllers would be expected to appear.

Where the win comes from. The advantage is not uniform; it is largest on several mid- and low-load sites. Figure 6.6 places the independent ensemble and MAPPO side by side per

cluster. On high-load clusters (c3, c9) every controller correctly stays on SD-CORE DPDK and the bars coincide. The gains appear on the mid- and low-load, data-poor clusters (c0, c4, c5, c7), where the shared policy outperforms independently trained policies. Pooling experience across sites is consistent with this pattern, but the comparison also changes critic structure. The per-cluster result supports the shared-actor, centralised-critic design without isolating either mechanism.

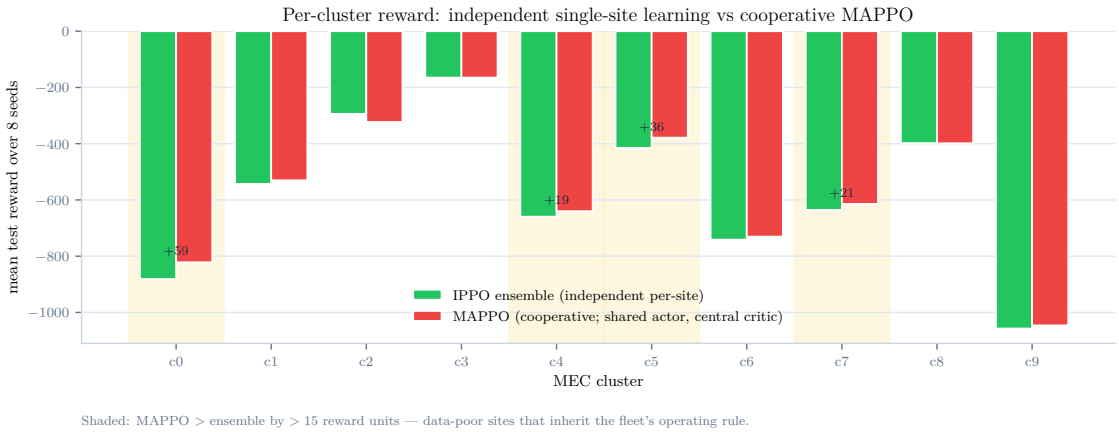


Figure 6.6: Per-cluster test reward, independent single-site learners (IPPO ensemble) versus cooperative MAPPO, mean over eight seeds. Shaded clusters are where MAPPO exceeds the ensemble by more than 15 reward units, identifying sites with the largest observed benefit from the shared-policy design.

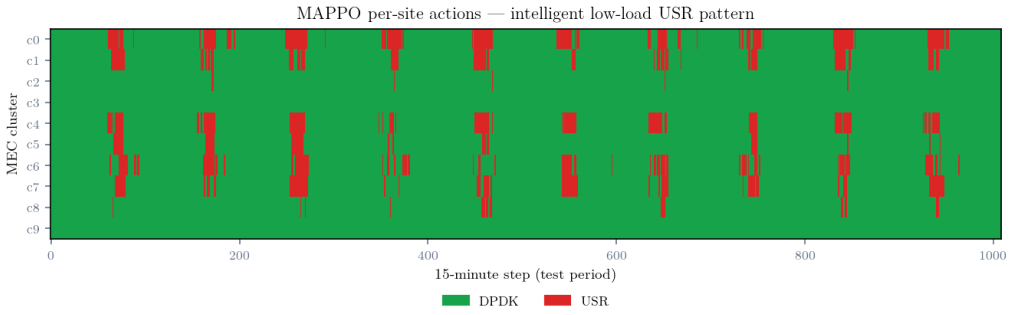


Figure 6.7: MAPPO per-site, per-step actions on the test period. The low-load clusters show the expected pattern of selecting USR-UPF at low predicted load, which the collapsed centralised policy (Figure 6.3) did not learn.

Stability and safety. MAPPO is also the most seed-stable trained controller in this comparison, and the safety–reward trade-off separates the architectures cleanly: the trained

cooperative and independent controllers cluster in the high-reward, low-unsafe corner, while the centralised policy scatters along the safety axis, visual evidence that the wrong coordination structure carries a real safety cost, not merely a reward cost (Figure 6.8).

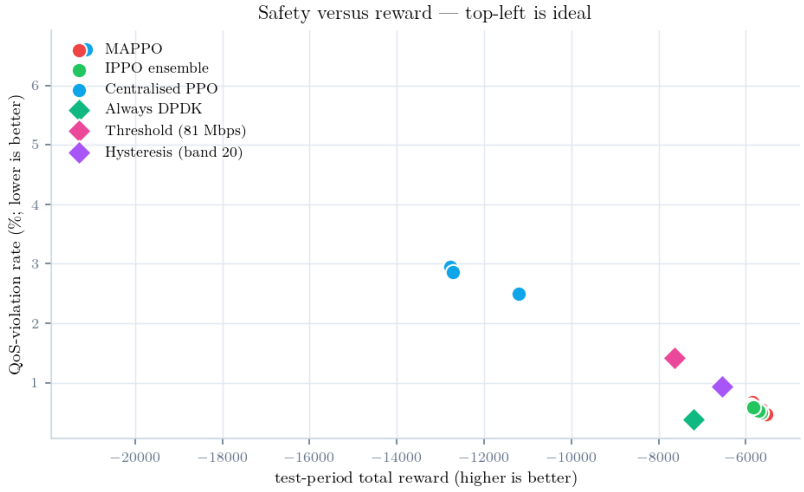


Figure 6.8: Safety versus reward on the test period. Top-left is ideal. Cooperative and independent learners (eight seeds each) cluster there; the centralised policy spreads along the unsafe axis.

6.8 Sensitivity to Fleet Size

The headline comparison is made at $K = 10$ clusters, which raises the question of whether MAPPO's advantage over classical control is a property of that particular partition. It also raises a second, subtler one: the load calibration α and the cluster count K are not independent. Total offered traffic is conserved and divided among clusters, so per-cluster load scales as $1/K$ and the effective operating point is the ratio α/K. Sweeping K at fixed α would therefore mostly re-measure the load sensitivity already reported in Section 5.9.

To isolate fleet size from operating regime, α is scaled with K so that α/K is held constant, placing every configuration at the same per-cluster load. Six clusterings of the same Lyon deployment are used, from $K = 4$ to $K = 100$, each with eight training seeds.

The ordering is invariant; the margin is not. At every fleet size the ranking is the same: MAPPO, then hysteresis, then the threshold rule, then static SD-CORE DPDK, then static USR-UPF. Only the first two are shown in Table 6.3, which reports the closest competitor; the

Table 6.3: MAPPO advantage over hysteresis as a function of fleet size, at constant per-cluster load ($\alpha/K = 0.05$). Reward is given per site per step (total reward divided by K and by the 1009 evaluation steps), because the total scales with fleet size and a growing total would say nothing on its own; the total margin is retained alongside for reference. MAPPO is the mean over eight seeds with the seed standard deviation shown; the classical controllers are deterministic. *Note that no row reproduces Table 6.2*, which is computed at $\alpha = 1.0$, $K = 10$, i.e. at $\alpha/K = 0.1$, twice the per-cluster load used here. The $K = 10$ row below sits at $\alpha = 0.5$.

		Reward per site per step			Total	Unsafe steps	
K	α	MAPPO	Hyst.	Margin	margin	MAPPO	Hyst.
4	0.20	-0.556 ± 0.005	-0.572	0.016	63	0.05 %	0.10 %
6	0.30	-0.605 ± 0.006	-0.628	0.022	136	0.15 %	0.20 %
10	0.50	-0.659 ± 0.005	-0.744	0.085	860	0.24 %	0.58 %
20	1.00	-0.700 ± 0.001	-0.797	0.097	1951	0.31 %	0.70 %
50	2.50	-0.726 ± 0.003	-0.960	0.234	11814	0.35 %	1.44 %
100	5.00	-0.813 ± 0.007	-1.183	0.370	37305	0.59 %	**2.24** %

full comparison at each K yields the same ordering. The conclusion of Section 6.7 is therefore not an artefact of the $K = 10$ partition.

What does change, and change monotonically, is the size of the advantage. It is reported per site and per step as well as in total, because a fleet twenty-five times larger accumulates roughly twenty-five times the reward and a growing total would say nothing on its own. Normalising removes that scaling and the growth survives it: the per-site, per-step margin rises from 0.016 at $K = 4$ to 0.370 at $K = 100$, a factor of twenty-three. Absolute units are used rather than a ratio because the reward is negative, unbounded below, and has no meaningful zero. Against the seed standard deviation the growth is unambiguous: at $K = 4$ the margin is roughly three seed standard deviations, at $K = 100$ over fifty.

What drives the widening. Holding α/K constant fixes the *mean* per-cluster load at 234 Mbps for every K in the sweep, but it does not fix the shape of the per-cluster series. A finer partition of the same city puts fewer base stations in each cluster and so multiplexes less traffic together: the coefficient of variation of the per-cluster load rises from 1.00 at $K = 4$ to 1.40 at $K = 100$, and the largest per-cluster excursion in the test window grows from 1.5 to 7.8 Gbps. The fraction of steps above the decision point barely moves,

remaining between 0.64 and 0.68 at every K, so the operating regime is not shifting; what changes is how sharply the load moves within it.

That is a burstier control problem, and it penalises a fixed-width band more than it penalises a policy with state. The hysteresis rule sees only whether the forecast lies inside a 20 Mbps guard; the learned controller sees eight steps of history, the forecast, and the twin's QoS and energy signals, and can act ahead of an excursion rather than after it. Both controllers do degrade per site as K grows: MAPPO from -0.556 to -0.813, hysteresis from -0.572 to -1.183. This is the signature of a harder problem rather than of one controller scaling and the other collapsing. MAPPO degrades roughly 2.4 times more slowly.

What the sweep does not establish. It compares MAPPO against the classical baselines only, so the widening margin cannot be attributed to *cooperation* specifically. The paragraph above credits history, forecast and QoS state, inputs that an ensemble of independent per-site learners also has. An IPPO ensemble might therefore track the same trend, in which case the widening would be a benefit of learning at scale rather than of parameter sharing between sites. Separating the two would require running the ensemble across K, which is not done here: the ensemble trains one policy per site, so its cost grows with K exactly where the comparison would matter most. What the sweep does establish is narrower and still useful: the ranking reported at $K = 10$ is not an artefact of that partition, and it does not weaken as the fleet grows. The cooperative-versus-independent comparison at $K = 10$ in Section 6.7, where the ensemble is actually run, remains the only evidence in this thesis that isolates cooperation.

Safety widens too. The classical controller degrades on QoS as K grows, from 0.10 % to 2.24 % unsafe steps, while MAPPO rises only from 0.05 % to 0.59 %: at one hundred sites the cooperative controller is ahead by 37 305 reward units, over fifty times the seed standard deviation, and violates QoS on roughly a quarter as many steps. The same burstiness accounts for it. A band of fixed physical width guards against forecast error of a fixed size, but the excursions it must absorb grow with K while the band does not, so the rule is more and more often on the wrong side of the QoS limit when a spike arrives. The learned controller does not share this failure because it acts on the twin's counterfactual QoS signal rather than on the forecast alone.

6.8.1 Load-calibration sweep

Load calibration α changes the traffic presented to a fixed surrogate. The sweep records contain 80 MAPPO runs: four $K = 10$ load calibrations and the six fleet sizes above at constant $\alpha/K = 0.05$, each with eight seeds numbered 1–8. Table 6.4 reports the load calibrations at $K = 10$.

Table 6.4: MAPPO load-calibration sweep at $K = 10$: mean reward ± sample SD, deterministic classical rewards and mean MAPPO model-predicted QoS violations.

α	MAPPO reward	Hysteresis	Always SD-CORE DPDK	MAPPO QoSv (%)
0.25	-8068 ± 40	-8468	-24817	0.27
0.5	-6661 ± 42	-7724	-12413	0.24
0.75	-5734 ± 62	-6969	-8406	0.27
1	-5610 ± 32	-6544	-7201	0.50

MAPPO leads hysteresis and always-SD-CORE DPDK at every calibration. The fleet-size reward totals are normalised by $K \times 1009$ before comparing fleet sizes. Neither sweep introduces a new city or a new physical measurement campaign.

6.9 Additional runs: shared actor with local or centralised critic

After the experiments above, a further set of runs was carried out for the conference paper [50]. Its purpose was to separate the two ingredients of MAPPO, the shared actor and the centralised critic, by adding a Shared-PPO controller that uses the same shared-actor trainer as MAPPO with a shared *local* critic. For a like-for-like comparison, MAPPO, Shared-PPO, IPPO and centralised PPO were all retrained in this set under one environment version, with the same eight seeds $\{1, 7, 13, 23, 42, 64, 77, 99\}$, the chronological train, validation and test slices (5073, 1009 and 1009 intervals), a 200 000-step budget and validation-only checkpoint selection, followed by deterministic evaluation on the full ten-cluster test episode. These runs form a separate set: their numbers are reported here and do not replace those of the preceding sections.

The hysteresis reference in this set uses a 50 Mbps band ($\lambda_{\downarrow} = 31$ Mbps) instead of the 20 Mbps band used above. The two bands encode different objectives: the 20 Mbps band returns to USR-UPF as early as possible and favours energy, whereas the 50 Mbps band is the

value that maximises the reward of Equation (6.1) in a sweep over the test trace. Because it is tuned on the evaluation data, it is an optimistic reference for the learned policies.

Table 6.5: Results of the additional runs: reward mean ± sample SD and median over eight seeds (all seeds retained), mean attributed energy and model-predicted QoS-violation rate. Classical policies are deterministic.

Controller	Reward	Median	Energy (Wh)	QoSv (%)
MAPPO	−5624 ± 22	−5626	1967.3	0.52
Shared-PPO	−5707 ± 77	−5704	1963.1	0.57
IPPO	−8236 ± 6912	−5794	2026.3	1.47
Centralised PPO	−15025 ± 6931	−13365	2131.6	4.03
Hysteresis (50 Mbps band)	−6262	−6262	1994.8	0.68
Always SD-CORE DPDK	−7201	−7201	2070.7	0.38

MAPPO again attains the best reward and the smallest seed spread. Relative to the reward-tuned hysteresis it reduces the combined cost by 10.2 % and energy by 1.4 %; relative to always-SD-CORE DPDK the reductions are 21.9 % and 5.0 %. The IPPO mean is strongly affected by seed 99, which collapsed on one cluster (reward −25341); its median (−5794) describes a typical seed. The seed is retained, because excluding it would conceal a real training instability, but the difference between the MAPPO and IPPO means in this set should not be read entirely as a benefit of cooperation. Centralised PPO remains worse than always-SD-CORE DPDK in every seed.

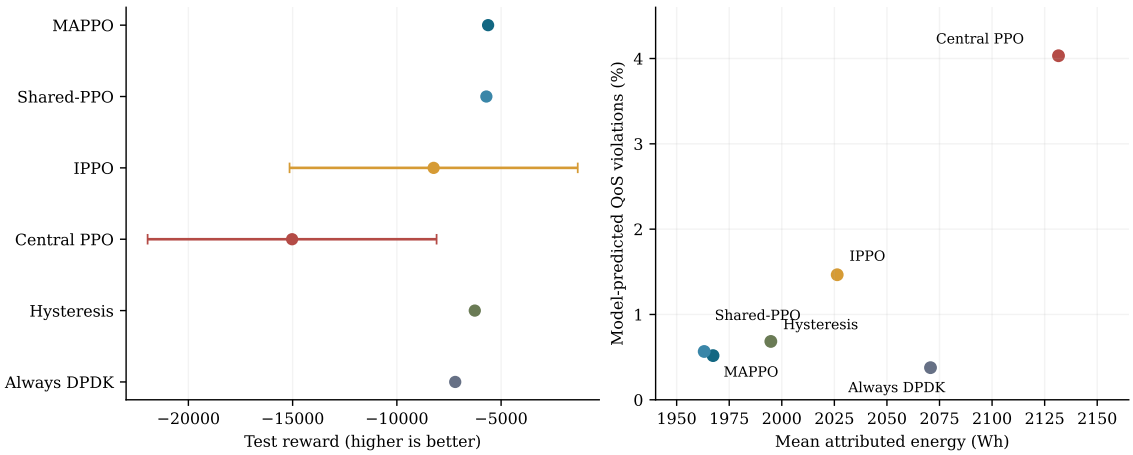


Figure 6.9: Additional runs. Left: reward mean with sample SD; right: attributed energy and model-predicted QoS violations. Deterministic baselines have no training-seed error bar; “Hysteresis” is the 50 Mbps band.

6.9.1 Shared actor, local or centralised critic

The matched MAPPO–Shared-PPO reward contrast is 82.9 units, with a seed bootstrap 95% interval of $[31.0, 150.9]$. MAPPO wins on seven of eight seeds; the exact two-sided permutation test gives $p = 0.0234$, or 0.0469 after Holm adjustment within the reward-comparison family. This is approximately 1.45% of Shared-PPO's mean cost. Physical-energy separation is weaker: MAPPO uses 4.2 Wh more on average, with an interval of $[-2.7, 10.3]$ Wh and $p = 0.289$. Higher reward therefore does not establish a physical-energy win over Shared-PPO.

Under identical reward weights, Shared-PPO favours greater USR-UPF use and lower energy, whereas MAPPO accepts some additional energy to reduce QoS penalties. Figure 6.10(a) locates this trade-off by load. Below 25 Mbps both policies almost always select USR-UPF. In the 50–81 Mbps band, Shared-PPO selects USR-UPF in 27.3 % of steps against MAPPO's 18.4 %, although the two agree on 98.5 % of all cluster-steps. All 876 additional USR-UPF assignments made only by Shared-PPO are safe at that moment, so their consequence cannot be judged from current-load feasibility alone: a subsequent rise in demand exposes the outgoing USR-UPF to unsafe load while SD-CORE DPDK activates. MAPPO's more conservative choices reduce this exposure (14.1 penalised transitions per test episode against 19.1). Overall its QoS cost is 106.9 units lower, offset by 25.4 additional SEC-cost units, with activation and cooldown adding a further 1.4 units (Figure 6.10(b)).

The cluster breakdown (Figure 6.10(c)) shows that the sequence of operating conditions, rather than mean load alone, determines the benefit. At c3 and c9, USR-UPF opportunities are rare and the policies barely differ. At c5, c7 and c8, low-load intervals recur within otherwise higher-load operation, and avoiding USR-UPF assignments just before a load increase produces the largest gains. Cluster c1 also has low-load opportunities but fewer direct jumps from below 81 to above 149 Mbps, so MAPPO's caution forfeits more saving than it recovers (median gain −2.74); c2 is also slightly negative (−0.44).

This supports a difference in learned exposure to modelled risk; it does not establish the internal reason the critic produced that behaviour, since critic values and advantages were not recorded. Checkpoint selection and optimisation variability remain alternative explanations. Validation-based selection also has one specific limitation: 684 near-zero-load cluster-steps dominate the validation SEC-based return, whereas the test period has none, so the selected checkpoint depends on that validation distribution.

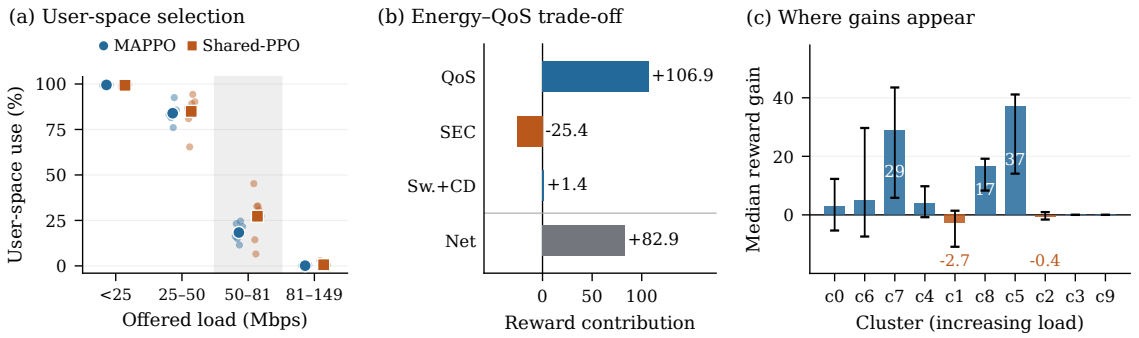


Figure 6.10: Energy–QoS behaviour of the two shared policies (MAPPO and Shared-PPO). (a) USR-UPF use by offered-load band, with individual seeds and means. (b) Contributions of the QoS, SEC and combined activation/cooldown terms to MAPPO's net reward gain over Shared-PPO. (c) Median reward gain per cluster, ordered by mean load, with whiskers spanning the middle 50 % of seeds; orange marks negative medians.

6.9.2 Sensitivity to surrogate calibration

Because the environment is fitted from a finite profiling campaign, the controller ranking could in principle reflect the surrogate fit rather than the policies. The saved policies of this set are therefore evaluated, without retraining, in ten alternative surrogate calibrations: five leave-one-sweep-out and five leave-one-load-group-out refits that retain the deployed model configurations (Section 5.7.1). Closed-policy replay lets observations and actions change with the refitted model; fixed-action replay keeps the original action sequence and recomputes its outcomes.

MAPPO beats IPPO and hysteresis in all ten refits and ranks first among all six controllers in eight (Figure 6.11(a)). All five leave-one-sweep-out refits favour MAPPO; the two exceptions are load-group refits. Group L2 omits the 1, 10, 100 and 1000 Mbps levels, removing the measured 100 Mbps point near the USR-UPF QoS transition; the refitted model predicts the withheld (measured-safe) samples unsafe, and the predicted safe-range endpoint drops from 149 to 81 Mbps (Figure 6.11(b)), so always-SD-CORE DPDK ranks first. Group L3 omits 2, 20, 200 and 2000 Mbps and moves the endpoint up to 170 Mbps, giving Shared-PPO a 27.3-unit advantage. MAPPO's actions change by at most 0.171 % (Figure 6.11(c)), and the winning controller is the same under closed-policy and fixed-action replay in every refit: recalibration mainly changes the estimated consequences of almost identical decisions. These refits come from the same campaign; they quantify sensitivity to calibration, not the physical boundary errors of Section 5.7.1.

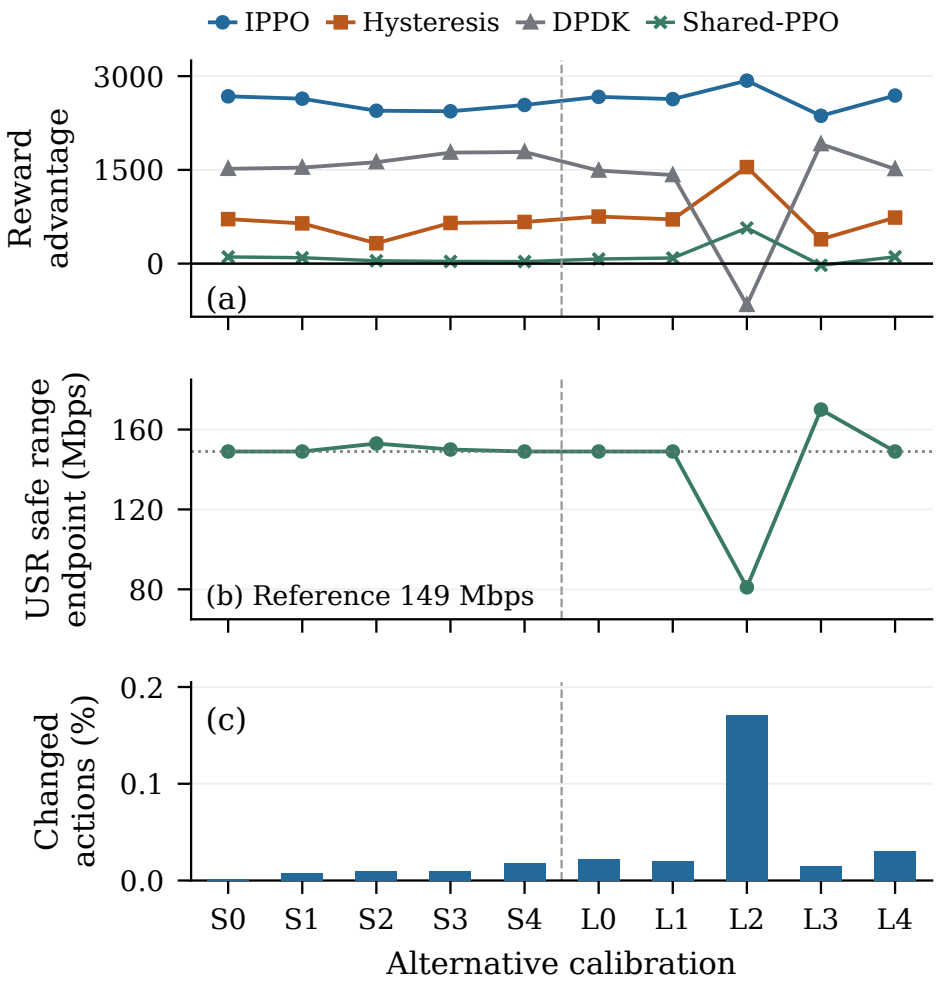


Figure 6.11: Sensitivity to surrogate calibration across five omitted sweeps (S0–S4) and five omitted load groups (L0–L4). (a) MAPPO's mean reward advantage over each controller. (b) Predicted onset of USR-UPF QoS violations, expressed as offered load; the dotted line is the reference 149 Mbps. (c) MAPPO actions that differ from the reference calibration.

6.9.3 Sensitivity to reward weights

The saved decisions of all six controllers were also rescored with each of the four cost weights (SEC, QoS, switching and cooldown) multiplied by 0.5, 1 or 2, giving $3^4 = 81$ combinations. MAPPO ranks first in all of them, with mean reward margins of 3.3–203.6 units (Figure 6.12). The smallest margin occurs when SEC receives twice its weight and the other costs half, which makes Shared-PPO's energy-saving behaviour more attractive.

This rescoring tests the recorded policies; it is not an evaluation of policies retrained under different objectives.

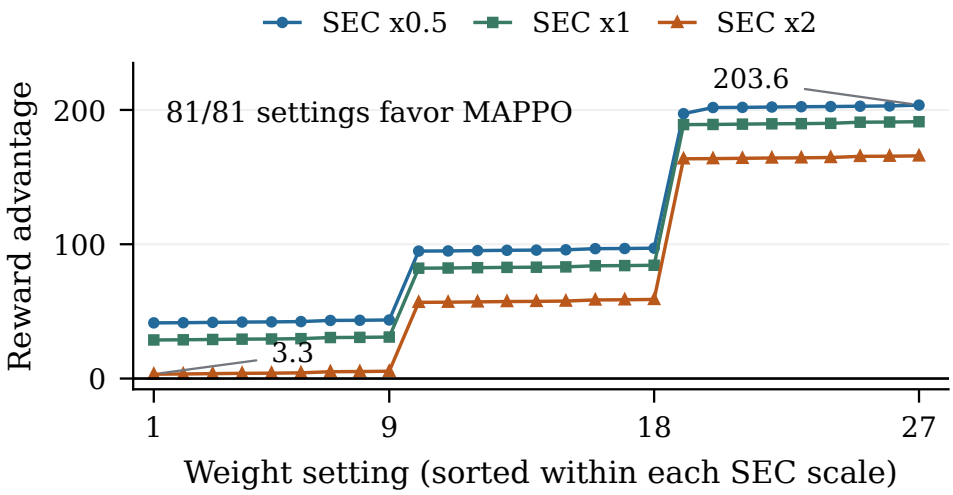


Figure 6.12: Ranking sensitivity to the reward weights, using the saved decisions of all six controllers. Each curve sorts the 27 QoS, switching and cooldown weight combinations for one SEC multiplier. All 81 margins are positive.

6.10 Synthesis: what the control results do and do not establish

The controllers form a ladder, and each rung consumes more of the augmented-observability pipeline than the one below it. The derived threshold acts on a single inferred signal, the one-step forecast, and is brittle precisely because that is all it sees. The single-site learner adds the twin's counterfactual QoS and energy signals and an explicit temporal history, and improves reward and safety relative to the threshold at similar energy. The cooperative fleet controller adds cross-site policy transfer, making one site's experience visible to another, and beats the best classical controller by a margin larger than seed noise, at the lowest QoS-violation rate of any learner.

The chapter draws on two sets of runs, and they answer different questions. The runs of Sections 6.4–6.8 compare shared and independent learning against an energy-focused classical reference: the hysteresis band of 20 Mbps returns to USR-UPF as early as possible, and the comparison is read mainly in energy and QoS-violation terms. They establish that a shared policy outperforms independent and centralised learners at $K = 10$, and that its

lead over classical control holds across load calibrations and fleet sizes. The additional runs of Section 6.9, carried out for the conference paper, focus instead on the combined reward. Their classical reference is therefore the hysteresis band optimised for that reward (50 Mbps), and they add a Shared-PPO controller to isolate what the centralised critic contributes beyond the shared actor, together with sensitivity to surrogate calibration and reward weights. Read together, the two sets support the same ordering: shared-policy learning first, classical hysteresis next, static SD-CORE DPDK last. Absolute rewards differ between the sets because the controllers were retrained and the classical reference changed; the independent learners in the additional runs include one collapsed seed, so the cooperative-versus-independent margin is best read from the first set.

These comparisons evaluate complete controller designs. The threshold and learning policies differ in observation content, policy class, and training, so their performance differences do not isolate the causal value of any one inferred signal. The MAPPO–IPPO comparison at $K = 10$ supports the benefit of the shared-actor, centralised-critic design relative to independent policies; the Shared-PPO ablation of Section 6.9 tests critic structure while retaining a shared actor, and finds a modest reward advantage under the deployed calibration. It does not isolate every optimisation or selection mechanism. Controlled input ablations would still be required to attribute each improvement to a particular observability mechanism.

These comparisons assess control performance within a common model. They do not independently validate the twin: every policy is evaluated against the same surrogates, and the classical thresholds are derived from those surrogates. A policy optimised against a surrogate can also exploit its errors [19]. Independent physical evaluation is therefore needed to establish performance in deployment, as illustrated by learned network-control studies [40, 42, 39].

What the results do establish is *internal consistency*: the attributed power, the forecast and the counterfactual QoS are mutually coherent, and are informative enough that a policy acting on them makes systematically better decisions than rules acting on fewer of them. This supports their use for control within the modelled operating conditions.

The evidence that bears on *fidelity* is separate and lives elsewhere: the held-out surrogate procedure against physical measurement (Sections 3.9.2 and 5.7.1), including its poor resolution near the QoS boundary; the sensitivity of the conclusions to the declared load calibration (Section 5.9); and the invariance of the controller ordering across fleet sizes (Section 6.8). Two limits remain. The load calibration is a declared assumption, not a measured constant, and the operating regime follows from it. And the evaluation is

an offline replay, not a live deployment; the measurements that would close the gap are identified in Chapter 7.

6.11 Summary

This chapter trained and evaluated controllers that act on the inferred signals assembled in Chapters 3–5. The principal points are:

1. **Progressive use of inferred signals.** The controllers are ordered by how much of the augmented-observability pipeline each consumes: a single-signal threshold rule, a single-site learner that adds counterfactual QoS and energy plus temporal history, and a cooperative fleet learner that adds one site's experience to another's.

2. **Cooperation improves performance.** MAPPO attains the best test reward in both sets of runs. Against independent learners its advantage is concentrated on several low- and mid-load clusters, consistent with a benefit from pooled experience. In the additional runs, which focus on the combined reward, its margin over Shared-PPO (same shared actor, local critic) has a positive seed-bootstrap interval but changes sign in one alternative calibration, so the critic's contribution is modest.

3. **Coordination structure matters more than capacity.** The centralised policy, given the same observation content, is the worst controller in the study and the least stable across seeds; the result is consistent with difficulty optimising the joint action space under the study's training budget, rather than proving an inherent limitation of centralised control.

4. **Safety–energy trade-off.** Static SD-CORE DPDK remains the safest option in the twin, and the learned controllers trade a small amount of safety for a reduction in energy. The clairvoyant reference of Chapter 5 contextualises that saving, while static SD-CORE DPDK provides a safety reference. The critic comparison of the additional runs does not establish an energy advantage of MAPPO over Shared-PPO.

5. **Evaluation domain.** Results concern offline replay of the Lyon traces through the measurement-grounded twin. Physical deployment remains necessary to assess the transfer of these policies to a live network (Section 6.10).

CHAPTER 7

General conclusion

7.1 Synthesis

This thesis has investigated augmented observability as a means of supplying the information required for energy-aware UPF control. Physical profiling, traffic forecasting and a digital twin provide complementary estimates of attributed power, future service-area demand and the consequences of selecting an alternative UPF realisation. Controller experiments then test how those estimates can be used, including the effect of sharing policies across sites. The resulting pipeline connects measurement and inference to a concrete network decision while making the boundaries of the supporting evidence explicit.

The contribution spans the complete path from the quantity to be estimated to the decision that consumes it. The forecasting target is chosen at the service-area scale; the twin exposes the assumptions governing power, performance and transitions; and controller evaluation reports the resulting energy and QoS behaviour alongside the combined objective. This connection allows the component models to be assessed in relation to their use, as well as by their individual prediction scores.

7.2 Answers to the research questions

RQ1 (measurement and attribution). The profiling study shows that the recorded measurements support useful models of attributed power and performance over the studied operating range (Chapters 3 and 5). Power prediction and safety classification nevertheless require different judgements. A strong whole-range regression fit does not ensure that the model resolves the QoS transition accurately. Boundary-focused validation exposes false-safe and false-unsafe predictions, making this region a priority for further measurement (Section 5.7.1). The models characterise the two recorded deployments, with their implementation and hardware effects remaining combined.

RQ2 (forecasting granularity). The forecasting study establishes that target granularity is an important modelling choice. At a comparable model budget, forecasting the cluster aggregate achieves 0.21–0.23 skill over persistence, whereas forecasting individual stations and summing their outputs does not improve on persistence (Section 4.11). Geographic contiguity defines a meaningful local service area; the partition comparison shows that it does not explain all of the predictive gain. The separate decision analysis finds that forecast error changes 6.6 % of threshold decisions under its stated calibration (Section 4.13). Forecast quality should therefore also be assessed at the decision it supports.

RQ3 (counterfactual evaluation). The digital twin supplies a common method for evaluating candidate UPF realisations, including activation delay and switching energy (Chapter 5). Its operating thresholds follow from the fitted models, traffic scaling and QoS budgets. They make the assumptions of the comparison explicit, but remain model-derived operating points whose physical fidelity requires independent measurement. The clairvoyant replay supplies a reference under realised load, without establishing a globally optimal switching sequence. The twin enables repeatable offline comparisons with consistent accounting across controllers.

RQ4 (shared-policy control). The controller results favour shared-policy learning under the evaluated protocol. In the multi-site comparison of Section 6.7, MAPPO outperforms independently trained per-site learners by 141 reward units (unpaired bootstrap 95 % interval $[54, 220]$, Mann–Whitney $p = 0.002$), winning on seven of eight seeds, with the largest gains on mid- and low-load clusters. In the additional runs of Section 6.9, in which all learned controllers are retrained under one protocol, MAPPO again obtains the highest mean reward across eight training seeds. Its 82.9-unit advantage over Shared-PPO has a positive seed-bootstrap 95 % interval of $[31.0, 150.9]$, while the energy comparison does not establish an energy benefit. The additional reward reflects a different balance of attributed energy and model-predicted QoS penalties. Static SD-CORE DPDK remains the lowest-violation reference, and Shared-PPO consumes slightly less mean energy. Relative to the test-tuned hysteresis reference, MAPPO reduces the combined cost by 10.2 % under the stated weights.

In the additional runs, one failed IPPO seed is retained, and its influence on the mean is reported alongside the median and spread. The mean difference from IPPO consequently includes training instability as well as typical policy performance. The Shared-PPO comparison examines critic structure within a shared-actor design; establishing that pooled

experience specifically benefits data-poor sites requires a targeted data-budget or transfer experiment.

RQ5 (integrated evidence and validity). The experiments demonstrate that the inferred quantities can be combined and used for control within the studied modelled conditions. Calibration analysis bounds this result: MAPPO ranks first among the six compared controllers in eight of ten alternative surrogate calibrations, with reversals favouring SD-CORE DPDK and Shared-PPO in different refits (Section 6.9.2). This is evidence of conditional robustness within the recorded campaign. The refits do not constitute ten independent physical validations.

The fleet-size and load-calibration studies provide complementary evidence under their respective protocols. The former compares MAPPO with classical control from four to one hundred sites; the latter examines rule-based ordering across load calibrations. These studies do not extend the critic ablation to new hardware or independent traffic sources. Repeated training seeds quantify variability under the fixed evaluation protocol and test trace. Physical deployment fidelity remains a separate empirical question.

7.3 Limits and future work

The principal limitation is the absence of live closed-loop evaluation. Controller performance is measured in the same family of calibrated environments used to generate the inferred power and QoS signals. The surrogates are grounded in physical data, but model-based safety near the operating boundary is uncertain, and the exact deployed surrogate artefacts have not been assessed against an independent measurement campaign. The trace-to-load mapping is also a scenario assumption, and the recorded deployments do not establish hardware equivalence.

The next experimental step is a boundary-focused profiling campaign on documented target hardware, followed by independent validation of the exact deployed models and end-to-end measurement of the switching process. A subsequent closed-loop test should assess whether the offline behaviour survives actuation delay, measurement error and changes in operating conditions. Monitoring prediction error during operation would make the reliability of the inferred information itself an observable quantity.

Further controller studies should separate the benefit of pooled experience from critic structure and optimisation variability through controlled data-budget and transfer experi-

ments. Broader evaluations should vary traffic geography and deployment hardware, and extend the binary selection action to include horizontal scaling.

7.4 Concluding perspective

The contribution of this thesis is a measurement-grounded framework for making additional network information available to control, and an empirical account of its utility and limits. Energy-aware UPF selection demonstrates how attributed, predictive and counterfactual estimates can be combined into a common decision process. The evaluated 5G case provides a basis for extending augmented observability toward more autonomous mobile networks, including future 6G settings, provided that the validity of the inferred information is assessed alongside the performance of the controller that uses it. The associated work on digital twins for sustainable multimedia delivery [7] places this direction within a broader 6G edge architecture linking user-plane orchestration with communication and compute resources.

Acknowledgements

I would like to thank my supervisor, Professor Raffaele Bolla, and my co-supervisor, Roberto Bruschi, for their guidance throughout this doctoral research. Their supervision helped me connect the study of network systems with the practical questions of measurement, modelling and control that shape this thesis.

I am grateful to Chiara Lombardo and Cristina Emilia Costa for their collaboration on the research reported in the associated papers. I also thank my colleagues at DITEN, University of Genoa, and the CNIT S2N National Laboratory for the research environment in which this work developed.

The research reported in the associated manuscripts was supported in part by the European Union's Horizon Europe programme through the SUNRISE-6G project (Grant Agreement No. 101139075), and in part by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, Mission 4, Component 2, Investment 1.3, CUP F83C22001690001, partnership on "Telecommunications of the Future" (PE00000001, RESTART). I gratefully acknowledge this support.

Publication record

The following conference papers and manuscripts are associated with this research.

Conference papers

1. S. Afshar Borji, R. Bruschi, C. E. Costa, and C. Lombardo, "Dynamic Energy-Efficient User Plane Function Selection in 5G Networks," *IFIP Networking 2025, SLICES Workshop*, Limassol, Cyprus, 26–30 May 2025, pp. 91–96. Published paper.

   This paper establishes the earlier forecast-assisted UPF selection setting.

2. C. E. Costa, S. Afshar Borji, N. Sobanapuram Muruganandame, S. Pramanik, and F. Granelli, "Digital Twins for Sustainable Multimedia Delivery in 6G Edge Systems," *Workshop on Next-Generation Multimedia Services at the Edge: Leveraging 5G and Beyond (NGMSE 2026)*, Vilamoura, Algarve, Portugal, 2026. Open version on Zenodo, doi:10.5281/zenodo.20844686.

   This paper provides the broader 6G digital-twin context of the thesis.

Journal manuscript

3. S. Afshar Borji, R. Bruschi, C. Lombardo, R. Bolla, and C. E. Costa, "AI-Driven Energy-Efficient Orchestration of User Plane Functions in 5G Core Networks," manuscript for *Computer Communications*, 2026 (manuscript COMCOM-S-26-00430).

   Its profiling, digital-twin and reinforcement-learning methods relate to Chapters 3, 5, and 6.

Conference manuscript under review

4. S. Afshar Borji, R. Bruschi, C. Lombardo, and R. Bolla, "Evaluating Centralized Value Learning in Shared-Policy MARL for Energy-Aware Multi-Site 5G UPF Selection," submitted to *IEEE WCNC 2027*.

   This manuscript studies shared policies and critic designs across sites (Chapter 6).

Bibliography

[1] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in action," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 3, no. 2, pp. 1–26, 2018.

[2] 3GPP, "Management and orchestration; 5g end-to-end key performance indicators (KPI)," 3rd Generation Partnership Project, Tech. Rep. TS 28.554, Release 18, V18.7.0, 2025.

[3] ——, "Management and orchestration; management data analytics (MDA)," 3rd Generation Partnership Project, Tech. Rep. TS 28.104, Release 18, V18.7.0, 2025.

[4] ——, "Architecture enhancements for 5g system (5GS) to support network data analytics services," 3rd Generation Partnership Project, Tech. Rep. TS 23.288, Release 18, V18.6.0, 2024.

[5] ITU-T, "Digital twin network – requirements and architecture," International Telecommunication Union, Tech. Rep. Recommendation Y.3090, 2022.

[6] ETSI, "Zero-touch network and service management (ZSM); closed-loop automation; part 1: Enablers," European Telecommunications Standards Institute, Tech. Rep. GS ZSM 009-1 V1.1.1, 2021.

[7] C. E. Costa, S. Afshar Borji, N. Sobanapuram Muruganandame, S. Pramanik, and F. Granelli, "Digital twins for sustainable multimedia delivery in 6G edge systems," in *Workshop on Next-Generation Multimedia Services at the Edge: Leveraging 5G and Beyond (NGMSE 2026)*, Vilamoura, Algarve, Portugal, 2026, open version deposited on Zenodo. [Online]. Available: https://doi.org/10.5281/zenodo.20844686

[8] M. Jay, V. Ostapenco, L. Lefèvre, D. Trystram, A.-C. Orgerie, and B. Fichel, "An experimental comparison of software-based power meters: focus on CPU and GPU," in *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023, pp. 106–118.

[9] M. Colmant, M. Kurpicz-Briki, P. Felber, L. Huertas, R. Rouvoy, and A. Sobe, "Process-level power estimation in VM-based systems," in *European Conference on Computer Systems (EuroSys)*, 2015, pp. 1–14.

[10] G. Fieni, R. Rouvoy, and L. Seinturier, "SmartWatts: Self-calibrating software-defined power meter for containers," in *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2020, pp. 479–488.

[11] M. Amaral, H. Chen, T. Chiba, R. Nakazawa, S. Choochotkaew, E. K. Lee, and T. Eilam, "Kepler: A framework to calculate the energy consumption of containerized applications," in *IEEE International Conference on Cloud Computing (CLOUD)*, 2023, pp. 69–71.

[12] H. Hè, M. Friedman, and T. Rekatsinas, "EnergAt: Fine-grained energy attribution for multi-tenancy," in *ACM Workshop on Sustainable Computer Systems (HotCarbon)*, 2023, pp. 1–8.

[13] M. Colmant, R. Rouvoy, M. Kurpicz-Briki, A. Sobe, P. Felber, and L. Seinturier, "The next 700 CPU power models," *Journal of Systems and Software*, vol. 144, pp. 382–396, 2018.

[14] A. Bellin, F. Granelli, and D. Munaretto, "A measurement-based approach to analyze the power consumption of the softwarized 5g core," *Computer Networks*, vol. 244, p. 110312, 2024.

[15] A. Bellin, M. Centenaro, and F. Granelli, "A preliminary study on the power consumption of virtualized edge 5g core networks," in *IEEE International Conference on Network Softwarization (NetSoft)*, 2023, pp. 420–425.

[16] P. Kadlec, B. Gabrys, and S. Strandt, "Data-driven soft sensors in the process industry," *Computers & Chemical Engineering*, vol. 33, no. 4, pp. 795–814, 2009.

[17] M. Kano and K. Fujiwara, "Virtual sensing technology in process industries: Trends and challenges revealed by recent industrial applications," *Journal of Chemical Engineering of Japan*, vol. 46, no. 1, pp. 1–17, 2013.

[18] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," *Journal of Global Optimization*, vol. 13, no. 4, pp. 455–492, 1998.

[19] Y. Jin, "Surrogate-assisted evolutionary computation: Recent advances and future challenges," *Swarm and Evolutionary Computation*, vol. 1, no. 2, pp. 61–70, 2011.

[20] P. Almasan, M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, D. Perino, D. López, A. A. Pastor Perales, P. Harvey, L. Ciavaglia, L. Wong, V. Ram, S. Xiao, X. Shi, X. Cheng, A. Cabellos-Aparicio, and P. Barlet-Ros, "Network digital twin: Context, enabling technologies, and opportunities," *IEEE Communications Magazine*, vol. 60, no. 11, pp. 22–27, 2022.

[21] A. Hakiri, A. Gokhale, S. Ben Yahia, and N. Mellouli, "A comprehensive survey on digital twin for future networks and emerging internet of things industry," *Computer Networks*, vol. 244, p. 110350, 2024.

[22] X. Wang, Z. Zhou, F. Xiao, K. Xing, Z. Yang, Y. Liu, and C. Peng, "Spatio-temporal analysis and prediction of cellular traffic in metropolis," *IEEE Transactions on Mobile Computing*, vol. 18, no. 9, pp. 2190–2202, 2019.

[23] C. Zhang, H. Zhang, D. Yuan, and M. Zhang, "Citywide cellular traffic prediction based on densely connected convolutional neural networks," *IEEE Communications Letters*, vol. 22, no. 8, pp. 1656–1659, 2018.

[24] W. Jiang, "Cellular traffic prediction with machine learning: A survey," *Expert Systems with Applications*, vol. 201, p. 117163, 2022.

[25] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *International Conference on Learning Representations (ICLR)*, 2018.

[26] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.

[27] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[28] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *AAAI Conference on Artificial Intelligence*, 2023.

[29] X. Zuo, M. Höweler, D. You, M. Reisslein, and F. H. P. Fitzek, "X-MAN: A non-intrusive power manager for energy-adaptive cloud-native network functions," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1017–1035, 2022.

[30] M. S. Q. Z. Nine, T. Kosar, M. F. Bulut, and J. Hwang, "GreenNFV: Energy-efficient network function virtualization with service level agreement constraints," in *ACM Symposium on Cloud Computing (SoCC)*, 2023, pp. 1–12.

[31] Á. Cañete, M. Amor, and L. Fuentes, "HADES: An NFV solution for energy-efficient placement and resource allocation in heterogeneous infrastructures," *Journal of Network and Computer Applications*, vol. 221, p. 103764, 2024.

[32] R. Bolla, R. Bruschi, F. Davoli, and J. F. Pajo, "A model-based approach towards real-time analytics in NFV infrastructures," *IEEE Transactions on Green Communications and Networking*, vol. 4, no. 2, pp. 529–541, 2020.

[33] R. Bolla, R. Bruschi, D. Cassinelli, F. Davoli, C. Lombardo, N. S. Martinelli, and B. Siccardi, "Performance and power analysis of a 5g UPF varying the c-states of the hosting server," in *IEEE International Conference on Communications Workshops*, 2025.

[34] R. Bolla, R. Bruschi, F. Davoli, C. Lombardo, and N. S. Martinelli, "Analyzing the power consumption in cloud-native 5/6g ecosystems," in *IEEE International Conference on Communications Workshops*, 2023, pp. 611–617.

[35] J. Rischke, C. L. Vielhaus, P. Sossalla, J. Wang, and F. H. P. Fitzek, "Comparison of UPF acceleration technologies and their tail-latency for URLLC," in *IEEE Conference on Network Function Virtualization and Software Defined Networks*, 2022, pp. 19–25.

[36] M. Fuentes-Gonzales, M. Catalan-Cid, and D. Camps-Mur, "Energy-efficient CPU resource management in DPDK-based UPFs: Experimental insights and strategies," *IEEE Open Journal of the Communications Society*, vol. 7, pp. 4731–4746, 2026.

[37] ETSI, "Zero-touch network and service management (ZSM); reference architecture," European Telecommunications Standards Institute, Tech. Rep. GS ZSM 002 V1.1.1, 2019.

[38] M. Akbari, R. Bolla, R. Bruschi, C. Lombardo, N. S. Martinelli, and B. Siccardi, "A management data analytics function for ethical 6g networks," *Computer Networks*, vol. 270, p. 111487, 2025.

[39] W. Zhao, J. Pena Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *IEEE Symposium Series on Computational Intelligence*, 2020, pp. 737–744.

[40] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar, "A deep reinforcement learning perspective on internet congestion control," in *International Conference on Machine Learning (ICML)*, 2019.

[41] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with Pensieve," in *ACM SIGCOMM*, 2017, pp. 197–210.

[42] F. Y. Yan, J. Ma, G. D. Hill, D. Raghavan, R. S. Wahby, P. Levis, and K. Winstein, "Pantheon: The training ground for internet congestion-control research," in *USENIX Annual Technical Conference (ATC)*, 2018.

[43] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[44] Z. Zhu, K. Lin, A. K. Jain, and J. Zhou, "Transfer learning in deep reinforcement learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 13 344–13 362, 2023.

[45] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, and D. I. Kim, "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019.

[46] ITU-T, "Energy efficiency metrics and measurement methods for telecommunication equipment," International Telecommunication Union, Tech. Rep. Recommendation L.1310, 2024.

[47] ETSI, "Environmental engineering; assessment of mobile network energy efficiency," European Telecommunications Standards Institute, Tech. Rep. ES 203 228 V1.4.1, 2022.

[48] R. M. Assunção, M. C. Neves, G. Câmara, and C. da Costa Freitas, "Efficient regionalization techniques for socio-economic geographical units using minimum spanning trees," *International Journal of Geographical Information Science*, vol. 20, no. 7, pp. 797–811, 2006.

[49] S. Afshar Borji, R. Bruschi, C. E. Costa, and C. Lombardo, "Dynamic energy-efficient user plane function selection in 5G networks," in *IFIP Networking 2025, SLICES Workshop*, Limassol, Cyprus, 2025, pp. 91–96.

[50] S. Afshar Borji, R. Bruschi, C. Lombardo, and R. Bolla, "Shared policies and centralized critics for energy-aware multi-site 5G UPF orchestration," 2026, submitted to IEEE WCNC 2027.

Advanced AI Tools for Augmented Observability in the 5/6G Environment

Shima Afshar Borji